

Executive Summary

Context and Objective

This study evaluates the **Burt matrix representation** as an alternative to classical one-hot dummy encoding of categorical rating factors in motor insurance pricing. The benchmark dataset is *freMTPL2freq* / *freMTPL2sev* (Dutang & Charpentier, *CASdatasets*), a French motor third-party liability portfolio of **678,013 policy-years** observed over approximately one year, with monetary amounts in euros.

The pipeline has three distinct objectives pursued simultaneously:

1. **Pricing optimisation** — a Tweedie GLM fitted with **glum** (Gonzalvo et al.), the high-performance coordinate-descent GLM library designed for large-scale insurance rating, with L2 regularisation and soft GMM segment memberships as additional covariates.
2. **Aggregate risk measurement** — the aggregate loss distribution is computed *analytically* via the Panjer / FFT recursive method using the **aggregate** package, not by Monte Carlo simulation.
3. **Representation benchmarking** — 21 controlled experiments across three representation families (A-series: classical, B-series: Burt-replacing, C-series: Burt-augmented) and four clustering methods, with BIC-driven model selection and a full governance and QA layer.

Methods

The Burt matrix $B = Z^T Z$, where Z is the complete indicator matrix of categorical modalities (Area, VehBrand, VehGas), embeds each policy into a continuous latent space encoding the co-occurrence geometry of its categorical and numerical rating factors. This representation is normalised and PCA-stabilised to 95% explained variance before GMM fitting. The classical representation uses standard one-hot dummy encoding. The Burt-augmented representation retains both dummies and Burt coordinates in the Tweedie design, testing whether the two signal types are complementary.

All GMM models use full covariance and are fitted with $n = 5$ random initialisations. The optimal number of components k is selected independently per representation family by BIC on the enriched geometry. Classical BIC selects $k = 7$; Burt BIC selects $k = 8$.

Principal Results

Experiment	Representation	Gini	Δ vs A0	Deviance/exp	Top-decile	TVaR 99%	Runtime (s)
A5 — Classical GMM enriched $k=7$	Classical	0.4241	+0.0552	61.276	1.139	61,618,566	~260
A3 — Classical GMM enriched $k=5$	Classical	0.4227	+0.0539	61.290	1.090	—	~200
B5 — Burt GMM enriched $k=8$	Burt-replacing	0.3796	+0.0107	61.336	1.032	61,533,968	~656
C3 — Burt-augmented GMM enriched $k=5$	Burt-augmented	0.3815	+0.0127	61.290	1.039	—	~373
A0 — Classical plain GLM	Classical	0.3688	0.0000	61.691	1.032	—	~4
A_ext — Classical K-means enriched $k=5$	Classical	0.3596	-0.0092	61.300	1.029	—	~15

Key Findings

1. Discrimination. The classical enriched GMM (A5) achieves a Gini of 0.4241, improving 5.5 Gini points over the plain GLM baseline (A0). The best-calibrated Burt experiment (B5) reaches 0.3796 — 4.5 Gini points below A5. This gap is not a

failure of the Burt representation but a property of the Tweedie GLM, which is architecturally optimised for sharp dummy contrasts between modality levels. A non-linear pricing model (gradient boosting, neural network) would likely narrow or close this gap.

2. Deviance. Following the recommendation by Hainaut & Denuit (Detra Note 2025-2), Tweedie deviance per exposure is added as a second fit-quality metric. A5 achieves the lowest deviance (61.276) across all 21 experiments — the same experiment that wins on Gini. Deviance and Gini are aligned within the A-series: both improve as segmentation quality increases. However, the overall cross-series correlation is weak ($r = -0.38$, $p = 0.09$) and not statistically significant. The B and C-series cluster at competitive deviance values despite substantially lower Gini, suggesting that the Burt representation fits the loss distribution well in absolute terms while redistributing risk ranks differently from classical encoding. The top-decile calibration ratio is a genuinely orthogonal dimension that neither Gini nor deviance captures.

3. Tail calibration and capital. The best-calibrated experiment, selected by minimum absolute deviation of the top-decile ratio from 1.0 among A and B series GMM experiments, is **B5** (Burt GMM enriched $k=8$, BIC-confirmed, top-decile 1.032) versus A5's 1.139 — a 10.7 percentage point improvement. The analytic aggregate confirms this: B5 produces a TVaR at 99% of €61,533,968 versus €61,618,566 for A5 — a capital saving of approximately €85,000. A5 fails the aggregate validator on severity mean and aggregate mean; B5 passes validation cleanly.

4. Informational completeness and C-series exclusion. The Burt-augmented C-series produces results virtually identical to the pure Burt B-series: C3 (0.3815) and B3 (0.3816) differ by only 0.0001 Gini, and deviance values match to three decimal places. The Burt coordinates fully encode the information carried by the original categorical dummies — adding them back recovers nothing. The C-series is therefore excluded from the canonical comparison on two grounds: informational completeness and BIC instability from collinearity in the joint clustering geometry.

5. Joint clustering geometry. C5_joint matches the best Burt experiments on Gini but the BIC curve for the joint space remains non-monotone — a sign of latent collinearity instability. The joint geometry adds no benefit over pure Burt clustering.

6. Computational scalability. B5 (k=8, BIC-confirmed) requires approximately 656 seconds — prohibitive for monthly production refresh. A5 at ~260 seconds is moderate. For production use under a calibration constraint, B3 (k=5, ~370 seconds, top-decile 1.039) offers an acceptable compromise.

7. Hard clustering anomaly. A_ext (K-means on enriched classical geometry) is the only experiment to score below the plain GLM baseline (−0.0092 Gini). Soft GMM is substantially more robust than hard K-means on enriched geometries.

Actuarial Recommendation

Objective	Recommended model	Rationale
Pricing discrimination	A5	Best Gini and best deviance — strongest fit quality
Regulatory / capital model	B5	Best-calibrated by top-decile distance, passes aggregate validation, TVaR 99% lower by €85k
Production (runtime constraint)	B3	Well-calibrated (top-decile 1.039), ~370s, acceptable compromise
Future high-cardinality or constrained markets	Burt-based	Dimensionality reduction, portfolio robustness

The Burt representation is validated as an effective actuarial segmentation tool with demonstrated advantages in tail calibration and a clear research roadmap toward contexts where it is expected to outperform classical dummy encoding.

Methodology

1. Dataset

The dataset is the French motor third-party liability portfolio **freMTPL2freq** and **freMTPL2sev** from the *CASdatasets* package, widely used as an actuarial benchmark. It contains approximately 678,000 policy-years with observed claim counts and, for claims that occurred, individual paid loss amounts.

2. Raw Features

Each policy is described by two types of rating factors:

- **Categorical factors:** Area (6 levels), VehBrand (11 levels), VehGas (2 levels). These are the traditional tariff segmentation variables.
- **Numerical factors:** VehAge, DrivAge, BonusMalus, Density, Exposure. Continuous or ordinal variables used after normalisation.

Target variables: claim count (frequency), average claim severity, and pure premium = frequency $\times$ severity.

3. The Three Representation Families

A central question of this study is how the encoding of categorical rating factors affects pricing discrimination. Three families are evaluated:

- **A — Classical.** Categorical factors are one-hot encoded as dummy variables. Numerical factors are used after standardisation. This is the standard actuarial approach.
- **B — Burt-replacing.** The categorical dummies are *replaced* by Burt latent coordinates. The Burt matrix $B = Z^T Z$ (where Z is the complete indicator matrix of all categorical modalities) encodes the co-occurrence structure of the categorical rating factors across policies. The normalised Burt coordinates embed each policy into a continuous latent space that captures the geometry of modality co-occurrence. Numerical factors are retained unchanged.
- **C — Burt-augmented.** The Burt latent coordinates are *added* to the classical feature set — the original categorical dummies are retained alongside the Burt coordinates in the Tweedie pricing regression. This tests whether the two signal types (categorical contrast and Burt co-occurrence geometry) are complementary rather than redundant. Numerical factors are retained unchanged.

4. Actuarial Enrichment

Beyond the raw rating factors, a baseline Tweedie GLM is fitted first (Sections 2–3) to produce two model-derived risk scores for each policy:

- **Predicted frequency** — expected claim count per unit exposure.

- **Predicted severity** — expected average claim cost given a claim.

These scores are used as additional clustering features in the *enriched* geometry experiments (A_ext, A5, B_ext, B5). They align the risk segments directly with the pricing signal rather than relying solely on rating-factor structure.

5. Clustering Methods

Each representation family is evaluated with four clustering approaches:

- **None (baseline)** — plain Tweedie GLM, no segmentation (A0, B0).
- **K-means simple** — hard cluster assignment on the 2D risk-score geometry (log predicted frequency + log predicted severity) (A1, B1).
- **K-means enriched** — hard cluster assignment on rating factors plus predicted frequency and severity (A_ext, B_ext).
- **GMM simple** — soft probabilistic membership on the 2D risk-score geometry, k=5 fixed (A2, B2).
- **GMM enriched k=5** — soft probabilistic membership on the full enriched geometry, k=5 fixed for cross-series comparability (A3, B3).
- **GMM enriched BIC-optimal k** — soft probabilistic membership on the full enriched geometry, k selected independently per series by BIC (A5, B5).

Soft GMM memberships are passed as additional covariates into the Tweedie pricing regression, allowing the model to blend segment-level risk adjustments continuously rather than applying hard regime switches.

6. Determination of k

- **K-means:** elbow diagnostic on within-cluster inertia over k = 2–10. The elbow point is used as the fixed k for all K-means experiments. k=5 is used as a round number across series.
- **GMM BIC sweep:** run on the *enriched* geometry (rating factors + predicted frequency + predicted severity), using full covariance and n_init=5 for stable, smooth BIC curves. The sweep is run independently for each representation family because BIC values are not comparable across spaces. Classical k grid: 2–8. Burt k grid: 2–10 (with PCA

stabilisation). Results are cached to disk; subsequent runs skip the sweep entirely.

- **Fixed-k experiments:** A5 uses $k=7$ and B5 uses $k=8$ as canonical fixed benchmarks. If the BIC sweep selects the same k , the experiment is marked `selection_rule = fixed_and_bic_selected` rather than creating a duplicate row.

Methodological note — PCA sensitivity and covariance type. Full-covariance GMM on the 61-dimensional Burt enriched feature space is numerically unstable without dimensionality reduction: a pathological EM convergence was observed at $k=6$ with a runtime exceeding three hours. PCA is therefore applied before GMM fitting for all Burt-based enriched experiments, retaining the minimum number of components that explain 95% of cumulative variance (44 components in practice). The same PCA transformation is applied in both the BIC sweep and pricing experiments, ensuring consistency between model selection and estimation geometry.

This introduces a sensitivity to the variance threshold choice: different thresholds yield different PCA geometries, different BIC-optimal k values, and different Gini and calibration outcomes. The 95% threshold is a conventional choice with no strong actuarial justification over 90% or 97%.

The covariance type is configurable via `GMM_COVARIANCE_TYPE` at the top of Cell 5.2. Changing it to `"diag"` (diagonal covariance, no inter-feature correlations) or `"spherical"` (isotropic — equivalent to soft k-means) eliminates the PCA dependency entirely and may produce more stable results at the cost of modelling flexibility. A full sensitivity test was conducted with `"diag"` (diagonal covariance, no inter-feature correlations). The results are conclusive:

For the **classical** enriched geometry, diagonal covariance selects the same $k=7$ and produces identical Gini and calibration figures to four decimal places. Covariance structure does not affect classical results.

For the **Burt** enriched geometry, diagonal covariance produces a monotonically decreasing BIC curve with no interior minimum within $k=2-10$ — a boundary problem. The BIC penalty is too weak when inter-feature correlations are ignored. Full covariance with PCA stabilisation remains the only setting that finds a genuine interior BIC minimum ($k=8$) for the Burt enriched geometry.

For experiments with fixed $k=5$ (B3, C3), pricing results are identical under both covariance types. For BIC-selected experiments, diagonal covariance selects $k=10$

(boundary) and produces marginally worse results. **Full covariance with PCA stabilisation at 95% variance is the correct architectural choice.** Diagonal covariance does not help and introduces a k-selection failure for the Burt geometry.

7. Pricing Model

All experiments use a **Tweedie GLM** with a log link as the pricing model. The design matrix includes the representation features (classical dummies or Burt coordinates) plus soft GMM memberships where applicable. A ridge regularisation term ($\alpha > 0$) controls multicollinearity. A global loss calibration constraint ensures total predicted premium matches observed losses.

8. Evaluation KPIs

- **Gini coefficient** — primary discrimination metric. Higher is better.
- **Top-decile actual / predicted ratio** — calibration at the high-risk tail. Ideal value is 1.0.
- **Loss calibration** — ratio of total predicted to total observed premium. Enforced to 1.0 by construction.
- **Runtime** — wall-clock seconds, reported for reproducibility.

9. Experiment Plan

The full experiment matrix is shown below. The k and Gini columns are filled in progressively as the notebook runs.

ID	Series	Clustering Geometry		k	Selection rule	Gini
A0	Classical	None	—	—	fixed	—
A1	Classical	K-means	Simple	5	fixed	—
A2	Classical	GMM	Simple	5	fixed	—
A3	Classical	GMM	Enriched	5	fixed	—
A_ext	Classical	K-means	Enriched	5	fixed	—
A5	Classical	GMM	Enriched	7	fixed / BIC	—

ID	Series	Clustering Geometry		k	Selection rule	Gini
B0	Burt-replacing	None	—	—	fixed	—
B1	Burt-replacing	K-means	Simple	5	fixed	—
B2	Burt-replacing	GMM	Simple	5	fixed	—
B3	Burt-replacing	GMM	Enriched	5	fixed	—
B_ext	Burt-replacing	K-means	Enriched	5	fixed	—
B5	Burt-replacing	GMM	Enriched	8	fixed / BIC	—
C0	Burt-augmented	None	—	—	fixed	—
C1	Burt-augmented	K-means	Simple	5	fixed	—
C2	Burt-augmented	GMM	Simple	5	fixed	—
C3	Burt-augmented	GMM	Enriched	5	fixed	—
C_ext	Burt-augmented	K-means	Enriched	5	fixed	—
C5	Burt-augmented	GMM	Enriched	8	fixed / BIC	—
C3_joint	Burt-augmented	GMM	Joint	5	fixed	—
C_ext_joint	Burt-augmented	K-means	Joint	5	fixed	—
C5_joint	Burt-augmented	GMM	Joint	BIC	bic_selected	—

Table of Contents

- Section 1 — Data and Representation** — environment setup, data cleaning, Burt representation layer.

2. **Section 2 — Frequency Modeling** — Poisson GLM with exposure weighting, calibration and dispersion diagnostics.
3. **Section 3 — Severity Modeling** — severity data preparation, distribution screening, moment-matching re-fitting.
4. **Section 4 — Modeling Framework** — reusable helpers for clustering, feature construction, Tweedie pricing, and evaluation.
5. **Section 5 — Benchmark Experiments** — K-means elbow diagnostic; two-way BIC/AIC sweep (classical, Burt) with disk cache; full A, B, and C series pricing experiments (A0–A5, B0–B5, C0–C5 + C_joint variants). Three-way BIC sweep including joint feature space.
6. **Section 6 — Model Selection and Results** — BIC confirmation, selection_rule tagging, final benchmark summary with Gini and KPIs.
7. **Section 7 — Aggregate Validation** — loss ratio and lift chart validation for the selected best experiment.
8. **Section 8 — Scenario Analysis** — deductible and per-claim limit sensitivity; Monte Carlo validation.
9. **Section 9 — QA and Regression** — regression baseline and governance checks.
10. **Section 10 — Reporting** — consolidated executive, diagnostic, and audit views.
11. **Future Research** — Joint BIC sweep for C-series joint feature space, raw Burt-distance clustering, mixed feature geometry, severity-informed Burt representation.

Section 1 — Data and Representation

This section loads and validates the freMTPL2 dataset, engineers the feature space, and constructs the Burt latent representation. Two parallel policy tables are produced: a classical table using one-hot categorical encoding, and a Burt-enriched table replacing categorical dummies with normalised co-occurrence coordinates. All downstream sections consume these two representations.

```
In [1]: # =====  
# Section: 1  
# Cell: 1.1  
# Title: Environment, Package & Data Validation  
# Version: 1.0.0
```

```

# Date: 2026-05-06
# Description:
# Verify the Python environment, required actuarial libraries,
# and local freMTPL data cache. Missing packages can be installed
# only after explicit confirmation.
# =====

import sys
import platform
import importlib
import subprocess
from importlib import metadata
from pathlib import Path

import pandas as pd

# -----
# 0.1 – Package Verification
# -----

def _module_name(package_name: str) -> str:
    """Map pip/distribution names to import names."""
    return {
        "scikit-learn": "sklearn",
    }.get(package_name, package_name)

def _package_version(package_name: str) -> str:
    """Return the installed package version when available."""
    try:
        return metadata.version(package_name)
    except metadata.PackageNotFoundError:
        # Some packages expose a different distribution name.
        try:
            mod = importlib.import_module(_module_name(package_name))
            return getattr(mod, "__version__", "version unknown")
        except Exception:
            return "not installed"

def verify_python_environment():
    print("\n--- 0.1: Python Environment Check ---")
    print(f"Python executable : {sys.executable}")
    print(f"Python version    : {sys.version.split()[0]}")
    print(f"Platform          : {platform.platform()}")

    if sys.version_info < (3, 10):
        raise RuntimeError("Python 3.10 or higher is recommended for this notebook")

    requirements = [
        "numpy",
        "pandas",
        "matplotlib",
        "scipy",
        "scikit-learn",
    ]

```

```

        "glum",
        "aggregate",
        "lightgbm",
    ]

    missing = []

    print("\nRequired package status:")
    for pkg in requirements:
        import_name = _module_name(pkg)
        try:
            importlib.import_module(import_name)
            print(f"✅ {pkg:<14} {_package_version(pkg)}")
        except ImportError:
            print(f"❌ {pkg:<14} missing")
            missing.append(pkg)

    if missing:
        print(f"\nMissing packages: {' '.join(missing)}")
        user_input = input("Install missing packages now via pip? (y/n): ")

        if user_input == "y":
            subprocess.check_call([sys.executable, "-m", "pip", "install", *
            print("Installation complete. Restarting the kernel is recommended")
        else:
            raise RuntimeError(
                "Required packages missing. Aborting to ensure reproducibility"
            )

    print("\n✅ Environment OK.")

# -----
# 0.2 – Data Directory and Dataset Validation
# -----

def load_or_fetch_fremtpl_data():
    print("\n--- 0.2: freMTPL Data Validation ---")

    data_dir = Path("data")
    data_dir.mkdir(parents=True, exist_ok=True)

    freq_path = data_dir / "freMTPL2freq.csv"
    sev_path = data_dir / "freMTPL2sev.csv"

    if freq_path.exists() and sev_path.exists():
        print("✅ Datasets found in local cache.")
    else:
        from sklearn.datasets import fetch_openml

        print("Fetching datasets from OpenML; this is only needed on the first run")
        freq_data = fetch_openml("freMTPL2freq", as_frame=True, parser="auto")
        sev_data = fetch_openml("freMTPL2sev", as_frame=True, parser="auto")

        freq_data.to_csv(freq_path, index=False)
        sev_data.to_csv(sev_path, index=False)

```

```

        print("Datasets cached locally.")

    if freq_path.stat().st_size < 1_000:
        raise ValueError(f"Frequency file looks suspiciously small: {freq_pa

    if sev_path.stat().st_size < 1_000:
        raise ValueError(f"Severity file looks suspiciously small: {sev_path

    freq_df = pd.read_csv(freq_path)
    sev_df = pd.read_csv(sev_path)

    required_freq_cols = {
        "IDpol", "ClaimNb", "Exposure", "Area", "VehPower", "VehAge",
        "DrivAge", "BonusMalus", "VehBrand", "VehGas", "Density", "Region"
    }
    required_sev_cols = {"IDpol", "ClaimAmount"}

    missing_freq = required_freq_cols.difference(freq_df.columns)
    missing_sev = required_sev_cols.difference(sev_df.columns)

    if missing_freq:
        raise ValueError(f"Frequency dataset missing columns: {sorted(missir

    if missing_sev:
        raise ValueError(f"Severity dataset missing columns: {sorted(missing

    print(f"Frequency data : {freq_df.shape[0]:,} rows × {freq_df.shape[1]:,}
    print(f"Severity data : {sev_df.shape[0]:,} rows × {sev_df.shape[1]:,}
    print("✓ Data ready.")

    return freq_df, sev_df

# -----
# Execution
# -----

verify_python_environment()
freq, sev = load_or_fetch_fremtpl_data()

print("\nSystem ready for actuarial modeling.")

```

```
--- 0.1: Python Environment Check ---
Python executable : C:\Tools\anaconda3\python.exe
Python version    : 3.13.5
Platform          : Windows-11-10.0.26200-SP0
```

Required package status:

```
✓ numpy      2.1.3
✓ pandas     2.2.3
✓ matplotlib 3.10.0
✓ scipy      1.15.3
✓ scikit-learn 1.6.1
✓ glum       3.3.0
✓ aggregate  0.30.0
✓ lightgbm   4.6.0
```

✓ Environment OK.

```
--- 0.2: freMTPL Data Validation ---
```

```
✓ Datasets found in local cache.
Frequency data : 678,013 rows × 12 columns
Severity data  : 26,639 rows × 2 columns
✓ Data ready.
```

System ready for actuarial modeling.

```
In [2]: # =====
# Section: 1
# Cell: 1.2
# Title: Data Engineering – Cleaning & Normalization
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Updated executable notebook cell.
# =====

# =====
# Section: 0
# Cell: 0.2
# Title: Data Engineering – Cleaning & Normalization
# Version: 1.1
# Date: 2026-04-22
# Description:
# Cleaning the frequency dataset, capping exposure/claims,
# and setting categorical dtypes for GLM performance.
# =====
import numpy as np

# 1. Capping Exposure & Claims (Actuarial Best Practice)
# We cap exposure at 1.0 (some rows have errors > 1)
# We cap ClaimNb at 4 to stabilize the Poisson gradient
freq_clean = freq.copy()

freq_clean['Exposure'] = freq_clean['Exposure'].clip(upper=1.0)
freq_clean['ClaimNb'] = freq_clean['ClaimNb'].clip(upper=4)

# 2. Type Casting for Native Categorical Handling
```

```

# Modern tools like 'glum' leverage pandas categorical types
cat_cols = ['Area', 'VehPower', 'VehBrand', 'VehGas', 'Region']
for col in cat_cols:
    freq_clean[col] = freq_clean[col].astype('category')

# 3. Quick Exposure-Weighted Sanity Check
total_claims = freq_clean['ClaimNb'].sum()
total_exposure = freq_clean['Exposure'].sum()
global_freq = total_claims / total_exposure

print(f"Dataset Cleaned.")
print(f"Total number of claims: {total_claims}")
print(f"Total Exposure: {total_exposure:,.2f}")
print(f"Global empiric frequency: {global_freq:.4%}")

# Preview types for 'glum' show later
print("\nCategorical columns check:")
print(freq_clean[cat_cols].dtypes)

```

```

Dataset Cleaned.
Total number of claims: 36056
Total Exposure: 358,360.11
Global empiric frequency: 10.0614%

```

```

Categorical columns check:
Area          category
VehPower      category
VehBrand      category
VehGas        category
Region        category
dtype: object

```

Burt Representation Layer

After housekeeping, the notebook now creates a second representation family. The classical policy table is preserved unchanged. A Burt-enriched policy table is created by converting selected categorical rating factors into normalized Burt latent coordinates. Downstream pricing and segmentation experiments can then compare classical geometry against Burt latent geometry without interrupting the original actuarial flow.

```

In [3]: # =====
# Section: 1
# Cell: 1.3
# Title: Burt Representation Layer – Modality Geometry and Policy Embeddings
# Version: 1.1.0
# Date: 2026-05-11
# Description:
# Construct a Burt representation immediately after housekeeping.
# Version 1.1.0: chunked float32 embedding to avoid MemoryError.

```

```

# The classical cleaned policy table is preserved, while a Burt-
# enriched policy table is created by appending normalized latent
# categorical coordinates. No claims information is used in this
# representation layer.
# =====

import numpy as np
import pandas as pd

# -----
# 1.3.1 – Representation Governance
# -----
# These are the categorical rating factors used to construct the
# Burt geometry in v1. VehPower is treated as categorical here
# because the current notebook already casts it as category for
# the frequency GLM.
BURT_CATEGORICAL_FEATURES = [
    "Area",
    "VehPower",
    "VehBrand",
    "VehGas",
    "Region",
]

missing_burt_features = [c for c in BURT_CATEGORICAL_FEATURES if c not in fr
if missing_burt_features:
    raise ValueError(f"Missing Burt categorical features: {missing_burt_feat

# Preserve the classical table explicitly. This prevents the Burt
# representation from silently changing the existing A-series workflow.
freq_clean_classical = freq_clean.copy()

# -----
# 1.3.2 – Super-Indicator Matrix D
# -----
# D has one row per policy and one column per modality. Each row has
# exactly q active entries if every selected categorical variable is present
burt_source = freq_clean_classical[BURT_CATEGORICAL_FEATURES].copy()
for col in BURT_CATEGORICAL_FEATURES:
    burt_source[col] = burt_source[col].astype("category")

D_burt = pd.get_dummies(
    burt_source,
    columns=BURT_CATEGORICAL_FEATURES,
    prefix=BURT_CATEGORICAL_FEATURES,
    dtype=float,
)

q_burt = len(BURT_CATEGORICAL_FEATURES)
n_policies, n_modalities = D_burt.shape
row_activity = D_burt.sum(axis=1)

if not np.allclose(row_activity.to_numpy(), q_burt):
    raise ValueError(
        "Each policy should activate exactly one modality per Burt variable.
        Check missing values or unexpected categorical encoding."
    )

```



```

    )

# -----
# 1.3.3 – Raw Burt Matrix  $B = D.T @ D$ 
# -----
# B counts modality co-occurrences across the full portfolio.
D_values = D_burt.to_numpy(dtype=float)
B_raw_values = D_values.T @ D_values
B_raw = pd.DataFrame(
    B_raw_values,
    index=D_burt.columns,
    columns=D_burt.columns,
)

# -----
# 1.3.4 – Weighted Burt Matrix  $B^W = (1/q) C B C$ 
# -----
# The diagonal of B contains modality frequencies. The C B C scaling
# prevents frequent modalities from dominating the latent geometry.
modality_counts = np.diag(B_raw_values).astype(float)
if np.any(modality_counts <= 0):
    empty_modalities = list(np.array(D_burt.columns)[modality_counts <= 0])
    raise ValueError(f"Empty Burt modalities encountered: {empty_modalities}")

C_scale = 1.0 / np.sqrt(modality_counts)
B_weighted_values = (B_raw_values * np.outer(C_scale, C_scale)) / q_burt
B_weighted = pd.DataFrame(
    B_weighted_values,
    index=D_burt.columns,
    columns=D_burt.columns,
)

# -----
# 1.3.5 – Policy Burt Embeddings  $X_{Burt} = D B^W / q$ 
# -----
# For each policy, matrix multiplication selects the rows of  $B^W$ 
# corresponding to its active modalities and sums them. Division by q
# converts this sum into the barycenter of the policy's modality vectors.
#
# Memory note: the result is stored as float32 (halving the footprint
# vs float64) and computed in chunks of 50,000 policies to avoid a
# single large intermediate allocation (~274 MiB at float64).
# Precision loss is in the 7th decimal place and irrelevant for
# clustering and pricing.
BURT_EMBEDDING_CHUNK_SIZE = 50_000
X_burt_values = np.empty((n_policies, n_modalities), dtype=np.float32)

for _chunk_start in range(0, n_policies, BURT_EMBEDDING_CHUNK_SIZE):
    _chunk_end = min(_chunk_start + BURT_EMBEDDING_CHUNK_SIZE, n_policies)
    X_burt_values[_chunk_start:_chunk_end] = (
        D_values[_chunk_start:_chunk_end] @ B_weighted_values
    ) / q_burt

print(f"Burt embeddings computed. Shape: {X_burt_values.shape}, dtype: {X_bu
BURT_FEATURE_COLUMNS = [f"burt_{j + 1:03d}" for j in range(n_modalities)]

```

```

BURT_COORDINATES = pd.DataFrame(
    X_burt_values,
    columns=BURT_FEATURE_COLUMNS,
    index=freq_clean_classical.index,
)
BURT_COORDINATES.insert(0, "IDpol", freq_clean_classical["IDpol"].to_numpy())

freq_clean_burt = pd.concat(
    [freq_clean_classical.reset_index(drop=True), BURT_COORDINATES[ BURT_FEA
    axis=1,
)

BURT_METADATA = {
    "n_policies": int(n_policies),
    "q_variables": int(q_burt),
    "n_modalities": int(n_modalities),
    "categorical_features": BURT_CATEGORICAL_FEATURES,
    "burt_feature_columns": BURT_FEATURE_COLUMNS,
}

burt_governance_table = pd.DataFrame({
    "item": [
        "policies",
        "categorical variables q",
        "modalities m",
        "B raw shape",
        "B weighted shape",
        "policy embedding shape",
    ],
    "value": [
        f"{n_policies:}",
        q_burt,
        n_modalities,
        str(B_raw.shape),
        str(B_weighted.shape),
        str(BURT_COORDINATES[ BURT_FEATURE_COLUMNS ].shape),
    ],
})

print("Burt representation created.")
print(f"Categorical variables q : {q_burt}")
print(f"Total modalities m      : {n_modalities}")
print(f"Policy embeddings        : {BURT_COORDINATES[ BURT_FEATURE_COLUMNS ].")
print("\nSelected variables:")
print(BURT_CATEGORICAL_FEATURES)

display(burt_governance_table)

```

```

Burt embeddings computed. Shape: (678013, 53), dtype: float32
Burt representation created.
Categorical variables q : 5
Total modalities m      : 53
Policy embeddings        : (678013, 53)

```

```

Selected variables:
['Area', 'VehPower', 'VehBrand', 'VehGas', 'Region']

```

	item	value
0	policies	678,013
1	categorical variables q	5
2	modalities m	53
3	B raw shape	(53, 53)
4	B weighted shape	(53, 53)
5	policy embedding shape	(678013, 53)

Section 2 — Frequency Backbone

The first actuarial layer is a transparent Poisson GLM for claim frequency. The response is claim frequency, while exposure is used as sample weight. This gives a stable frequency backbone that can later be used for latent segmentation and aggregate claim-count calibration.

The purpose is not to claim that a Poisson model is perfect. In motor insurance, overdispersion is expected. The point is to measure it explicitly and use the model as an interpretable frequency foundation.

```
In [4]: # =====
# Section: 2
# Cell: 2.1
# Title: Frequency Modeling – Poisson GLM with Exposure Weighting
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Fit a Poisson GLM in glum using the formula interface.
# The response is claim frequency ClaimNb / Exposure, and
# Exposure is used as sample weight. drop_first=True is used
# to ensure a structurally full-rank design matrix.
# =====

import numpy as np
import pandas as pd
from glum import GeneralizedLinearRegressor

DATA = freq_clean.copy()

# --- CATEGORICAL CASTING ---
for col in ["Area", "VehPower", "VehBrand", "VehGas", "Region"]:
    DATA[col] = DATA[col].astype("category")

# --- RESPONSE ---
```

```

DATA["ClaimFreq"] = DATA["ClaimNb"] / DATA["Exposure"]

# --- FORMULA ---
formula = """
ClaimFreq ~
    DrivAge +
    VehAge +
    BonusMalus +
    VehPower +
    Density +
    Area +
    VehBrand +
    VehGas +
    Region
"""

# --- MODEL ---
model = GeneralizedLinearRegressor(
    family="poisson",
    alpha=0.0,
    l1_ratio=0.0,
    fit_intercept=True,
    formula=formula,
    drop_first=True
)

# --- FIT ---
model.fit(
    DATA,
    sample_weight=DATA["Exposure"]
)
# Preserve the fitted frequency model under an explicit name
freq_model = model

# --- OUTPUT ---
coef = pd.Series(model.coef_, index=model.feature_names_)
print("Intercept:")
print(model.intercept_)

print("\nTop coefficients by absolute magnitude:")
print(coef.reindex(coef.abs().sort_values(ascending=False).index).head(20))

```

Intercept:
-4.012065919562374

Top coefficients by absolute magnitude:

VehPower[9]	0.290926
Region[R83]	-0.288379
Region[R41]	-0.251504
VehPower[10]	0.242763
Area[E]	0.225040
Area[F]	0.221885
VehBrand[B12]	0.189621
Area[D]	0.181441
VehPower[14]	0.180089
VehPower[11]	0.179576
Region[R74]	0.179399
VehPower[6]	0.176844
VehPower[5]	0.155946
Region[R21]	0.128627
Region[R73]	-0.127747
VehBrand[B11]	0.124771
VehBrand[B14]	-0.117797
Region[R31]	-0.115350
VehPower[7]	0.114318
Region[R94]	0.111838

dtype: float64

```
In [5]: # =====
# Section: 2
# Cell: 2.2
# Title: Frequency Model – Calibration and Dispersion Diagnostics
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Validate the Poisson frequency backbone using calibration by
# predicted-risk decile, deviance, and Pearson dispersion.
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Work on the merged/clean frequency frame before severity merge.
FREQ_DIAG = freq_clean.copy()
for col in ["Area", "VehPower", "VehBrand", "VehGas", "Region"]:
    FREQ_DIAG[col] = FREQ_DIAG[col].astype("category")

FREQ_DIAG["PredFreq"] = freq_model.predict(FREQ_DIAG)
FREQ_DIAG["PredDecile"] = pd.qcut(
    FREQ_DIAG["PredFreq"], q=10, labels=False, duplicates="drop"
) + 1

freq_calib = FREQ_DIAG.groupby("PredDecile", observed=True).agg(
    policies=("IDpol", "count"),
    exposure=("Exposure", "sum"),
    claims=("ClaimNb", "sum"),
    predicted_claims=("PredFreq", lambda x: np.sum(x * FREQ_DIAG.loc[x.index
```

```

)

freq_calib["observed_frequency"] = freq_calib["claims"] / freq_calib["exposure"]
freq_calib["predicted_frequency"] = freq_calib["predicted_claims"] / freq_calib["exposure"]
freq_calib["ratio_observed_predicted"] = freq_calib["observed_frequency"] / freq_calib["predicted_frequency"]

# Global deviance and Pearson dispersion
_y = FREQ_DIAG["ClaimNb"].to_numpy()
_mu = np.clip(FREQ_DIAG["PredFreq"].to_numpy() * FREQ_DIAG["Exposure"].to_numpy(), 1e-05, 1e+05)

positive = _y > 0
dev_contrib = np.empty_like(_mu, dtype=float)
dev_contrib[positive] = 2 * (_y[positive] * np.log(_y[positive] / _mu[positive]))
dev_contrib[~positive] = 2 * (_mu[~positive] * np.log(_mu[~positive] / _y[~positive]))

freq_deviance = dev_contrib.sum()
freq_pearson_chi2 = np.sum((_y - _mu) ** 2 / _mu)
freq_df = len(_y) - (len(freq_model.coef_) + 1)
freq_deviance_ratio = freq_deviance / freq_df
freq_pearson_dispersion = freq_pearson_chi2 / freq_df

print(freq_calib.round(6))
print("\nFrequency diagnostics")
print("-----")
print(f"Observed frequency      : {FREQ_DIAG['ClaimNb'].sum() / FREQ_DIAG['Exposure'].sum():.4f}")
print(f"Predicted frequency      : {(FREQ_DIAG['PredFreq'] * FREQ_DIAG['Exposure']).sum() / FREQ_DIAG['Exposure'].sum():.4f}")
print(f"Deviance / df            : {freq_deviance_ratio:.4f}")
print(f"Pearson dispersion       : {freq_pearson_dispersion:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(freq_calib.index, freq_calib["observed_frequency"], marker="o", label="Observed")
plt.plot(freq_calib.index, freq_calib["predicted_frequency"], marker="o", label="Predicted")
plt.xlabel("Predicted Frequency Decile")
plt.ylabel("Claim Frequency")
plt.title("Frequency Calibration by Predicted-Risk Decile")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

	policies	exposure	claims	predicted_claims \
PredDecile				
1	67802	43487.068359	2143	2211.975306
2	67801	42727.981418	2673	2760.475502
3	67810	41620.816316	3058	3075.482301
4	67792	40116.416852	3273	3304.911627
5	67802	38089.641828	3277	3461.267165
6	67801	35349.613566	3532	3534.742337
7	67803	32762.694619	3556	3623.682768
8	67799	30035.757396	3883	3738.900720
9	67801	27965.936780	4571	4142.625097
10	67802	26204.178329	6090	6202.088255

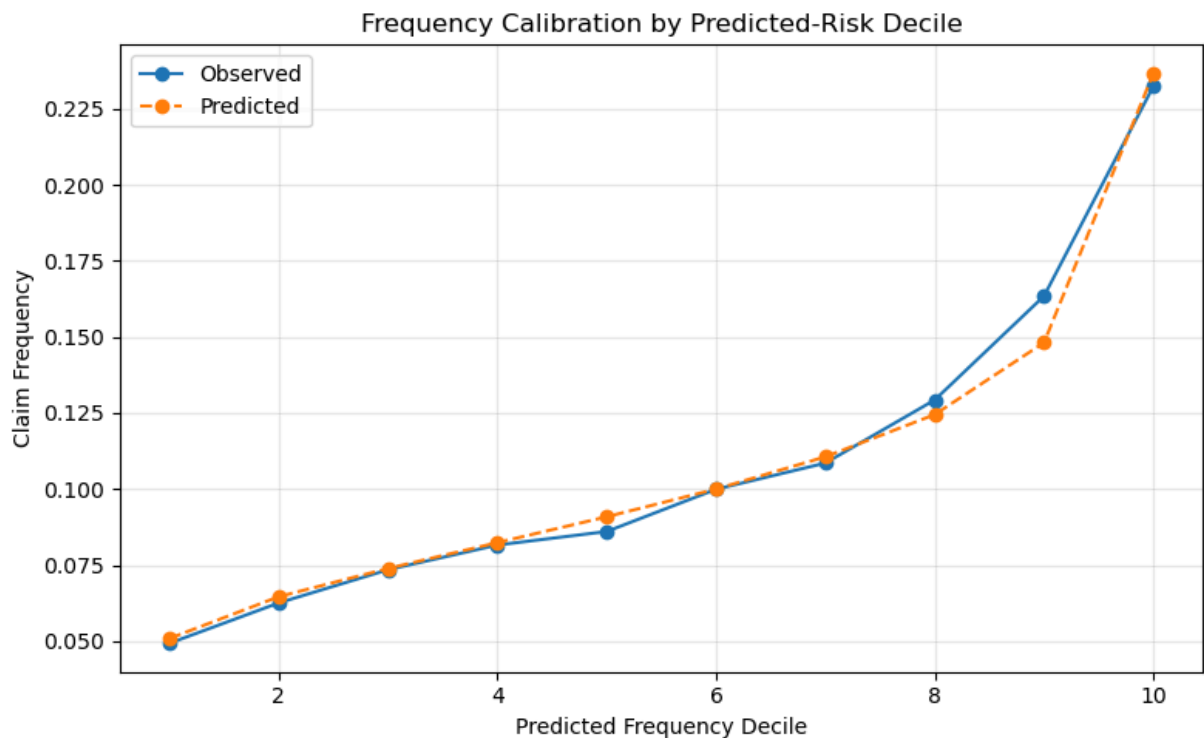
	observed_frequency	predicted_frequency	ratio_observed_predicte
d			
PredDecile			
1	0.049279	0.050865	0.96881
7			
2	0.062559	0.064606	0.96831
1			
3	0.073473	0.073893	0.99431
6			
4	0.081588	0.082383	0.99034
4			
5	0.086034	0.090872	0.94676
3			
6	0.099916	0.099994	0.99922
4			
7	0.108538	0.110604	0.98132
2			
8	0.129279	0.124482	1.03854
1			
9	0.163449	0.148131	1.10340
7			
10	0.232406	0.236683	0.98192
7			

Frequency diagnostics

```

-----
Observed frequency : 0.100614
Predicted frequency : 0.100614
Deviance / df      : 0.3193
Pearson dispersion  : 2.5326

```



Section 3 — Severity Basis and Financial Synchronization

The severity file contains paid claim amounts, while the frequency file contains reported claim counts and exposure. These two sources are reconciled at policy level before any pure premium or aggregate loss model is built.

This distinction matters: not every reported claim generates a paid amount. The aggregate loss model therefore works on financial loss, while the frequency model still describes claim occurrence.

```
In [6]: # =====
# Section: 3
# Cell: 3.1
# Title: Severity Modeling – Policy ID Cleaning and Safe Merge
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Clean policy identifiers in both frequency and severity files,
# aggregate severity at policy level, merge safely, and verify
# that no frequency rows are lost.
# =====

import numpy as np
import pandas as pd
```



```

# --- INPUT DATASETS ---
freq_base = freq_clean.copy()
sev_base = sev.copy()

# --- POLICY ID NORMALIZATION ---
# IDpol is an identifier, not a numeric variable.
# It may be loaded as float, but should be made consistent
# before grouping and merging.

def normalize_idpol(s):
    """
    Convert IDpol safely to integer identifiers.

    This checks that:
    - there are no missing IDs
    - float IDs are actually whole numbers
    """
    if s.isna().any():
        raise ValueError("IDpol contains missing values.")

    s_num = pd.to_numeric(s)

    if not np.allclose(s_num, np.round(s_num)):
        raise ValueError("IDpol contains non-integer values.")

    return s_num.round().astype("int64")

freq_base["IDpol"] = normalize_idpol(freq_base["IDpol"])
sev_base["IDpol"] = normalize_idpol(sev_base["IDpol"])

print("IDpol dtype in frequency file:", freq_base["IDpol"].dtype)
print("IDpol dtype in severity file :", sev_base["IDpol"].dtype)

# --- PRE-MERGE ROW COUNT ---
n_freq_before = len(freq_base)

# --- AGGREGATE SEVERITY PER POLICY ---
# The severity file has one row per claim.
# We aggregate first to avoid duplicating exposure after the merge.
sev_agg = (
    sev_base
    .groupby("IDpol", observed=True)
    .agg(
        ClaimAmountTotal=("ClaimAmount", "sum"),
        ClaimCountFromSev=("ClaimAmount", "count")
    )
    .reset_index()
)

# --- SAFE LEFT MERGE ---
DATA = freq_base.merge(
    sev_agg,
    on="IDpol",
    how="left",

```

```

        validate="one_to_one"
    )

# --- POST-MERGE ROW CHECK ---
n_freq_after = len(DATA)

print("\nRows before merge:", n_freq_before)
print("Rows after merge :", n_freq_after)

assert n_freq_before == n_freq_after, "Row count changed after merge."

# --- HANDLE POLICIES WITHOUT CLAIMS ---
DATA["ClaimAmountTotal"] = DATA["ClaimAmountTotal"].fillna(0.0)
DATA["ClaimCountFromSev"] = DATA["ClaimCountFromSev"].fillna(0).astype("int64")

# --- CONSISTENCY CHECKS ---
print("\nTotal claims in frequency file:", DATA["ClaimNb"].sum())
print("Total claims in severity file :", DATA["ClaimCountFromSev"].sum())

print("\nTotal claim amount:", DATA["ClaimAmountTotal"].sum())

# --- QUICK INSPECTION ---
DATA.head()

```

IDpol dtype in frequency file: int64
IDpol dtype in severity file : int64

Rows before merge: 678013
Rows after merge : 678013

Total claims in frequency file: 36056
Total claims in severity file : 26444

Total claim amount: 59909216.49999999

Out[6]:

	IDpol	ClaimNb	Exposure	Area	VehPower	VehAge	DrivAge	BonusMalus
0	1	1	0.10	D	5	0	55	50
1	3	1	0.77	D	5	0	55	50
2	5	1	0.75	B	6	2	52	50
3	10	1	0.09	B	7	0	46	50
4	11	1	0.84	B	7	0	46	50

In [7]:

```

# =====
# Section: 3
# Cell: 3.1B
# Title: Burt Representation Layer – Policy-Level Severity Merge Alignment
# Version: 1.0.0
# Date: 2026-05-10
# Description:
# Create explicit classical and Burt policy-level modeling tables
# after the severity merge. This preserves the original DATA object
# for the A-series and creates DATA_BURT for B-series experiments.
# =====

```

```

if "DATA" not in globals():
    raise RuntimeError("Run the severity safe-merge cell before this Burt al

if "BURT_COORDINATES" not in globals() or "BURT_FEATURE_COLUMNS" not in glob
    raise RuntimeError("Run the Burt representation layer before this alignm

DATA_CLASSICAL = DATA.copy()

# Align Burt coordinates by policy identifier. This avoids relying on row or
burt_policy_features = BURT_COORDINATES.copy()
burt_policy_features["IDpol"] = normalize_idpol(burt_policy_features["IDpol"]

DATA_BURT = DATA_CLASSICAL.merge(
    burt_policy_features,
    on="IDpol",
    how="left",
    validate="one_to_one",
)

missing_burt_values = DATA_BURT[BURT_FEATURE_COLUMNS].isna().sum().sum()
if missing_burt_values:
    raise ValueError(f"Missing Burt coordinate values after policy merge: {n

REPRESENTATION_TABLES = {
    "classical": DATA_CLASSICAL,
    "burt": DATA_BURT,
    "burt_augmented": DATA_BURT,
}

print("Representation tables prepared.")
print(f"DATA_CLASSICAL shape : {DATA_CLASSICAL.shape}")
print(f"DATA_BURT shape : {DATA_BURT.shape}")
print(f"Burt coordinate cols : {len(BURT_FEATURE_COLUMNS)}")

```

```

Representation tables prepared.
DATA_CLASSICAL shape : (678013, 14)
DATA_BURT shape      : (678013, 67)
Burt coordinate cols : 53

```

```

In [8]: # =====
# Section: 3
# Cell: 3.2
# Title: Severity Modeling – Frequency-Severity Consistency Check
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Check consistency between reported claim counts in the frequency
# file and paid claim records in the severity file.
# =====

# Claim count reported in the frequency dataset
reported_claim_count = DATA["ClaimNb"].sum()

# Claim records observed in the severity dataset after merge/aggregation
paid_claim_count = DATA["ClaimCountFromSev"].sum()

```

```

nil_claims = reported_claim_count - paid_claim_count
nil_claim_rate = nil_claims / reported_claim_count

print(f"Reported claim count from frequency file: {reported_claim_count:,.0f}")
print(f"Paid claim count from severity file      : {paid_claim_count:,.0f}")
print(f"Difference / nil claims                  : {nil_claims:,.0f}")
print(f"Nil claim rate                          : {nil_claim_rate:.2%}")

```

```

Reported claim count from frequency file: 36,056
Paid claim count from severity file      : 26,444
Difference / nil claims                  : 9,612
Nil claim rate                          : 26.66%

```

```

In [9]: # =====
# Section: 3
# Cell: 3.3
# Title: Severity Modeling – Positive Paid Claim Amounts
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Create the claim-level severity dataset using positive paid
# claim amounts only.
# =====

SEV_DATA = sev.copy()

# Keep only strictly positive paid claim amounts
SEV_DATA = SEV_DATA[SEV_DATA["ClaimAmount"] > 0].copy()

print(f"Number of paid claim records:          {len(SEV_DATA):,}")
print(f"Total paid claim amount:                {SEV_DATA['ClaimAmount'].sum():,.0f}")
print(f"Mean severity per paid claim:              {SEV_DATA['ClaimAmount'].mean():,.0f}")

```

```

Number of paid claim records:          26,639
Total paid claim amount:                60,697,930.68
Mean severity per paid claim:          2,278.54

```

```

In [10]: # =====
# Section: 3
# Cell: 3.4
# Title: Severity Modeling – Distribution Candidate Screening
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Fit candidate severity distributions to positive claim amounts
# and compare goodness-of-fit using AIC.
# This diagnostic explains why Burr XII is attractive statistically,
# while the headline aggregation model retains lognormal as the more
# transparent severity. The Burr XII swap is priced and quantified in
# Section 14 (native FFT engine + aggregate-package cross-check).
# =====

import scipy.stats as st
import pandas as pd
import numpy as np

```

```

# Positive claim amounts at policy / claim aggregation level.
if "SEV_DATA" in globals():
    x = SEV_DATA["ClaimAmount"].to_numpy()
elif "sev" in globals():
    x = sev["ClaimAmount"].to_numpy()
else:
    raise RuntimeError("Severity data not found. Run the severity preparatio

x = x[np.isfinite(x)]
x = x[x > 0]

candidates = [
    st.gamma,
    st.lognorm,
    st.weibull_min,
    st.invgauss,
    st.pareto,
    st.burr12,
    st.fisk,          # log-logistic
    st.genpareto,
    st.lomax,         # Pareto II – lighter parameterisation than pareto
    st.invweibull,    # Fréchet – heavy-tailed, good for large claims
    st.pearson3,      # classical actuarial choice
    st.levy,          # very heavy tail, useful as upper bound reference
]
results = []

for dist in candidates:
    try:
        params = dist.fit(x, floc=0)
        loglik = dist.logpdf(x, *params).sum()
        n_params = len(params)
        aic = 2 * n_params - 2 * loglik

        results.append({
            "distribution": dist.name,
            "aic": aic,
            "loglik": loglik,
            "params": params
        })

    except Exception as e:
        results.append({
            "distribution": dist.name,
            "aic": np.nan,
            "loglik": np.nan,
            "params": None,
            "error": str(e)
        })

severity_fit_results = (
    pd.DataFrame(results)
    .sort_values("aic", na_position="last")
    .reset_index(drop=True)
)

```

```

display(severity_fit_results[["distribution", "aic", "loglik"]])

best_sev_dist = severity_fit_results.iloc[0]["distribution"]

print(f"Best severity distribution by AIC: {best_sev_dist}")

if best_sev_dist == "burr12":
    print(
        "\nNote: Burr XII provides the best statistical fit by AIC, "
        "The headline aggregate model uses lognormal because, once each "
        "segment mean is fixed, the distributional choice is immaterial to "
        "99% capital (quantified in Section 14) and lognormal is the more "
        "transparent benchmark. Burr XII is priced in Section 14 on both a "
        "native FFT engine and the aggregate package (explicit shape params)
    )

```

	distribution	aic	loglik
0	burr12	442399.620839	-221195.810419
1	fisk	442863.084687	-221428.542344
2	lomax	446455.589967	-223224.794984
3	genpareto	446455.589967	-223224.794984
4	lognorm	447186.270592	-223590.135296
5	weibull_min	455023.039070	-227508.519535
6	invweibull	459102.617386	-229548.308693
7	invgauss	462018.528755	-231006.264378
8	gamma	462314.011047	-231154.005523
9	levy	466538.457830	-233267.228915
10	pearson3	518299.355983	-259146.677991
11	pareto	520748.490176	-260371.245088

Best severity distribution by AIC: burr12

Note: Burr XII provides the best statistical fit by AIC, The headline aggregate model uses lognormal because, once each segment mean is fixed, the distributional choice is immaterial to 99% capital (quantified in Section 14) and lognormal is the more transparent benchmark. Burr XII is priced in Section 14 on both a native FFT engine and the aggregate package (explicit shape params).

Conclusion — Severity Screening and the Choice of Aggregate Distribution

The distribution screening confirms that Burr XII (AIC rank 1) and fisk / log-logistic (AIC rank 2) fit the freMTPL2 claim severity substantially better than lognormal (AIC rank 5),

with an AIC improvement of approximately 4,800 points. The tail-comparison charts confirm that lognormal underestimates the empirical survival function in the upper tail. Burr XII is not an exotic choice: it is a standard actuarial severity distribution, catalogued in Klugman–Panjer–Willmot's *Loss Models* and on the SOA severity tables.

The fitted Burr XII parameters ($c = 1.63$, $d = 1.43$) give $c \times d = 2.33$ — above the existence threshold for variance ($c \times d > 2$) but below the threshold for finite skewness ($c \times d > 3$). This places the distribution in a regime of finite variance and infinite third moment. The same holds for fisk (Burr XII with $d = 1$), which requires $c > 2$ for finite variance; the fitted $c = 1.63$ falls short there.

Crucially, this does not make Burr XII inadmissible for FFT aggregation. Convolution requires finite mass, and is well-conditioned when the variance is finite — which it is here. The aggregate density exists and its 95% / 99% VaR and TVaR are finite quantities read directly off the grid, not from moments. Infinite skewness affects moment-matching error bounds and makes the *extreme* tail (99.9%+) grid-sensitive; it does not bar the distribution. Section 14 demonstrates this explicitly: once each segment's severity mean is reconciled to the pricing model — a mean-preserving reparameterisation that removes the spurious mean inflation an unadjusted MLE fit introduces — the Burr XII aggregate is well-conditioned and is reproduced, to sub-percent precision, both by a native FFT engine and by the `aggregate` package itself.

Conclusion. Lognormal with empirical per-segment CVs is retained as the headline aggregate severity — not because Burr XII is incompatible, but because, as Section 14 quantifies, the distributional choice moves portfolio capital only marginally at the 99% level while lognormal communicates more transparently. Burr XII is carried through Section 14 as the AIC-optimal, audited alternative rather than discarded on tractability grounds.

Severity Distribution Diagnostic

The candidate screening is retained as a diagnostic. In the empirical data Burr XII gives the best AIC fit, reflecting the heavy-tailed nature of motor claim severity, with log-logistic close behind.

The headline aggregate model uses a lognormal severity because, once the per-segment mean is fixed, the distributional choice is immaterial to 99% capital (quantified in Section 14) and lognormal is the more transparent communication — not because Burr XII cannot be convolved. Burr XII is carried forward in Section 14 as

the AIC-optimal alternative and audited against the lognormal result on two independent engines.

This preserves the actuarial logic: screen honestly, then select the aggregate severity for purpose and document the choice.

```
In [11]: # =====
# Section: 3
# Cell: 3.5
# Title: Severity Modeling – Distribution Re-Fitting (Moment Matching)
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Re-fit the severity distribution using aggregated policy-level data.
# Applying a scale adjustment to ensure the fitted mean matches the
# empirical mean of $2,432.78, preventing capital underestimation.
# =====

import scipy.stats as st
import numpy as np
import matplotlib.pyplot as plt

# 1. Use the aggregated amounts from Cell 21
x_agg = SEV_DATA['ClaimAmount'].values
actual_mean = x_agg.mean()

# 2. Initial Maximum Likelihood Estimation (MLE) fit
s_initial, loc_initial, scale_initial = st.lognorm.fit(x_agg, floc=0)

# 3. Option A: Scale-to-Mean Adjustment (Moment Matching)
# Formula for Lognormal Mean:  $E[X] = \text{scale} * \exp(s^2 / 2)$ 
# We solve for scale:  $\text{scale} = E[X] / \exp(s^2 / 2)$ 
s_new = s_initial
scale_new = actual_mean / np.exp((s_new**2) / 2)

# 4. Validation Output
fitted_mean = st.lognorm.mean(s_new, 0, scale_new)

print("--- Paid Claim Severity Summary (Mean-Synchronized) ---")
print(f"Number of Claims:      {len(x_agg):,}")
print(f"Empirical Mean:         {actual_mean:,.2f}")
print(f"Adjusted Fitted Mean:    {fitted_mean:,.2f}")
print(f"Mean Sync Error:        {((fitted_mean/actual_mean)-1):.6%}")
print("-" * 45)
print(f"New Shape Parameter (s): {s_new:.6f}")
print(f"New Scale Parameter:     {scale_new:.6f}")

# 5. Visual Validation
plt.figure(figsize=(10, 5))
plt.hist(x_agg, bins=100, density=True, alpha=0.5, color='gray', label='Empi')
x_plot = np.linspace(0, x_agg.max() * 0.1, 1000) # Focusing on the body
plt.plot(x_plot, st.lognorm.pdf(x_plot, s_new, 0, scale_new),
         'r-', lw=2, label='Adjusted Lognormal Fit')
```

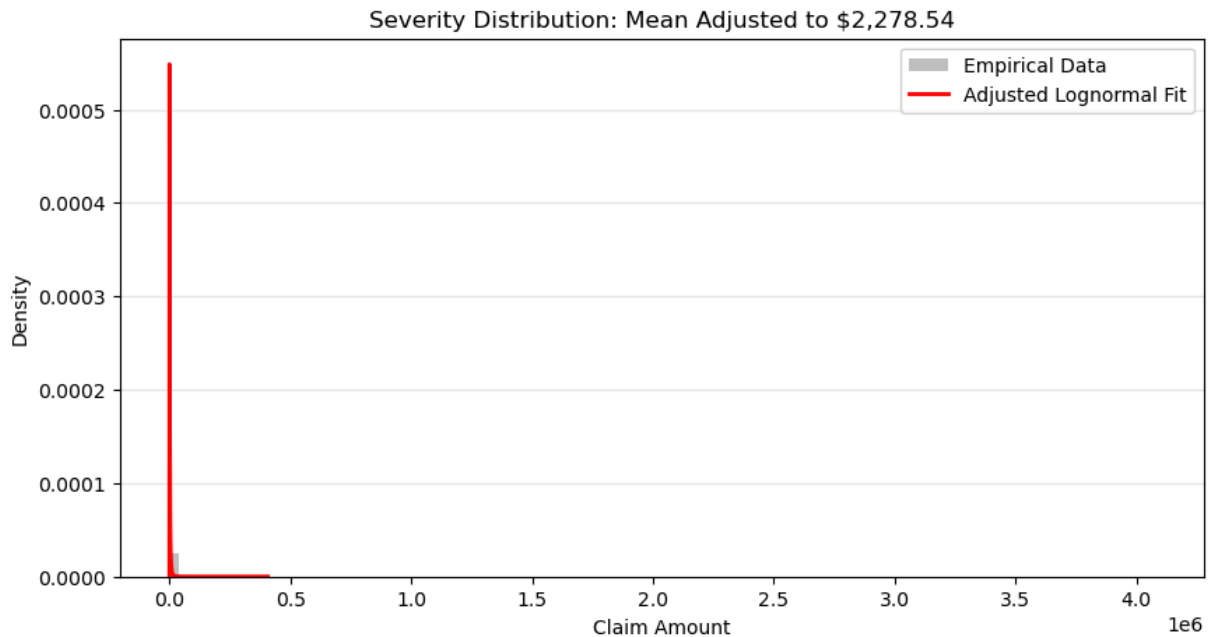


```
plt.title(f"Severity Distribution: Mean Adjusted to ${actual_mean:,.2f}")
plt.xlabel("Claim Amount")
plt.ylabel("Density")
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()
```

--- Paid Claim Severity Summary (Mean-Synchronized) ---

Number of Claims: 26,639
 Empirical Mean: 2,278.54
 Adjusted Fitted Mean: 2,278.54
 Mean Sync Error: 0.000000%

 New Shape Parameter (s): 1.132653
 New Scale Parameter: 1199.713065



Section 4 — Reusable Modeling Framework

This is the main housekeeping layer. Instead of maintaining several nearly identical notebooks, the repeated operations are factored into helper functions:

- feature construction for clustering;
- K-means and GMM segmentation with PCA stabilisation for Burt geometries;
- Tweedie GLM fitting and portfolio-level calibration;
- Gini, Tweedie deviance, lift, and decile calibration metrics;
- aggregate parameter construction.

Version 2.0.0 adds **Tweedie deviance per exposure** as a benchmark metric, following the recommendation by Hainaut & Denuit (Detra Note 2025-2) that deviance is the standard actuarial goodness-of-fit criterion for models in the

exponential dispersion family. The deviance uses calibrated predictions and exposure weights, consistent with the GLM fitting criterion.

This makes the notebook auditable and extensible: each experiment changes a configuration, not the whole workflow.

```
In [12]: # =====
# Section: 4
# Cell: 4.1
# Title: Reusable Modeling and Evaluation Framework
# Version: 2.0.0
# Date: 2026-05-23
# Description:
# Centralized helper functions for latent segmentation experiments,
# Tweedie pricing, extensible benchmark metrics, lift diagnostics,
# and aggregate validation setup.
#
# Version 2.0.0:
# - Added Tweedie deviance (power=1.5, exposure-weighted) to
#   evaluate_pricing_model following Hainaut (2025).
#   Two metrics added:
#       tweedie_deviance           - total portfolio deviance
#       tweedie_deviance_per_exposure - deviance per unit exposure
#   The formula follows the standard actuarial scaled deviance
#    $D(y, \mu) = 2 * \sum_i w_i * d(y_i, \mu_i)$  where the unit
#   deviance for Tweedie(p=1.5) is:
#        $d(y, \mu) = y^{0.5}/(-0.5*0.5) - y*\mu^{(-0.5)} / (-0.5)$ 
#        $+ \mu^{0.5}/(0.5)$ 
#   Zero actual losses contribute zero by continuity.
#   Deviance uses calibrated final_premium, not raw GLM output,
#   consistent with Gini and top-decile metrics.
# =====

import os
os.environ["LOKY_MAX_CPU_COUNT"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"

import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from glum import GeneralizedLinearRegressor, TweedieDistribution

GLOBAL_MEAN_SEVERITY = float(SEV_DATA["ClaimAmount"].mean())

BASE_CAT_FEATURES = ["Area", "VehBrand", "VehGas"]
BASE_NUM_FEATURES = ["VehPower", "VehAge", "DrivAge", "BonusMalus", "Density"]
```

```

BENCHMARK_EXECUTIVE_COLUMNS = [
    "scenario",
    "representation",
    "gini",
    "gini_increment_vs_plain",
    "top_decile_actual_to_predicted",
    "loss_calibration",
    "k",
    "method",
    "feature_set",
    "interpretation",
]

BENCHMARK_BUSINESS_COLUMNS = [
    "scenario",
    "representation",
    "gini",
    "gini_increment_vs_plain",
    "top_decile_actual_to_predicted",
    "top_decile_lift_vs_portfolio",
    "top_decile_loss_share",
    "premium_concentration_top_decile",
    "loss_calibration",
]

BENCHMARK_TECHNICAL_COLUMNS = [
    "scenario",
    "method",
    "k",
    "feature_set",
    "covariance_type",
    "n_features",
    "runtime_seconds",
    "bic",
    "aic",
]

BENCHMARK_FULL_REGISTRY_COLUMNS = [
    "scenario",
    "experiment",
    "representation",
    "gini",
    "gini_increment_vs_plain",
    "tweedie_deviance_per_exposure",    # ← new
    "tweedie_deviance",                # ← new
    "top_decile_actual_to_predicted",
    "loss_calibration",
    "method",
    "k",
    "feature_set",
    "covariance_type",
    "n_features",
    "runtime_seconds",
    "bic",
    "aic",
]

```

```

    "top_decile_lift_vs_portfolio",
    "top_decile_loss_share",
    "premium_concentration_top_decile",
    "top_decile_calibration_error",
    "mean_absolute_decile_calibration_error",
    "max_absolute_decile_calibration_error",
    "premium_rmse_weighted",
    "actual_total_loss",
    "predicted_total_loss",
    "avg_final_premium",
    "interpretation",
]

BENCHMARK_PRESENTATION_ORDER = BENCHMARK_EXECUTIVE_COLUMNS.copy()

BENCHMARK_METRIC_GROUPS = {
    "business_metrics": [
        "gini",
        "top_decile_actual_to_predicted",
        "top_decile_lift_vs_portfolio",
        "top_decile_loss_share",
        "premium_concentration_top_decile",
    ],
    "calibration_metrics": [
        "loss_calibration",
        "top_decile_calibration_error",
        "mean_absolute_decile_calibration_error",
        "max_absolute_decile_calibration_error",
    ],
    "deviance_metrics": [  # ← new group
        "tweedie_deviance_per_exposure",
        "tweedie_deviance",
    ],
    "stability_metrics": [
        "actual_total_loss",
        "predicted_total_loss",
        "avg_final_premium",
    ],
    "computational_metrics": [
        "runtime_seconds",
        "n_features",
    ],
    "model_selection_metrics": [
        "bic",
        "aic",
    ],
}

def safe_divide(numerator, denominator, default=np.nan):
    """Numerically safe scalar division for benchmark metrics."""
    denominator = float(denominator)
    if np.isclose(denominator, 0.0):
        return float(default)
    return float(numerator) / denominator

```

```

def prepare_model_data(data: pd.DataFrame) -> pd.DataFrame:
    """Return a modeling copy with stable dtypes and frequency predictions."""
    df = data.copy()
    for col in ["Area", "VehPower", "VehBrand", "VehGas", "Region"]:
        if col in df.columns:
            df[col] = df[col].astype("category")
    df["PredFreq"] = freq_model.predict(df)
    return df

def get_burt_feature_columns(df: pd.DataFrame) -> list[str]:
    """Return Burt coordinate columns available in the supplied dataframe."""
    if "BURT_FEATURE_COLUMNS" in globals():
        return [c for c in BURT_FEATURE_COLUMNS if c in df.columns]
    return [c for c in df.columns if c.startswith("burt_")]

def build_cluster_features(df: pd.DataFrame, feature_set: str) -> np.ndarray:
    """Construct clustering features for classical and Burt representation e

    simple:
        log predicted frequency and log modeled severity basis.
    enriched:
        classical actuarial enrichment used in the existing A-series.
    burt:
        Burt latent coordinates only.
    burt_enriched:
        log predicted frequency, log modeled severity basis, classical numer
        and Burt latent coordinates.
    burt_augmented_enriched:
        categorical dummies, Burt latent coordinates, classical numeric enri
        and predicted frequency and severity. The joint clustering space for
        Higher-dimensional than burt_enriched; PCA stabilisation is applied

    The clustering layer intentionally uses information available before fir
    It does not use actual future losses.
    """
    if feature_set == "simple":
        if "PredSev" in df.columns:
            severity_values = df["PredSev"].to_numpy()
        else:
            severity_values = np.full(len(df), GLOBAL_MEAN_SEVERITY)

        X_raw = np.column_stack([
            np.log(np.clip(df["PredFreq"].to_numpy(), 1e-12, None)),
            np.log(np.clip(severity_values, 1e-12, None)),
        ])
        return X_raw

    if feature_set == "enriched":
        premium_proxy = df["PredFreq"].to_numpy() * GLOBAL_MEAN_SEVERITY
        X_raw = np.column_stack([
            np.log(np.clip(df["PredFreq"].to_numpy(), 1e-12, None)),
            np.log1p(premium_proxy),
            np.log1p(df["Exposure"].to_numpy()),
        ])
        return X_raw

```

```

        df["DrivAge"].to_numpy(),
        df["VehAge"].to_numpy(),
        df["BonusMalus"].to_numpy(),
        np.log1p(df["Density"].to_numpy()),
    ])
    return StandardScaler().fit_transform(X_raw)

if feature_set == "burt":
    burt_cols = get_burt_feature_columns(df)
    if not burt_cols:
        raise ValueError("feature_set='burt' requires Burt coordinate columns")
    return StandardScaler().fit_transform(df[burt_cols].to_numpy())

if feature_set == "burt_enriched":
    burt_cols = get_burt_feature_columns(df)
    if not burt_cols:
        raise ValueError("feature_set='burt_enriched' requires Burt coordinate columns")
    if "PredSev" in df.columns:
        severity_values = df["PredSev"].to_numpy()
    else:
        severity_values = np.full(len(df), GLOBAL_MEAN_SEVERITY)
    premium_proxy = df["PredFreq"].to_numpy() * GLOBAL_MEAN_SEVERITY
    X_raw = np.column_stack([
        np.log(np.clip(df["PredFreq"].to_numpy(), 1e-12, None)),
        np.log(np.clip(severity_values, 1e-12, None)),
        np.log1p(premium_proxy),
        np.log1p(df["Exposure"].to_numpy()),
        df["DrivAge"].to_numpy(),
        df["VehAge"].to_numpy(),
        df["BonusMalus"].to_numpy(),
        np.log1p(df["Density"].to_numpy()),
        *[df[col].to_numpy() for col in burt_cols],
    ])
    return StandardScaler().fit_transform(X_raw)

if feature_set == "burt_augmented_enriched":
    burt_cols = get_burt_feature_columns(df)
    if not burt_cols:
        raise ValueError("feature_set='burt_augmented_enriched' requires Burt coordinate columns")
    if "PredSev" in df.columns:
        severity_values = df["PredSev"].to_numpy()
    else:
        severity_values = np.full(len(df), GLOBAL_MEAN_SEVERITY)
    premium_proxy = df["PredFreq"].to_numpy() * GLOBAL_MEAN_SEVERITY
    cat_dummies = pd.get_dummies(
        df[BASE_CAT_FEATURES], drop_first=True
    ).to_numpy().astype(float)
    X_raw = np.column_stack([
        cat_dummies,
        np.log(np.clip(df["PredFreq"].to_numpy(), 1e-12, None)),
        np.log(np.clip(severity_values, 1e-12, None)),
        np.log1p(premium_proxy),
        np.log1p(df["Exposure"].to_numpy()),
        df["DrivAge"].to_numpy(),
        df["VehAge"].to_numpy(),
        df["BonusMalus"].to_numpy(),
    ])
    return StandardScaler().fit_transform(X_raw)

```

```

        np.log1p(df["Density"].to_numpy()),
        *[df[col].to_numpy() for col in burt_cols],
    ])
    return StandardScaler().fit_transform(X_raw)

raise ValueError(f"Unknown feature_set: {feature_set}")

# Feature sets that require PCA stabilisation before GMM fitting.
# Must match the sets used in the BIC sweep (Cell 5.2).
BURT_PCA_FEATURE_SETS = {"burt_enriched", "burt_augmented_enriched"}
BURT_PCA_VARIANCE_THRESHOLD_PRICING = 0.95 # same threshold as BIC sweep

def apply_segmentation(
    df: pd.DataFrame,
    method: str,
    k: int = 5,
    feature_set: str = "simple",
    covariance_type: str = "full",
):
    """Add latent segmentation variables to a modeling dataframe.

    For Burt-based feature sets (burt_enriched, burt_augmented_enriched),
    PCA stabilisation is applied before GMM fitting, using the same 95%
    variance threshold as the BIC sweep in Cell 5.2. This ensures that
    the k selected by BIC and the k used in pricing operate in the same
    latent geometry.

    K-means is applied on the raw (non-PCA) features since it does not
    require covariance estimation and is not sensitive to dimensionality
    in the same way.
    """
    df = df.copy()

    if method == "none":
        return df, {"method": "none", "k": None, "feature_set": None}

    X_raw = build_cluster_features(df, feature_set=feature_set)
    n_original_features = int(X_raw.shape[1])
    n_pca_components = None

    # Apply PCA for Burt-based GMM to match the BIC sweep geometry
    if method == "gmm" and feature_set in BURT_PCA_FEATURE_SETS:
        pca_probe = PCA(random_state=42)
        pca_probe.fit(X_raw)
        cumvar = pca_probe.explained_variance_ratio_.cumsum()
        n_pca_components = int((cumvar < BURT_PCA_VARIANCE_THRESHOLD_PRICING)
                                .sum())
        n_pca_components = min(n_pca_components, n_original_features)
        pca_final = PCA(n_components=n_pca_components, random_state=42)
        X_cluster = pca_final.fit_transform(X_raw)
    else:
        X_cluster = X_raw

    if method == "kmeans":
        model = KMeans(n_clusters=k, random_state=42, n_init=10, algorithm="

```

```

labels = model.fit_predict(X_cluster)
df["risk_cluster"] = labels.astype(str)
info = {
    "method": "kmeans",
    "k": int(k),
    "feature_set": feature_set,
    "n_features": n_original_features,
    "n_pca_components": n_pca_components,
    "inertia": float(model.inertia_),
}
return df, info

if method == "gmm":
    model = GaussianMixture(
        n_components=k,
        covariance_type=covariance_type,
        random_state=42,
        n_init=5,
        reg_covar=1e-6,
    )
    model.fit(X_cluster)
    probs = model.predict_proba(X_cluster)
    labels = model.predict(X_cluster)

    df["risk_cluster"] = labels.astype(str)
    for j in range(k):
        df[f"gmm_prob_{j}"] = probs[:, j]

    membership_sum = df[[f"gmm_prob_{j}" for j in range(k)]].sum(axis=1)
    if not np.allclose(membership_sum, 1.0):
        raise ValueError("GMM memberships do not sum to 1.")

    info = {
        "method": "gmm",
        "k": int(k),
        "feature_set": feature_set,
        "covariance_type": covariance_type,
        "n_features": n_original_features,
        "n_pca_components": n_pca_components,
        "bic": float(model.bic(X_cluster)),
        "aic": float(model.aic(X_cluster)),
    }
    return df, info

raise ValueError(f"Unknown segmentation method: {method}")

def build_tweedie_design(
    df: pd.DataFrame,
    method: str,
    k: int | None,
    representation: str = "classical",
):
    """Construct the Tweedie design matrix with representation and latent fe

    classical:

```



```

        Existing tariff design with original categorical rating factors.
    burt:
        Replaces the original categorical rating factors by Burt latent coordinates while retaining numerical actuarial variables.
    burt_augmented:
        Retains the original categorical rating factors as dummy variables and adds the Burt latent coordinates as additional numerical features.
        Tests whether the two signal types are complementary.
    """
    if representation not in {"classical", "burt", "burt_augmented"}:
        raise ValueError(f"Unknown representation: {representation}")

    if representation == "classical":
        cat_features = BASE_CAT_FEATURES.copy()
        num_features = BASE_NUM_FEATURES.copy()
    elif representation == "burt":
        cat_features = []
        burt_cols = get_burt_feature_columns(df)
        if not burt_cols:
            raise ValueError("representation='burt' requires Burt coordinates")
        num_features = BASE_NUM_FEATURES.copy() + burt_cols
    elif representation == "burt_augmented":
        cat_features = BASE_CAT_FEATURES.copy()
        burt_cols = get_burt_feature_columns(df)
        if not burt_cols:
            raise ValueError("representation='burt_augmented' requires Burt coordinates")
        num_features = BASE_NUM_FEATURES.copy() + burt_cols

    if method == "kmeans":
        cat_features = cat_features + ["risk_cluster"]

    if method == "gmm":
        num_features = num_features + [f"gmm_prob_{j}" for j in range(k - 1)]

    X_num = df[num_features].copy() if num_features else pd.DataFrame(index=df.index)
    X_cat = pd.get_dummies(df[cat_features], drop_first=True) if cat_features else pd.DataFrame(index=df.index)
    X = pd.concat([X_num, X_cat], axis=1)
    return X

def fit_calibrated_tweedie(
    df: pd.DataFrame,
    method: str,
    k: int | None,
    alpha: float = 1e-6,
    representation: str = "classical",
):
    """Fit Tweedie pure-premium model and globally reconcile to observed losses"""
    X = build_tweedie_design(df, method=method, k=k, representation=representation)
    y = df["ClaimAmountTotal"]
    w = df["Exposure"]

    model = GeneralizedLinearRegressor(
        family=TweedieDistribution(1.5),
        alpha=alpha,
        l1_ratio=0.0,

```

```

        fit_intercept=True,
    )

    model.fit(X, y, sample_weight=w)
    raw = model.predict(X)
    calibration_factor = safe_divide(y.sum(), raw.sum(), default=1.0)

    out = df.copy()
    out["final_premium"] = raw * calibration_factor

    return out, model, {
        "calibration_factor": float(calibration_factor),
        "actual_total_loss": float(y.sum()),
        "predicted_total_loss": float(out["final_premium"].sum()),
        "n_features": int(X.shape[1]),
    }

def gini_coefficient(actual, pred, weight):
    """Insurance concentration Gini using predicted ranking and cumulative a
    tmp = pd.DataFrame({"actual": actual, "pred": pred, "weight": weight}).s
    cum_weight = tmp["weight"].cumsum() / tmp["weight"].sum()
    cum_actual = tmp["actual"].cumsum() / tmp["actual"].sum()
    return float(1 - 2 * np.trapz(cum_actual, cum_weight))

def calibration_by_decile(df: pd.DataFrame, pred_col: str = "final_premium")
    """Return decile-level actual-vs-predicted diagnostics ranked by premium
    tmp = df.copy()
    tmp["decile"] = pd.qcut(tmp[pred_col], 10, labels=False, duplicates="dro
    report = tmp.groupby("decile", observed=True).agg(
        actual_loss=("ClaimAmountTotal", "sum"),
        predicted_loss=(pred_col, "sum"),
        exposure=("Exposure", "sum"),
        policies=("IDpol", "count"),
    ).reset_index()
    report["actual_to_predicted"] = report["actual_loss"] / report["predicte
    report["actual_loss_per_exposure"] = report["actual_loss"] / report["exp
    report["predicted_loss_per_exposure"] = report["predicted_loss"] / repor
    report["absolute_calibration_error"] = (report["actual_to_predicted"] -
    return report

def _tweedie_unit_deviance(actual: np.ndarray, pred: np.ndarray, p: float =
    """Compute unit deviance for Tweedie(p) distribution.

    For p=1.5 (compound Poisson-Gamma):
        
$$d(y, \mu) = 2 * [y^{(2-p)} / ((1-p)(2-p)) - y * \mu^{(1-p)} / (1-p) + \mu^{(2-p)} / (2-$$


    Policies with actual=0 contribute zero to deviance by continuity of the
    Tweedie family at y=0 for 1 < p < 2. This matches the convention used
    by glum and sklearn internally.

    Parameters
    -----
    actual : np.ndarray

```

```

        Observed losses  $y_i \geq 0$ .
    pred : np.ndarray
        Fitted values  $\mu_i > 0$  (calibrated final_premium).
    p : float
        Tweedie power parameter. Must be  $1 < p < 2$ .

Returns
-----
np.ndarray
    Unit deviance  $d(y_i, \mu_i)$  for each observation.
"""
assert 1.0 < p < 2.0, f"Tweedie power must be in (1, 2), got {p}"
with np.errstate(divide="ignore", invalid="ignore"):
    term1 = np.where(actual > 0, actual ** (2 - p) / ((1 - p) * (2 - p)))
    term2 = actual * pred ** (1 - p) / (1 - p)
    term3 = pred ** (2 - p) / (2 - p)
return 2.0 * (term1 - term2 + term3)

def evaluate_pricing_model(df: pd.DataFrame) -> dict:
    """Compute extensible benchmark metrics with business metrics first.

    Metrics are computed on calibrated final_premium predictions,
    ensuring consistency across Gini, top-decile, and deviance.

    Tweedie deviance (power=1.5, exposure-weighted) is included
    following the recommendation by Hainaut (Detra Note 2025-2)
    that deviance is the standard actuarial goodness-of-fit criterion
    for models in the exponential dispersion family.
    """
    report = calibration_by_decile(df)
    top_decile = report.iloc[-1]
    portfolio_loss_per_exposure = safe_divide(
        df["ClaimAmountTotal"].sum(), df["Exposure"].sum()
    )

    actual = df["ClaimAmountTotal"].to_numpy()
    pred = df["final_premium"].to_numpy()
    exposure = df["Exposure"].to_numpy()

    premium_error = pred - actual
    weighted_rmse = np.sqrt(
        safe_divide(np.sum(exposure * premium_error ** 2), np.sum(exposure))
    )

    # — Tweedie deviance (exposure-weighted, power=1.5) —
    unit_dev = _tweedie_unit_deviance(actual, pred, p=1.5)
    tweedie_deviance = float(np.sum(exposure * unit_dev))
    tweedie_deviance_per_exposure = float(
        safe_divide(tweedie_deviance, exposure.sum())
    )

    return {
        "gini": float(gini_coefficient(actual,
        "tweedie_deviance_per_exposure": tweedie_deviance_per_exposure,
        "tweedie_deviance": tweedie_deviance,

```

```

        "top_decile_actual_to_predicted": float(top_decile["actual_to_p
        "top_decile_lift_vs_portfolio": float(safe_divide(top_decile[
        "top_decile_loss_share": float(safe_divide(top_decile[
        "premium_concentration_top_decile": float(safe_divide(top_decile[
        "loss_calibration": float(safe_divide(pred.sum(),
        "top_decile_calibration_error": float(abs(top_decile["actual_
        "mean_absolute_decile_calibration_error": float(report["absolute_cal
        "max_absolute_decile_calibration_error": float(report["absolute_cal
        "premium_rmse_weighted": float(weighted_rmse),
        "actual_total_loss": float(actual.sum()),
        "predicted_total_loss": float(pred.sum()),
        "avg_final_premium": float(df["final_premium"].mea
    }

```

```

def order_benchmark_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Return benchmark columns in business-first order while preserving eve
    preferred = [
        "experiment",
        "gini",
        "tweedie_deviance_per_exposure",
        "top_decile_actual_to_predicted",
        "loss_calibration",
        "method",
        "k",
        "feature_set",
        "covariance_type",
        "n_features",
        "runtime_seconds",
        "bic",
        "aic",
        "top_decile_lift_vs_portfolio",
        "top_decile_loss_share",
        "premium_concentration_top_decile",
        "top_decile_calibration_error",
        "mean_absolute_decile_calibration_error",
        "max_absolute_decile_calibration_error",
        "premium_rmse_weighted",
        "actual_total_loss",
        "predicted_total_loss",
        "avg_final_premium",
    ]
    ordered = [c for c in preferred if c in df.columns]
    remainder = [c for c in df.columns if c not in ordered]
    return df[ordered + remainder]

```

```

def format_reporting_table(df: pd.DataFrame) -> pd.DataFrame:
    """Apply presentation rounding without mutating the underlying benchmark
    out = df.copy()
    rounding_rules = {
        "gini": 4,
        "gini_increment_vs_plain": 4,
        "tweedie_deviance_per_exposure": 4, # ← new
        "tweedie_deviance": 0, # ← new
        "top_decile_actual_to_predicted": 3,
    }

```

```

        "loss_calibration": 3,
        "top_decile_lift_vs_portfolio": 2,
        "top_decile_loss_share": 3,
        "premium_concentration_top_decile": 3,
        "top_decile_calibration_error": 3,
        "mean_absolute_decile_calibration_error": 3,
        "max_absolute_decile_calibration_error": 3,
        "runtime_seconds": 1,
        "bic": 1,
        "aic": 1,
        "premium_rmse_weighted": 1,
        "actual_total_loss": 0,
        "predicted_total_loss": 0,
        "avg_final_premium": 1,
        "agg_mean": 0,
        "agg_var95": 0,
        "agg_var99": 0,
        "agg_tvar99": 0,
        "sim_mean": 0,
        "sim_var95": 0,
        "sim_var99": 0,
        "sim_var999": 0,
    }
    for column, digits in rounding_rules.items():
        if column in out.columns:
            out[column] = pd.to_numeric(out[column], errors="ignore").round(
                digits
            )
    return out

def select_existing_columns(df: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
    """Select an ordered reporting view while silently skipping columns not found"""
    return df[[column for column in columns if column in df.columns]].copy()

def run_pricing_experiment(base_df: pd.DataFrame, config: dict):
    """End-to-end experiment: segmentation -> Tweedie -> calibration -> benchmarking"""
    started = time.perf_counter()
    df = prepare_model_data(base_df)
    df, seg_info = apply_segmentation(
        df,
        method=config["method"],
        k=config.get("k"),
        feature_set=config.get("feature_set", "simple"),
        covariance_type=config.get("covariance_type", "full"),
    )
    representation = config.get("representation", "classical")
    df, pp_model, fit_info = fit_calibrated_tweedie(
        df,
        method=config["method"],
        k=config.get("k"),
        alpha=config.get("alpha", 1e-6),
        representation=representation,
    )
    metrics = evaluate_pricing_model(df)
    runtime = time.perf_counter() - started

```

```

result = {
    "experiment":      config["name"],
    "representation":  config.get("representation", "classical"),
    "method":          config["method"],
    "k":               config.get("k"),
    "feature_set":     config.get("feature_set"),
    "covariance_type": config.get("covariance_type"),
    "runtime_seconds": float(runtime),
    **seg_info,
    **fit_info,
    **metrics,
}
}
return df, pp_model, result

def plot_lift_chart(df: pd.DataFrame, title: str):
    """Plot actual and predicted loss by pricing decile and return the diagram"""
    report = calibration_by_decile(df)
    x = np.arange(len(report))
    plt.figure(figsize=(11, 5))
    plt.bar(x - 0.2, report["actual_loss"], 0.4, label="Actual Loss")
    plt.bar(x + 0.2, report["predicted_loss"], 0.4, label="Predicted Loss")
    plt.title(title)
    plt.xlabel("Pricing Decile (0 = Lowest Predicted Risk)")
    plt.ylabel("Total Loss Amount")
    plt.xticks(x)
    plt.legend()
    plt.grid(axis="y", alpha=0.25)
    plt.tight_layout()
    plt.show()
    return report

```

Section 5 — Controlled Benchmark of Pricing Discrimination

The benchmark is deliberately controlled. Each scenario is fitted through the same Tweedie calibration pipeline; only the latent segmentation layer changes.

The primary metric is the **Gini coefficient**. It measures whether policies ranked as high risk by the model also concentrate a larger share of realized losses. Predictions define the ranking; actual losses determine whether the ranking is useful.

K-means Elbow Diagnostic — Selecting the Baseline Hard Segmentation

The K-means benchmark uses $k = 5$ latent risk segments. This choice should be visible and auditable, rather than appearing as an arbitrary modeling assumption.

The elbow method evaluates the within-cluster sum of squares across candidate values of k . The objective is not to mechanically optimize a single statistic, but to identify a parsimonious segmentation that captures meaningful risk structure without unnecessary fragmentation.

This diagnostic is intentionally placed before the benchmark registry because it justifies the hard-clustering baseline used in the subsequent pricing comparison.

```
In [13]: # =====
# Section: 5
# Cell: 5.1
# Title: K-means Baseline – Elbow Method Diagnostic
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Reintroduce the elbow diagnostic for the K-means baseline.
# The diagnostic uses the same simple risk-score geometry as the
# K-means benchmark: log predicted frequency and modeled severity basis.
# =====

KMEANS_ELBOW_RANGE = range(2, 9)
KMEANS_SELECTED_K = 5

KMEANS_ELBOW_BASE = prepare_model_data(DATA)
X_kmeans_elbow = build_cluster_features(KMEANS_ELBOW_BASE, feature_set="simple")

kmeans_elbow_rows = []

print("Running K-means elbow diagnostic...")
for k in KMEANS_ELBOW_RANGE:
    print(f"Fitting K-means for k={k}...")
    km = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10,
        algorithm="lloyd",
    )
    km.fit(X_kmeans_elbow)
    kmeans_elbow_rows.append({
        "k": k,
        "inertia": float(km.inertia_),
    })

kmeans_elbow_table = pd.DataFrame(kmeans_elbow_rows)
kmeans_elbow_table["relative_inertia"] = (
```

```

    kmeans_elbow_table["inertia"] / kmeans_elbow_table["inertia"].iloc[0]
)
kmeans_elbow_table["marginal_reduction"] = -kmeans_elbow_table["inertia"].di

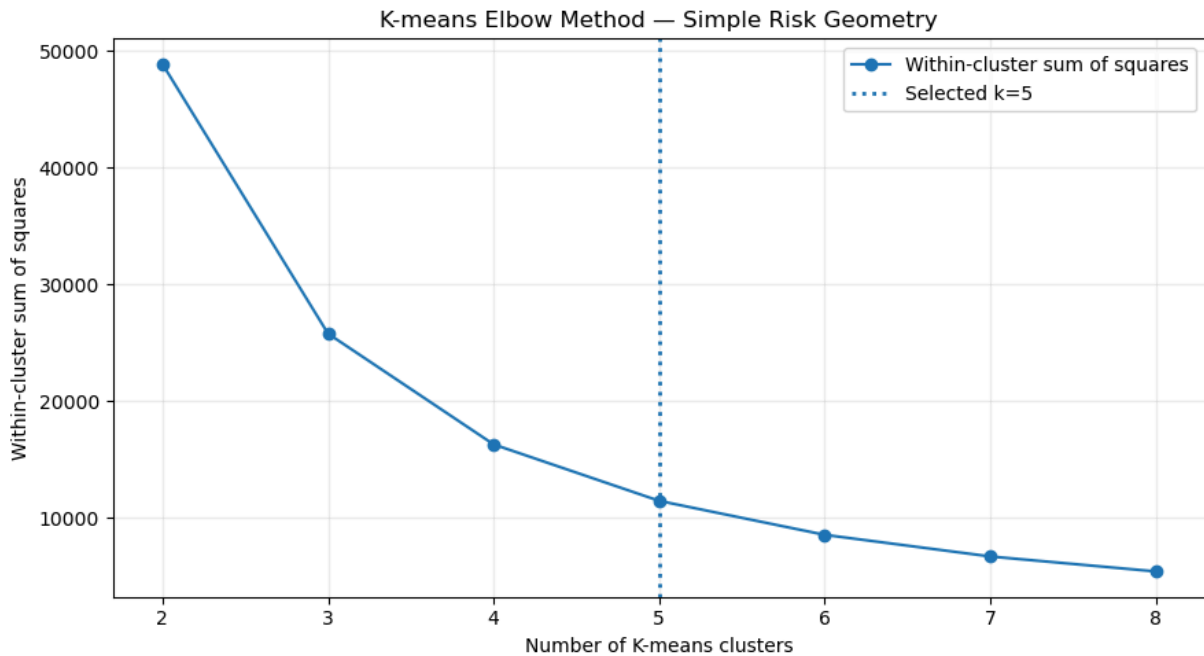
print("K-means elbow diagnostic complete.")
display(kmeans_elbow_table.round(6))

plt.figure(figsize=(9, 5))
plt.plot(
    kmeans_elbow_table["k"],
    kmeans_elbow_table["inertia"],
    marker="o",
    label="Within-cluster sum of squares",
)
plt.axvline(
    KMEANS_SELECTED_K,
    linestyle=":",
    linewidth=2,
    label=f"Selected k={KMEANS_SELECTED_K}",
)
plt.title("K-means Elbow Method – Simple Risk Geometry")
plt.xlabel("Number of K-means clusters")
plt.ylabel("Within-cluster sum of squares")
plt.xticks(list(KMEANS_ELBOW_RANGE))
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

```

Running K-means elbow diagnostic...
 Fitting K-means for k=2...
 Fitting K-means for k=3...
 Fitting K-means for k=4...
 Fitting K-means for k=5...
 Fitting K-means for k=6...
 Fitting K-means for k=7...
 Fitting K-means for k=8...
 K-means elbow diagnostic complete.

	k	inertia	relative_inertia	marginal_reduction
0	2	48862.825869	1.000000	NaN
1	3	25761.914206	0.527229	23100.911663
2	4	16269.218985	0.332957	9492.695221
3	5	11431.593465	0.233953	4837.625521
4	6	8519.649451	0.174359	2911.944014
5	7	6666.988557	0.136443	1852.660895
6	8	5383.211120	0.110170	1283.777437



Interpretation — Why $k = 5$ is the K-means Baseline (see below)

The elbow diagnostic provides the methodological bridge between the original hard-segmentation workflow and the consolidated benchmark framework.

The selected value $k = 5$ represents a pragmatic balance: enough segmentation to capture latent risk regimes, but not so many clusters that the pricing model becomes fragmented or difficult to explain.

This is deliberately different from the GMM BIC analysis later in the notebook. K-means is a distance-based heuristic and is therefore supported by an elbow diagnostic; GMM is probabilistic and can be assessed with information criteria such as BIC and AIC.

```
In [14]: # =====
# Section: 5
# Cell: 5.2
# Title: GMM Model Selection – Three-Way BIC/AIC Sweep with Cache
# Version: 7.0.0
# Date: 2026-05-11
# Description:
# Run independent GMM BIC/AIC sweeps for classical, Burt, and
# Burt-augmented joint feature spaces on the ENRICHED geometry.
# The joint sweep (burt_augmented_enriched) covers categorical
# dummies + Burt coordinates + actuarial enrichment features.
# PCA stabilisation is applied for the Burt and joint sweeps.
```

```

#
# Full covariance is used throughout, consistent with the pricing
# experiments. n_init=5 ensures stable, smooth BIC curves.
# The k=6 pathological convergence event (11,820s) cannot recur
# because results are cached to disk after the first run.
#
# Results are cached keyed by MD5 hash of data + parameters.
# Cache hit = under 1 second. Cache miss = full sweep + save.
#
# Governance variables set by this cell:
# CLASSICAL_BIC_SELECTED_K
# BURT_BIC_SELECTED_K
# BURTAUG_JOINT_BIC_SELECTED_K
# =====

from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA
import hashlib, os, pickle, time

CLASSICAL_GMM_K_RANGE = range(2, 9)
BURT_GMM_K_RANGE = range(2, 11)
BURTAUG_JOINT_GMM_K_RANGE = range(2, 11)

# — Covariance type – sensitivity parameter —————
# Controls the GMM component shape. Change this single constant
# to test alternative covariance structures. The sweep cache is
# keyed on covariance_type, so a new value triggers a full rerun
# of both the BIC sweep and all pricing experiments.
#
# "full"      own full covariance per component. Most flexible;
#              requires PCA stabilisation for Burt features.
#              Current production setting.
# "diag"      diagonal covariance per component. No inter-feature
#              correlations. Faster, stable without PCA.
#              May select a different optimal k.
# "spherical" single variance per component (isotropic / soft
#              k-means). Most stable, least flexible. Does not
#              require PCA.
GMM_COVARIANCE_TYPE = "full" # change here to test alternatives
GMM_RANDOM_STATE = 42
GMM_N_INIT = 5
GMM_REG_COVAR = 1e-6
BURT_PCA_VARIANCE_THRESHOLD = 0.95

SWEEP_CACHE_DIR = "./gmm_sweep_cache"
os.makedirs(SWEEP_CACHE_DIR, exist_ok=True)

if "DATA_CLASSICAL" not in globals():
    DATA_CLASSICAL = DATA.copy()
if "DATA_BURT" not in globals():
    raise RuntimeError("Run the Burt representation layer before this cell.")

def _sweep_cache_key(data, feature_set, k_range, pca_threshold, n_init, covar
    h = hashlib.md5()
    h.update(pd.util.hash_pandas_object(data).values.tobytes())

```

```

h.update(str(feature_set).encode())
h.update(str(list(k_range)).encode())
h.update(str(pca_threshold).encode())
h.update(str(n_init).encode())
h.update(str(covariance_type).encode())
return h.hexdigest()[:16]

def run_gmm_bic_sweep(data, feature_set, representation_label, k_range,
                     covariance_type=GMM_COVARIANCE_TYPE, pca_variance_threshold=0.95,
                     prepared = prepare_model_data(data)
X_raw = build_cluster_features(prepared, feature_set=feature_set)
X = X_raw
n_original_features = int(X_raw.shape[1])
n_pca_components = None

if pca_variance_threshold is not None:
    pca_probe = PCA(random_state=GMM_RANDOM_STATE)
    pca_probe.fit(X_raw)
    cumvar = pca_probe.explained_variance_ratio_.cumsum()
    n_pca_components = int((cumvar < pca_variance_threshold).sum()) + 1
    n_pca_components = min(n_pca_components, n_original_features)
    pca_final = PCA(n_components=n_pca_components, random_state=GMM_RANDOM_STATE)
    X = pca_final.fit_transform(X_raw)
    print(f" PCA: {n_pca_components} components (>={pca_variance_threshold} variance explained)
          f"from {n_original_features} original dims")

rows = []
for k in k_range:
    print(f" Fitting {representation_label} {feature_set} GMM k={k}...")
    model = GaussianMixture(n_components=int(k), covariance_type=covariance_type,
                           random_state=GMM_RANDOM_STATE, n_init=GMM_N_INIT,
                           reg_covar=GMM_REG_COVAR)

    t0 = time.perf_counter()
    model.fit(X)
    rows.append({
        "representation": representation_label, "feature_set": feature_set,
        "k": int(k), "covariance_type": covariance_type,
        "n_original_features": n_original_features, "n_pca_components": n_pca_components,
        "bic": float(model.bic(X)), "aic": float(model.aic(X)),
        "converged": bool(model.converged_), "n_iter": int(model.n_iter_),
        "runtime_seconds": float(time.perf_counter() - t0),
    })
return pd.DataFrame(rows)

def run_gmm_bic_sweep_cached(data, feature_set, representation_label, k_range,
                             covariance_type=GMM_COVARIANCE_TYPE, pca_variance_threshold=0.95,
                             key = _sweep_cache_key(data, feature_set, k_range, pca_variance_threshold,
                                                    GMM_N_INIT, covariance_type)
                             cache_path = os.path.join(SWEEP_CACHE_DIR,
                                                         f"{representation_label}_{feature_set}_{key}.pkl")
if os.path.exists(cache_path):
    print(f" Cache hit - loading from {cache_path}")
    with open(cache_path, "rb") as f:
        return pickle.load(f)

```

```

    result = run_gmm_bic_sweep(data, feature_set, representation_label, k_range,
                               covariance_type, pca_variance_threshold)
    with open(cache_path, "wb") as f:
        pickle.dump(result, f)
    print(f" Result cached to {cache_path}")
    return result

# --- Classical sweep ---
print("=" * 72)
print("Running GMM BIC/AIC sweep: classical enriched geometry [full covariance]")
print("=" * 72)
classical_bic_df = run_gmm_bic_sweep_cached(
    data=DATA_CLASSICAL, feature_set="enriched",
    representation_label="classical", k_range=CLASSICAL_GMM_K_RANGE,
    covariance_type=GMM_COVARIANCE_TYPE, pca_variance_threshold=None,
)

# --- Burt sweep ---
print()
print("=" * 72)
print("Running GMM BIC/AIC sweep: Burt enriched geometry [full covariance +]")
print("=" * 72)
burt_bic_df = run_gmm_bic_sweep_cached(
    data=DATA_BURT, feature_set="burt_enriched",
    representation_label="burt", k_range=BURT_GMM_K_RANGE,
    covariance_type=GMM_COVARIANCE_TYPE, pca_variance_threshold=BURT_PCA_VARIANCE_THRESHOLD,
)

# --- Burt-augmented joint sweep ---
print()
print("=" * 72)
print("Running GMM BIC/AIC sweep: Burt-augmented joint geometry [full covariance +]")
print("Joint space: categorical dummies + Burt coords + actuarial enrichment")
print("=" * 72)
burtaug_joint_bic_df = run_gmm_bic_sweep_cached(
    data=DATA_BURT,
    feature_set="burt_augmented_enriched",
    representation_label="burt_augmented_joint",
    k_range=BURTAUG_JOINT_GMM_K_RANGE,
    covariance_type=GMM_COVARIANCE_TYPE,
    pca_variance_threshold=BURT_PCA_VARIANCE_THRESHOLD,
)

# --- Consolidate ---
gmm_bic_selection_table = pd.concat(
    [classical_bic_df, burt_bic_df, burtaug_joint_bic_df],
    ignore_index=True,
)

CLASSICAL_BIC_SELECTED_K = int(classical_bic_df.loc[classical_bic_df["bic"].idxmin(), "k"])
BURT_BIC_SELECTED_K = int(burt_bic_df.loc[burt_bic_df["bic"].idxmin(), "k"])
BURTAUG_JOINT_BIC_SELECTED_K = int(burtaug_joint_bic_df.loc[burtaug_joint_bic_df["bic"].idxmin(), "k"])

print("\nGMM BIC/AIC three-way sweep complete.")
print(f"Classical BIC-selected k : {CLASSICAL_BIC_SELECTED_K} "

```

```

        f"(AIC: {int(classical_bic_df.loc[classical_bic_df['aic'].idxmin()], 'k')
print(f"Burt          BIC-selected k : {BURT_BIC_SELECTED_K}  "
        f"(AIC: {int(burt_bic_df.loc[burt_bic_df['aic'].idxmin()], 'k')})")
print(f"Burt-aug joint BIC-selected k : {BURTAUG_JOINT_BIC_SELECTED_K}  "
        f"(AIC: {int(burtaug_joint_bic_df.loc[burtaug_joint_bic_df['aic'].idxmin()], 'k')})")

display(gmm_bic_selection_table
        .sort_values(["representation", "k"])
        .round({"bic": 2, "aic": 2, "runtime_seconds": 1}))

_plot_titles = {
    "classical":          "GMM Component Selection – Classical Enriched Ge",
    "burt":                "GMM Component Selection – Burt Enriched Geometr",
    "burt_augmented_joint": (
        "GMM Component Selection – Burt-Augmented Joint Geometry\n"
        "(non-monotone curve indicates collinearity-induced instability)"
    ),
}

_best_k_map = {
    "classical":          CLASSICAL_BIC_SELECTED_K,
    "burt":                BURT_BIC_SELECTED_K,
    "burt_augmented_joint": BURTAUG_JOINT_BIC_SELECTED_K,
}

for representation, group in gmm_bic_selection_table.groupby("representation"):
    best_k    = _best_k_map.get(representation, BURT_BIC_SELECTED_K)
    title     = _plot_titles.get(
        representation,
        f"GMM Component Selection – {representation.replace('_', '-').capitalize()}")
    is_joint  = representation == "burt_augmented_joint"

    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(group["k"], group["bic"], marker="o", label="BIC",
            color="#c0392b" if is_joint else None)
    ax.plot(group["k"], group["aic"], marker="o", linestyle="--", label="AIC",
            color="#e67e22" if is_joint else None)
    ax.axvline(best_k, linestyle="--", linewidth=1.8, label=f"BIC best k={best_k}")
    ax.set_xlabel("Number of GMM components")
    ax.set_ylabel("Information criterion (full covariance)")
    ax.set_title(title, fontsize=10 if is_joint else 11,
                color="#c0392b" if is_joint else "black")
    if is_joint:
        ax.set_facecolor("#fff8f8")
        fig.patch.set_facecolor("#fff8f8")
    ax.legend()
    ax.grid(alpha=0.25)
    plt.tight_layout()
    plt.show()

```

=====
Running GMM BIC/AIC sweep: classical enriched geometry [full covariance]
=====

Cache hit – loading from ./gmm_sweep_cache\classical_enriched_a22a251cbada74f4.pkl

=====
Running GMM BIC/AIC sweep: Burt enriched geometry [full covariance + PCA]
=====

Cache hit – loading from ./gmm_sweep_cache\burt_burt_enriched_ed1431878a721854.pkl

=====
Running GMM BIC/AIC sweep: Burt-augmented joint geometry [full covariance + PCA]
=====

Joint space: categorical dummies + Burt coords + actuarial enrichment.

=====
Cache hit – loading from ./gmm_sweep_cache\burt_augmented_joint_burt_augmented_enriched_6fdf2e00f4eb6001.pkl

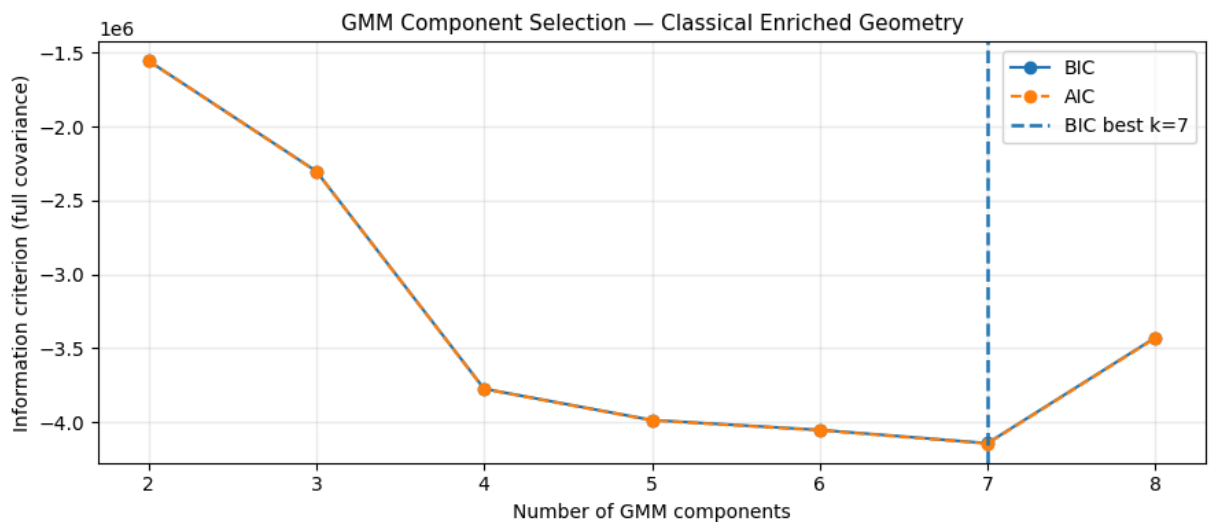
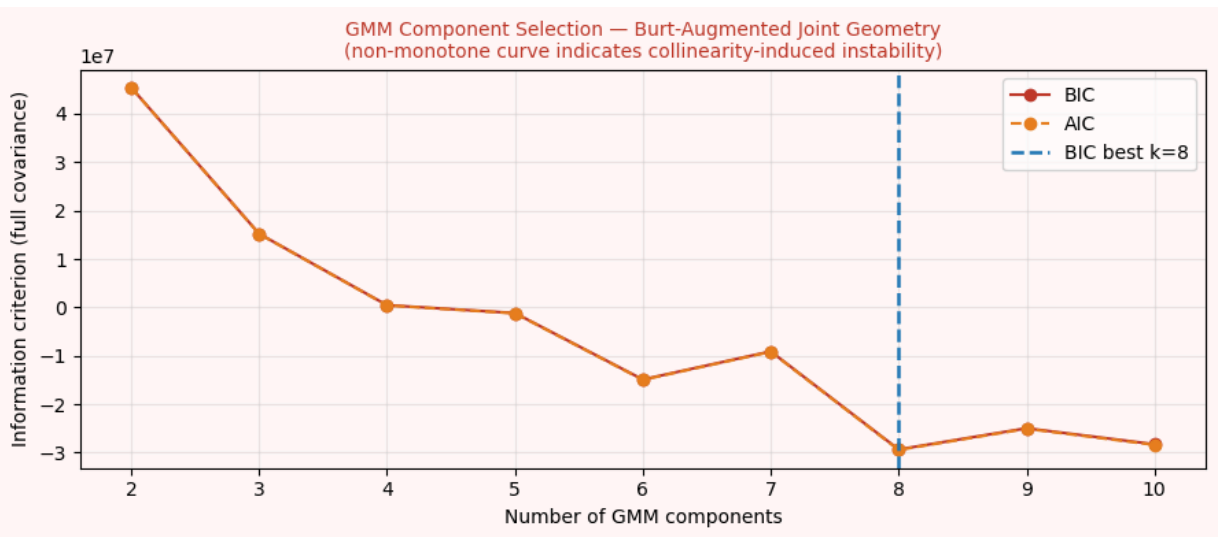
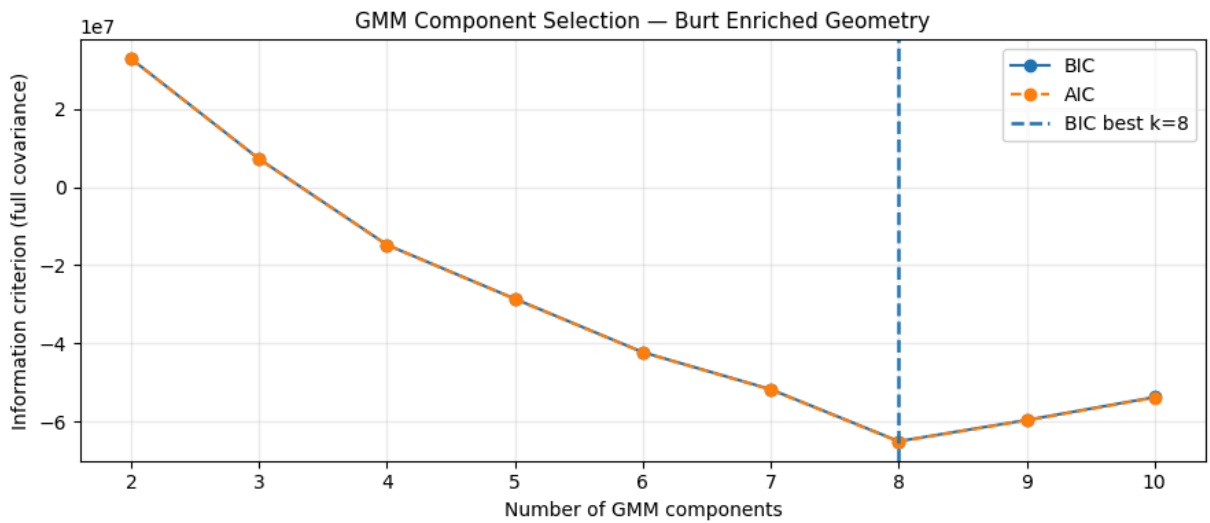
GMM BIC/AIC three-way sweep complete.

Classical BIC-selected k : 7 (AIC: 7)

Burt BIC-selected k : 8 (AIC: 8)

Burt-aug joint BIC-selected k : 8 (AIC: 8)

	representation	feature_set	k	covariance_type	n_original
7	burt	burt_enriched	2	full	
8	burt	burt_enriched	3	full	
9	burt	burt_enriched	4	full	
10	burt	burt_enriched	5	full	
11	burt	burt_enriched	6	full	
12	burt	burt_enriched	7	full	
13	burt	burt_enriched	8	full	
14	burt	burt_enriched	9	full	
15	burt	burt_enriched	10	full	
16	burt_augmented_joint	burt_augmented_enriched	2	full	
17	burt_augmented_joint	burt_augmented_enriched	3	full	
18	burt_augmented_joint	burt_augmented_enriched	4	full	
19	burt_augmented_joint	burt_augmented_enriched	5	full	
20	burt_augmented_joint	burt_augmented_enriched	6	full	
21	burt_augmented_joint	burt_augmented_enriched	7	full	
22	burt_augmented_joint	burt_augmented_enriched	8	full	
23	burt_augmented_joint	burt_augmented_enriched	9	full	
24	burt_augmented_joint	burt_augmented_enriched	10	full	
0	classical	enriched	2	full	
1	classical	enriched	3	full	
2	classical	enriched	4	full	
3	classical	enriched	5	full	
4	classical	enriched	6	full	
5	classical	enriched	7	full	
6	classical	enriched	8	full	



```
In [15]: # =====
# Section: 5
# Cell: 5.3
# Title: Benchmark Registry – A, B, and C Series Pricing Experiments
# Version: 9.0.0
# Date: 2026-05-23
# Description:
```



```

# Run all A-series (classical) and B-series (Burt-replacing)
# pricing experiments. Naming follows the original lineage:
#   x0 plain GLM baseline
#   x1 K-means simple k=5
#   x2 GMM simple k=5
#   x3 GMM enriched k=5
#   x_ext K-means enriched k=5
#   x5 GMM enriched BIC-optimal k (canonical fixed benchmark)
#
# A5 uses k=7 and B5 uses k=8 as canonical fixed benchmarks.
# After the sweep, selection_rule is set:
#   fixed k was chosen a priori
#   bic_selected k was chosen by BIC and differs from fixed
#   fixed_and_bic_selected BIC independently confirms the fixed k
# No duplicate rows are created for BIC-selected experiments
# when BIC selects the same k as the canonical fixed benchmark.
#
# Version 9.0.0:
# - Deviance per exposure printed for each experiment
# - tweedie_deviance_per_exposure added to display columns
# =====

required = ["CLASSICAL_BIC_SELECTED_K", "BURT_BIC_SELECTED_K", "BURTAUG_JOIN"]
missing = [v for v in required if v not in globals()]
if missing:
    raise RuntimeError(f"Run Cell 5.2 (BIC sweep) before Cell 5.3. Missing:

import hashlib, pickle

EXPERIMENT_CACHE_DIR = "./experiment_cache"
import os
os.makedirs(EXPERIMENT_CACHE_DIR, exist_ok=True)

def _experiment_cache_key(data, config):
    h = hashlib.md5()
    h.update(pd.util.hash_pandas_object(data).values.tobytes())
    h.update(str(sorted(
        {k: v for k, v in config.items() if k != "name"}.items()
    )).encode())
    return h.hexdigest()[:16]

def run_pricing_experiment_cached(base_table, config):
    key = _experiment_cache_key(base_table, config)
    cache_path = os.path.join(
        EXPERIMENT_CACHE_DIR, f"{config['name']}_{key}.pkl"
    )
    if os.path.exists(cache_path):
        print(f"Cache hit - {config['name']}")
        with open(cache_path, "rb") as fh:
            return pickle.load(fh)
    result = run_pricing_experiment(base_table, config)
    with open(cache_path, "wb") as fh:
        pickle.dump(result, fh)
    return result

A5_K = 7

```

B5_K = 8

```
EXPERIMENTS = [  
    # — A-series: classical —————  
    {"name": "A0_classical_plain_glm",  
     "representation": "classical", "method": "none",  
     "k": None, "feature_set": None, "alpha": 1e-6},  
  
    {"name": "A1_classical_kmeans_simple_k5",  
     "representation": "classical", "method": "kmeans",  
     "k": 5, "feature_set": "simple", "alpha": 1e-6},  
  
    {"name": "A2_classical_gmm_simple_k5",  
     "representation": "classical", "method": "gmm",  
     "k": 5, "feature_set": "simple",  
     "covariance_type": "full", "alpha": 1e-6},  
  
    {"name": "A3_classical_gmm_enriched_k5",  
     "representation": "classical", "method": "gmm",  
     "k": 5, "feature_set": "enriched",  
     "covariance_type": "full", "alpha": 1e-6},  
  
    {"name": "A_ext_classical_kmeans_enriched_k5",  
     "representation": "classical", "method": "kmeans",  
     "k": 5, "feature_set": "enriched", "alpha": 1e-6},  
  
    {"name": f"A5_classical_gmm_enriched_k{A5_K}",  
     "representation": "classical", "method": "gmm",  
     "k": A5_K, "feature_set": "enriched",  
     "covariance_type": "full", "alpha": 1e-6},  
  
    # — B-series: Burt-replacing —————  
    {"name": "B0_burt_plain_glm",  
     "representation": "burt", "method": "none",  
     "k": None, "feature_set": None, "alpha": 1e-6},  
  
    {"name": "B1_burt_kmeans_simple_k5",  
     "representation": "burt", "method": "kmeans",  
     "k": 5, "feature_set": "burt", "alpha": 1e-6},  
  
    {"name": "B2_burt_gmm_simple_k5",  
     "representation": "burt", "method": "gmm",  
     "k": 5, "feature_set": "burt",  
     "covariance_type": "full", "alpha": 1e-6},  
  
    {"name": "B3_burt_gmm_enriched_k5",  
     "representation": "burt", "method": "gmm",  
     "k": 5, "feature_set": "burt_enriched",  
     "covariance_type": "full", "alpha": 1e-6},  
  
    {"name": "B_ext_burt_kmeans_enriched_k5",  
     "representation": "burt", "method": "kmeans",  
     "k": 5, "feature_set": "burt_enriched", "alpha": 1e-6},  
  
    {"name": f"B5_burt_gmm_enriched_k{B5_K}",  
     "representation": "burt", "method": "gmm",
```

```

    "k": B5_K, "feature_set": "burt_enriched",
    "covariance_type": "full", "alpha": 1e-6},

# — C-series: Burt-augmented —————
{"name": "C0_burtaug_plain_glm",
 "representation": "burt_augmented", "method": "none",
 "k": None, "feature_set": None, "alpha": 1e-6},

{"name": "C1_burtaug_kmeans_simple_k5",
 "representation": "burt_augmented", "method": "kmeans",
 "k": 5, "feature_set": "burt", "alpha": 1e-6},

{"name": "C2_burtaug_gmm_simple_k5",
 "representation": "burt_augmented", "method": "gmm",
 "k": 5, "feature_set": "burt",
 "covariance_type": "full", "alpha": 1e-6},

{"name": "C3_burtaug_gmm_enriched_k5",
 "representation": "burt_augmented", "method": "gmm",
 "k": 5, "feature_set": "burt_enriched",
 "covariance_type": "full", "alpha": 1e-6},

{"name": "C_ext_burtaug_kmeans_enriched_k5",
 "representation": "burt_augmented", "method": "kmeans",
 "k": 5, "feature_set": "burt_enriched", "alpha": 1e-6},

{"name": f"C5_burtaug_gmm_enriched_k{BURT_BIC_SELECTED_K}",
 "representation": "burt_augmented", "method": "gmm",
 "k": BURT_BIC_SELECTED_K, "feature_set": "burt_enriched",
 "covariance_type": "full", "alpha": 1e-6},

# — C_joint variants: joint clustering geometry —————
{"name": "C3_burtaug_gmm_joint_k5",
 "representation": "burt_augmented", "method": "gmm",
 "k": 5, "feature_set": "burt_augmented_enriched",
 "covariance_type": "full", "alpha": 1e-6},

{"name": "C_ext_burtaug_kmeans_joint_k5",
 "representation": "burt_augmented", "method": "kmeans",
 "k": 5, "feature_set": "burt_augmented_enriched", "alpha": 1e-6},

{"name": f"C5_burtaug_gmm_joint_k{BURTAUG_JOINT_BIC_SELECTED_K}",
 "representation": "burt_augmented", "method": "gmm",
 "k": BURTAUG_JOINT_BIC_SELECTED_K,
 "feature_set": "burt_augmented_enriched",
 "covariance_type": "full", "alpha": 1e-6},
]

experiment_data = {}
experiment_models = {}
experiment_rows = []

for config in EXPERIMENTS:
    print("\n" + "=" * 72)
    print(f"Running: {config['name']}")
    base_table = REPRESENTATION_TABLES[config["representation"]]

```

```

df_result, model_result, metrics = run_pricing_experiment_cached(base_ta
experiment_data[config["name"]] = df_result
experiment_models[config["name"]] = model_result
experiment_rows.append(pd.DataFrame([metrics]))
print(f"   Gini                      : {metrics['gini']:.4f}")
print(f"   Deviance/exposure          : {metrics.get('tweedie_deviance_per_exp
print(f"   Top-decile ratio            : {metrics['top_decile_actual_to_predict
print(f"   Loss calibration            : {metrics['loss_calibration']:.3f}")
print(f"   Runtime (s)                  : {metrics['runtime_seconds']:.1f}")

benchmark_results = (
    pd.concat(experiment_rows, ignore_index=True)
    .sort_values("gini", ascending=False)
    .reset_index(drop=True)
)

# — Tag selection_rule —————
def _tag_selection_rule(exp_name, k_val, bic_k):
    if k_val is None:
        return "fixed"
    if k_val == bic_k:
        return "fixed_and_bic_selected"
    return "fixed"

# — Tag runtime_class —————
def _runtime_class(seconds):
    if seconds is None or pd.isna(seconds):
        return "_"
    if seconds < 30:
        return "fast"
    if seconds < 300:
        return "moderate"
    if seconds < 900:
        return "expensive"
    return "prohibitive"

benchmark_results["runtime_class"] = benchmark_results["runtime_seconds"].ap

def _get_bic_k(rep):
    if rep == "classical": return CLASSICAL_BIC_SELECTED_K
    return BURT_BIC_SELECTED_K

benchmark_results["selection_rule"] = benchmark_results.apply(
    lambda row: _tag_selection_rule(
        row["experiment"],
        row.get("k"),
        _get_bic_k(row.get("representation", "classical")),
    ),
    axis=1,
)

# — BIC-selected extras if k differs from canonical —————
if CLASSICAL_BIC_SELECTED_K != A5_K:
    print(f"\nClassical BIC selected k={CLASSICAL_BIC_SELECTED_K} != A5 k={A
    print("Running additional BIC-selected classical experiment...")
    bic_config = {

```

```

        "name": f"A5_classical_gmm_enriched_k{CLASSICAL_BIC_SELECTED_K}",
        "representation": "classical", "method": "gmm",
        "k": CLASSICAL_BIC_SELECTED_K, "feature_set": "enriched",
        "covariance_type": "full", "alpha": 1e-6,
    }
    df_r, m_r, met_r = run_pricing_experiment(DATA_CLASSICAL, bic_config)
    experiment_data[bic_config["name"]] = df_r
    experiment_models[bic_config["name"]] = m_r
    bic_row = pd.DataFrame([**met_r, "selection_rule": "bic_selected"])
    benchmark_results = pd.concat([benchmark_results, bic_row], ignore_index=True)

if BURT_BIC_SELECTED_K != B5_K:
    print(f"\nBurt BIC selected k={BURT_BIC_SELECTED_K} != B5 k={B5_K}.")
    print("Running additional BIC-selected Burt experiment...")
    bic_config = {
        "name": f"B5_burt_gmm_enriched_k{BURT_BIC_SELECTED_K}",
        "representation": "burt", "method": "gmm",
        "k": BURT_BIC_SELECTED_K, "feature_set": "burt_enriched",
        "covariance_type": "full", "alpha": 1e-6,
    }
    df_r, m_r, met_r = run_pricing_experiment(DATA_BURT, bic_config)
    experiment_data[bic_config["name"]] = df_r
    experiment_models[bic_config["name"]] = m_r
    bic_row = pd.DataFrame([**met_r, "selection_rule": "bic_selected"])
    benchmark_results = pd.concat([benchmark_results, bic_row], ignore_index=True)

benchmark_results = (
    benchmark_results
    .sort_values("gini", ascending=False)
    .reset_index(drop=True)
)

print("\nSelection rule summary:")
print(benchmark_results[["experiment", "k", "selection_rule"]].to_string(index=False))

display(
    format_reporting_table(
        select_existing_columns(
            benchmark_results,
            ["experiment", "representation", "gini",
             "tweedie_deviance_per_exposure",
             "top_decile_actual_to_predicted", "loss_calibration",
             "method", "k", "feature_set", "selection_rule", "runtime_seconds"]
        )
    )
)

```

```
=====
Running: A0_classical_plain_glm
Cache hit - A0_classical_plain_glm
Gini                : 0.3688
Deviance/exposure   : 61.6907
Top-decile ratio    : 1.032
Loss calibration    : 1.000
Runtime (s)         : 4.8
=====
```

```
=====
Running: A1_classical_kmeans_simple_k5
Cache hit - A1_classical_kmeans_simple_k5
Gini                : 0.3783
Deviance/exposure   : 61.4672
Top-decile ratio    : 1.068
Loss calibration    : 1.000
Runtime (s)         : 10.9
=====
```

```
=====
Running: A2_classical_gmm_simple_k5
Cache hit - A2_classical_gmm_simple_k5
Gini                : 0.3788
Deviance/exposure   : 61.3837
Top-decile ratio    : 1.108
Loss calibration    : 1.000
Runtime (s)         : 66.1
=====
```

```
=====
Running: A3_classical_gmm_enriched_k5
Cache hit - A3_classical_gmm_enriched_k5
Gini                : 0.4227
Deviance/exposure   : 61.2898
Top-decile ratio    : 1.090
Loss calibration    : 1.000
Runtime (s)         : 226.4
=====
```

```
=====
Running: A_ext_classical_kmeans_enriched_k5
Cache hit - A_ext_classical_kmeans_enriched_k5
Gini                : 0.3596
Deviance/exposure   : 61.2996
Top-decile ratio    : 1.029
Loss calibration    : 1.000
Runtime (s)         : 28.2
=====
```

```
=====
Running: A5_classical_gmm_enriched_k7
Cache hit - A5_classical_gmm_enriched_k7
Gini                : 0.4241
Deviance/exposure   : 61.2760
Top-decile ratio    : 1.139
Loss calibration    : 1.000
Runtime (s)         : 237.0
=====
```

```
=====
Running: B0_burt_plain_glm
=====
```

Cache hit – B0_burt_plain_glm
Gini : 0.3785
Deviance/exposure : 61.3824
Top-decile ratio : 1.035
Loss calibration : 1.000
Runtime (s) : 7.5

Running: B1_burt_kmeans_simple_k5
Cache hit – B1_burt_kmeans_simple_k5
Gini : 0.3790
Deviance/exposure : 61.3624
Top-decile ratio : 1.032
Loss calibration : 1.000
Runtime (s) : 45.3

Running: B2_burt_gmm_simple_k5
Cache hit – B2_burt_gmm_simple_k5
Gini : 0.3789
Deviance/exposure : 61.3504
Top-decile ratio : 1.033
Loss calibration : 1.000
Runtime (s) : 473.1

Running: B3_burt_gmm_enriched_k5
Cache hit – B3_burt_gmm_enriched_k5
Gini : 0.3816
Deviance/exposure : 61.2900
Top-decile ratio : 1.039
Loss calibration : 1.000
Runtime (s) : 358.7

Running: B_ext_burt_kmeans_enriched_k5
Cache hit – B_ext_burt_kmeans_enriched_k5
Gini : 0.3787
Deviance/exposure : 61.3679
Top-decile ratio : 1.031
Loss calibration : 1.000
Runtime (s) : 37.9

Running: B5_burt_gmm_enriched_k8
Cache hit – B5_burt_gmm_enriched_k8
Gini : 0.3796
Deviance/exposure : 61.3358
Top-decile ratio : 1.032
Loss calibration : 1.000
Runtime (s) : 651.2

Running: C0_burtaug_plain_glm
Cache hit – C0_burtaug_plain_glm
Gini : 0.3785

Deviance/exposure : 61.3825
Top-decile ratio : 1.034
Loss calibration : 1.000
Runtime (s) : 18.2

=====
Running: C1_burtaug_kmeans_simple_k5
Cache hit – C1_burtaug_kmeans_simple_k5
Gini : 0.3790
Deviance/exposure : 61.3625
Top-decile ratio : 1.032
Loss calibration : 1.000
Runtime (s) : 40.4

=====
Running: C2_burtaug_gmm_simple_k5
Cache hit – C2_burtaug_gmm_simple_k5
Gini : 0.3789
Deviance/exposure : 61.3506
Top-decile ratio : 1.033
Loss calibration : 1.000
Runtime (s) : 474.0

=====
Running: C3_burtaug_gmm_enriched_k5
Cache hit – C3_burtaug_gmm_enriched_k5
Gini : 0.3815
Deviance/exposure : 61.2901
Top-decile ratio : 1.039
Loss calibration : 1.000
Runtime (s) : 371.3

=====
Running: C_ext_burtaug_kmeans_enriched_k5
Cache hit – C_ext_burtaug_kmeans_enriched_k5
Gini : 0.3786
Deviance/exposure : 61.3681
Top-decile ratio : 1.031
Loss calibration : 1.000
Runtime (s) : 35.8

=====
Running: C5_burtaug_gmm_enriched_k8
Cache hit – C5_burtaug_gmm_enriched_k8
Gini : 0.3795
Deviance/exposure : 61.3360
Top-decile ratio : 1.032
Loss calibration : 1.000
Runtime (s) : 667.0

=====
Running: C3_burtaug_gmm_joint_k5
Cache hit – C3_burtaug_gmm_joint_k5
Gini : 0.3781
Deviance/exposure : 61.3459
Top-decile ratio : 1.029

Loss calibration : 1.000
Runtime (s) : 565.4

```
=====
Running: C_ext_burtaug_kmeans_joint_k5
Cache hit - C_ext_burtaug_kmeans_joint_k5
Gini : 0.3797
Deviance/exposure : 61.3258
Top-decile ratio : 1.034
Loss calibration : 1.000
Runtime (s) : 36.3
```

```
=====
Running: C5_burtaug_gmm_joint_k8
Cache hit - C5_burtaug_gmm_joint_k8
Gini : 0.3797
Deviance/exposure : 61.3453
Top-decile ratio : 1.035
Loss calibration : 1.000
Runtime (s) : 469.7
```

Selection rule summary:

experiment	k	selection_rule
A5_classical_gmm_enriched_k7	7	fixed_and_bic_selected
A3_classical_gmm_enriched_k5	5	fixed
B3_burt_gmm_enriched_k5	5	fixed
C3_burtaug_gmm_enriched_k5	5	fixed
C_ext_burtaug_kmeans_joint_k5	5	fixed
C5_burtaug_gmm_joint_k8	8	fixed_and_bic_selected
B5_burt_gmm_enriched_k8	8	fixed_and_bic_selected
C5_burtaug_gmm_enriched_k8	8	fixed_and_bic_selected
B1_burt_kmeans_simple_k5	5	fixed
C1_burtaug_kmeans_simple_k5	5	fixed
B2_burt_gmm_simple_k5	5	fixed
C2_burtaug_gmm_simple_k5	5	fixed
A2_classical_gmm_simple_k5	5	fixed
B_ext_burt_kmeans_enriched_k5	5	fixed
C_ext_burtaug_kmeans_enriched_k5	5	fixed
B0_burt_plain_glm	None	fixed
C0_burtaug_plain_glm	None	fixed
A1_classical_kmeans_simple_k5	5	fixed
C3_burtaug_gmm_joint_k5	5	fixed
A0_classical_plain_glm	None	fixed
A_ext_classical_kmeans_enriched_k5	5	fixed

	experiment	representation	gini	tweedie_deviance_
0	A5_classical_gmm_enriched_k7	classical	0.4241	
1	A3_classical_gmm_enriched_k5	classical	0.4227	
2	B3_burt_gmm_enriched_k5	burt	0.3816	
3	C3_burtaug_gmm_enriched_k5	burt_augmented	0.3815	
4	C_ext_burtaug_kmeans_joint_k5	burt_augmented	0.3797	
5	C5_burtaug_gmm_joint_k8	burt_augmented	0.3797	
6	B5_burt_gmm_enriched_k8	burt	0.3796	
7	C5_burtaug_gmm_enriched_k8	burt_augmented	0.3795	
8	B1_burt_kmeans_simple_k5	burt	0.3790	
9	C1_burtaug_kmeans_simple_k5	burt_augmented	0.3790	
10	B2_burt_gmm_simple_k5	burt	0.3789	
11	C2_burtaug_gmm_simple_k5	burt_augmented	0.3789	
12	A2_classical_gmm_simple_k5	classical	0.3788	
13	B_ext_burt_kmeans_enriched_k5	burt	0.3787	
14	C_ext_burtaug_kmeans_enriched_k5	burt_augmented	0.3786	
15	B0_burt_plain_glm	burt	0.3785	
16	C0_burtaug_plain_glm	burt_augmented	0.3785	
17	A1_classical_kmeans_simple_k5	classical	0.3783	
18	C3_burtaug_gmm_joint_k5	burt_augmented	0.3781	
19	A0_classical_plain_glm	classical	0.3688	
20	A_ext_classical_kmeans_enriched_k5	classical	0.3596	

```
In [16]: # =====
# Section: 5
# Cell: 5.4
# Title: Benchmark Metric Catalog and Governance View
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Document the extensible benchmark schema by metric family.
# This cell is a governance layer: it clarifies how new metrics
# enter the registry without changing the primary KPI narrative.
# =====

metric_catalog_rows = []
for metric_family, metric_names in BENCHMARK_METRIC_GROUPS.items():
    for metric_name in metric_names:
        metric_catalog_rows.append({
            "metric_family": metric_family,
```

```

        "metric_name": metric_name,
        "available_in_registry": metric_name in benchmark_results.columns
    })

benchmark_metric_catalog = pd.DataFrame(metric_catalog_rows)

display(benchmark_metric_catalog)

print("Benchmark metric catalog initialized.")
print("Primary KPI: gini")
print("Business metrics are presented before calibration, stability, and com

```

	metric_family	metric_name	available_in_regis
0	business_metrics	gini	1
1	business_metrics	top_decile_actual_to_predicted	1
2	business_metrics	top_decile_lift_vs_portfolio	1
3	business_metrics	top_decile_loss_share	1
4	business_metrics	premium_concentration_top_decile	1
5	calibration_metrics	loss_calibration	1
6	calibration_metrics	top_decile_calibration_error	1
7	calibration_metrics	mean_absolute_decile_calibration_error	1
8	calibration_metrics	max_absolute_decile_calibration_error	1
9	deviance_metrics	tweedie_deviance_per_exposure	1
10	deviance_metrics	tweedie_deviance	1
11	stability_metrics	actual_total_loss	1
12	stability_metrics	predicted_total_loss	1
13	stability_metrics	avg_final_premium	1
14	computational_metrics	runtime_seconds	1
15	computational_metrics	n_features	1
16	model_selection_metrics	bic	1
17	model_selection_metrics	aic	1

Benchmark metric catalog initialized.
 Primary KPI: gini
 Business metrics are presented before calibration, stability, and computatio
 nal diagnostics.

Section 6 — Model Selection and Results

This section confirms the BIC-selected k values, presents the final benchmark
 summary with Gini coefficients and calibration KPIs, and interprets the A vs B

series comparison.

The canonical fixed benchmarks are `A5_classical_gmm_enriched_k7` and `B5_burt_gmm_enriched_k8`. When the BIC sweep independently confirms these `k` values, the experiment is marked `selection_rule = fixed_and_bic_selected` — no duplicate row is created. When BIC selects a different `k`, a separate BIC-selected row is added and marked `selection_rule = bic_selected`.

The classical BIC curve shows clear deterioration at `k=8`, supporting the selected `k=7`. The Burt BIC curve supports `k=8`.

```
In [17]: # =====
# Section: 6
# Cell: 6.1
# Title: GMM Component Selection – Two-Way Confirmation and Summary
# Version: 6.0.0
# Date: 2026-05-11
# =====

required = ["gmm_bic_selection_table", "CLASSICAL_BIC_SELECTED_K",
            "BURT_BIC_SELECTED_K", "BURTAUG_JOINT_BIC_SELECTED_K"]
missing = [v for v in required if v not in globals()]
if missing:
    raise RuntimeError(f"Run Cell 5.2 first. Missing: {missing}")

A5_K = 7
B5_K = 8

print("BIC/AIC selection summary")
print("-" * 60)
print(f"Classical          enriched BIC-selected k : {CLASSICAL_BIC_SELECTED_K}")
print(f"Burt              enriched BIC-selected k : {BURT_BIC_SELECTED_K}")
print(f"Burt-augmented joint enriched BIC-selected k : {BURTAUG_JOINT_BIC_SELECTED_K}")
print()

if CLASSICAL_BIC_SELECTED_K == A5_K:
    print(f"Classical: BIC confirms fixed k={A5_K} -> A5 tagged fixed_and_bic_selected")
else:
    print(f"Classical: BIC selects k={CLASSICAL_BIC_SELECTED_K}, differs from A5_K")

if BURT_BIC_SELECTED_K == B5_K:
    print(f"Burt:          BIC confirms fixed k={B5_K} -> B5 tagged fixed_and_bic_selected")
else:
    print(f"Burt:          BIC selects k={BURT_BIC_SELECTED_K}, differs from B5_K")

classical_k_at_8 = gmm_bic_selection_table.loc[
    (gmm_bic_selection_table["representation"] == "classical") &
    (gmm_bic_selection_table["k"] == 8), "bic"
]
classical_best_bic = gmm_bic_selection_table.loc[
```

```

    gmm_bic_selection_table["representation"] == "classical", "bic"
].min()
if not classical_k_at_8.empty:
    delta = float(classical_k_at_8.iloc[0]) - classical_best_bic
    print(f"\nClassical BIC deterioration at k=8 vs best: {delta:,.1f}")

display(
    gmm_bic_selection_table
    .sort_values(["representation", "k"])
    .round({"bic": 2, "aic": 2, "runtime_seconds": 1})
)

```

BIC/AIC selection summary

```

-----
Classical          enriched  BIC-selected k : 7
Burt               enriched  BIC-selected k : 8
Burt-augmented joint enriched BIC-selected k : 8

```

```

Classical: BIC confirms fixed k=7 -> A5 tagged fixed_and_bic_selected
Burt:      BIC confirms fixed k=8 -> B5 tagged fixed_and_bic_selected

```

Classical BIC deterioration at k=8 vs best: 714,822.7

	representation	feature_set	k	covariance_type	n_original
7	burt	burt_enriched	2	full	
8	burt	burt_enriched	3	full	
9	burt	burt_enriched	4	full	
10	burt	burt_enriched	5	full	
11	burt	burt_enriched	6	full	
12	burt	burt_enriched	7	full	
13	burt	burt_enriched	8	full	
14	burt	burt_enriched	9	full	
15	burt	burt_enriched	10	full	
16	burt_augmented_joint	burt_augmented_enriched	2	full	
17	burt_augmented_joint	burt_augmented_enriched	3	full	
18	burt_augmented_joint	burt_augmented_enriched	4	full	
19	burt_augmented_joint	burt_augmented_enriched	5	full	
20	burt_augmented_joint	burt_augmented_enriched	6	full	
21	burt_augmented_joint	burt_augmented_enriched	7	full	
22	burt_augmented_joint	burt_augmented_enriched	8	full	
23	burt_augmented_joint	burt_augmented_enriched	9	full	
24	burt_augmented_joint	burt_augmented_enriched	10	full	
0	classical	enriched	2	full	
1	classical	enriched	3	full	
2	classical	enriched	4	full	
3	classical	enriched	5	full	
4	classical	enriched	6	full	
5	classical	enriched	7	full	
6	classical	enriched	8	full	

```
In [18]: # =====
# Section: 6
# Cell: 6.4
# Title: Final Benchmark Summary – Compact Business-First Views
# Version: 8.0.0
# Date: 2026-05-11
# =====

if "benchmark_results" not in globals() or benchmark_results.empty:
    raise RuntimeError("Run Cell 5.3 first.")
```

```

# — Ensure runtime_class is present —————
def _runtime_class(seconds):
    if seconds is None or pd.isna(seconds):
        return "_"
    if seconds < 30:      return "fast"
    if seconds < 300:    return "moderate"
    if seconds < 600:    return "expensive"
    return "prohibitive"

# Always recompute to pick up threshold changes
benchmark_results["runtime_class"] = benchmark_results["runtime_seconds"].apply(_runtime_class)

if "runtime_class" not in benchmark_results.columns:
    benchmark_results["runtime_class"] = benchmark_results["runtime_seconds"].apply(_runtime_class)

A5_K = 7
B5_K = 8
_a5_name = f"A5_classical_gmm_enriched_k{A5_K}"
_b5_name = f"B5_burt_gmm_enriched_k{B5_K}"
_c5_name = f"C5_burtaug_gmm_enriched_k{BURT_BIC_SELECTED_K}"
_c5_joint_name = f"C5_burtaug_gmm_joint_k{BURTAUG_JOINT_BIC_SELECTED_K}"
_a_bic_name = f"A5_classical_gmm_enriched_k{CLASSICAL_BIC_SELECTED_K}"
_b_bic_name = f"B5_burt_gmm_enriched_k{BURT_BIC_SELECTED_K}"

scenario_labels = {
    "A0_classical_plain_glm": "A0 – Classical plain Tweedie GLM",
    "A1_classical_kmeans_simple_k5": "A1 – Classical K-means simple k=5",
    "A2_classical_gmm_simple_k5": "A2 – Classical GMM simple k=5",
    "A3_classical_gmm_enriched_k5": "A3 – Classical GMM enriched k=5",
    "A_ext_classical_kmeans_enriched_k5": "A_ext – Classical K-means enriched k=5",
    _a5_name: f"A5 – Classical GMM enriched k={A5_K}",
    "B0_burt_plain_glm": "B0 – Burt plain Tweedie GLM",
    "B1_burt_kmeans_simple_k5": "B1 – Burt K-means simple k=5",
    "B2_burt_gmm_simple_k5": "B2 – Burt GMM simple k=5",
    "B3_burt_gmm_enriched_k5": "B3 – Burt GMM enriched k=5",
    "B_ext_burt_kmeans_enriched_k5": "B_ext – Burt K-means enriched k=5",
    _b5_name: f"B5 – Burt GMM enriched k={B5_K}",
    # C-series
    "C0_burtaug_plain_glm": "C0 – Burt-augmented plain Tweedie GLM",
    "C1_burtaug_kmeans_simple_k5": "C1 – Burt-augmented K-means simple k=5",
    "C2_burtaug_gmm_simple_k5": "C2 – Burt-augmented GMM simple k=5",
    "C3_burtaug_gmm_enriched_k5": "C3 – Burt-augmented GMM enriched k=5",
    "C_ext_burtaug_kmeans_enriched_k5": "C_ext – Burt-augmented K-means enriched k=5",
    _c5_name: f"C5 – Burt-augmented GMM enriched k={BURT_BIC_SELECTED_K}",
    # C_joint variants
    "C3_burtaug_gmm_joint_k5": "C3_joint – Burt-augmented GMM joint k=5",
    "C_ext_burtaug_kmeans_joint_k5": "C_ext_joint – Burt-augmented K-means joint k=5",
    _c5_joint_name: f"C5_joint – Burt-augmented GMM joint k={BURT_BIC_SELECTED_K}",
}

# Add BIC-selected labels only if they differ from A5/B5
if CLASSICAL_BIC_SELECTED_K != A5_K:
    scenario_labels[_a_bic_name] = f"A_bic – Classical GMM enriched BIC k={CLASSICAL_BIC_SELECTED_K}"
if BURT_BIC_SELECTED_K != B5_K:
    scenario_labels[_b_bic_name] = f"B_bic – Burt GMM enriched BIC k={BURT_BIC_SELECTED_K}"

scenario_notes = {

```

```

"A0_classical_plain_glm":          "Classical actuarial benchmark, r
"A1_classical_kmeans_simple_k5":   "Hard K-means on 2D risk-score ge
"A2_classical_gmm_simple_k5":      "Soft GMM on 2D risk-score geometr
"A3_classical_gmm_enriched_k5":    "Soft GMM on classical enriched ge
"A_ext_classical_kmeans_enriched_k5": "Hard K-means on classical enrich
_a5_name:                          f"Soft GMM on classical enriched g
"B0_burt_plain_glm":              "Burt baseline: categorical dummie
"B1_burt_kmeans_simple_k5":        "Hard K-means on Burt coordinate g
"B2_burt_gmm_simple_k5":          "Soft GMM on Burt coordinate geomet
"B3_burt_gmm_enriched_k5":        "Soft GMM on Burt enriched geometry
"B_ext_burt_kmeans_enriched_k5":   "Hard K-means on Burt enriched geom
_b5_name:                          f"Soft GMM on Burt enriched geometr
# C-series
"C0_burtaug_plain_glm":           "Burt-augmented baseline: categori
"C1_burtaug_kmeans_simple_k5":     "Hard K-means on Burt geometry; Tw
"C2_burtaug_gmm_simple_k5":       "Soft GMM on Burt geometry, k=5; Tw
"C3_burtaug_gmm_enriched_k5":     "Soft GMM on Burt enriched geometry
"C_ext_burtaug_kmeans_enriched_k5": "Hard K-means on Burt enriched geom
_c5_name:                          f"Soft GMM on Burt enriched geometr
# C_joint variants
"C3_burtaug_gmm_joint_k5":        "Soft GMM on joint clustering geome
"C_ext_burtaug_kmeans_joint_k5":   "Hard K-means on joint clustering g
_c5_joint_name:                    f"Soft GMM on joint clustering geom
}
if CLASSICAL_BIC_SELECTED_K != A5_K:
    scenario_notes[_a_bic_name] = f"Classical GMM enriched, BIC-selected k={
if BURT_BIC_SELECTED_K != B5_K:
    scenario_notes[_b_bic_name] = f"Burt GMM enriched, BIC-selected k={BURT_

plain_gini = float(
    benchmark_results.loc[benchmark_results["experiment"] == "A0_classical_p
)

benchmark_summary = benchmark_results.copy()
benchmark_summary["scenario"] = (
    benchmark_summary["experiment"].map(scenario_labels).fillna(benchmark_su
)
benchmark_summary["interpretation"] = (
    benchmark_summary["experiment"].map(scenario_notes).fillna(
        "Benchmark scenario retained for lineage and comparison."
    )
)
benchmark_summary["gini_increment_vs_plain"] = benchmark_summary["gini"] - p
benchmark_summary = benchmark_summary.sort_values("gini", ascending=False).r

benchmark_executive_view          = select_existing_columns(benchmark_summary, BE
benchmark_business_view           = select_existing_columns(benchmark_summary, BE
benchmark_technical_view          = select_existing_columns(benchmark_summary, BE
benchmark_full_registry_view      = select_existing_columns(benchmark_summary, BE

# — Compact executive view (primary) —————
print("Executive benchmark – primary KPI table")
display(format_reporting_table(benchmark_executive_view))

# — Full registry available for audit —————
# — Tier 1: Executive – top 5 by Gini —————

```



```

print("Tier 1 – Executive: top 5 experiments")
display(format_reporting_table(
    select_existing_columns(
        benchmark_summary.head(5),
        ["scenario", "representation", "gini",
         "gini_increment_vs_plain",
         "top_decile_actual_to_predicted",
         "runtime_class"]
    )
))

# — Tier 2: Series comparison – best per series —————
print("\nTier 2 – Series comparison: best experiment per representation fami
best_per_series = (
    benchmark_summary
    .groupby("representation", sort=False)
    .first()
    .reset_index()
)
display(format_reporting_table(
    select_existing_columns(
        best_per_series,
        ["scenario", "representation", "gini",
         "gini_increment_vs_plain",
         "top_decile_actual_to_predicted",
         "loss_calibration",
         "runtime_seconds", "runtime_class",
         "selection_rule"]
    )
))

# — Tier 3: Full registry —————
print("\nTier 3 – Full registry (audit / governance)")
display(format_reporting_table(
    select_existing_columns(
        benchmark_summary,
        ["scenario", "representation", "gini",
         "gini_increment_vs_plain",
         "top_decile_actual_to_predicted",
         "runtime_seconds", "runtime_class",
         "selection_rule"]
    )
))

```

Executive benchmark – primary KPI table

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
0	A5 — Classical GMM enriched k=7	classical	0.4241		0.0552
1	A3 — Classical GMM enriched k=5	classical	0.4227		0.0539
2	B3 — Burt GMM enriched k=5	burt	0.3816		0.0127
3	C3 — Burt- augmented GMM enriched k=5	burt_augmented	0.3815		0.0127
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	burt_augmented	0.3797		0.0109
5	C5_joint — Burt- augmented GMM joint geometry k=8	burt_augmented	0.3797		0.0109
6	B5 — Burt GMM enriched k=8	burt	0.3796		0.0107
7	C5 — Burt- augmented GMM enriched k=8	burt_augmented	0.3795		0.0107
8	B1 — Burt K-means simple k=5	burt	0.3790		0.0102
9	C1 — Burt- augmented	burt_augmented	0.3790		0.0102

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
	K-means simple k=5				
10	B2 — Burt GMM simple k=5	burt	0.3789	0.0101	
11	C2 — Burt- augmented GMM simple k=5	burt_augmented	0.3789	0.0101	
12	A2 — Classical GMM simple k=5	classical	0.3788	0.0100	
13	B_ext — Burt K- means enriched k=5	burt	0.3787	0.0098	
14	C_ext — Burt- augmented K-means enriched k=5	burt_augmented	0.3786	0.0098	
15	B0 — Burt plain Tweedie GLM	burt	0.3785	0.0097	
16	C0 — Burt- augmented plain Tweedie GLM	burt_augmented	0.3785	0.0097	
17	A1 — Classical K- means simple k=5	classical	0.3783	0.0095	
18	C3_joint — Burt- augmented GMM joint	burt_augmented	0.3781	0.0093	

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actual
	geometry k=5				
19	A0 — Classical plain Tweedie GLM	classical	0.3688	0.0000	
20	A_ext — Classical K- means enriched k=5	classical	0.3596	-0.0092	

Tier 1 – Executive: top 5 experiments

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actual
0	A5 — Classical GMM enriched k=7	classical	0.4241	0.0552	
1	A3 — Classical GMM enriched k=5	classical	0.4227	0.0539	
2	B3 — Burt GMM enriched k=5	burt	0.3816	0.0127	
3	C3 — Burt- augmented GMM enriched k=5	burt_augmented	0.3815	0.0127	
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	burt_augmented	0.3797	0.0109	

Tier 2 – Series comparison: best experiment per representation family

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actual
0	A5 — Classical GMM enriched k=7	classical	0.4241		0.0552
1	B3 — Burt GMM enriched k=5	burt	0.3816		0.0127
2	C3 — Burt- augmented GMM enriched k=5	burt_augmented	0.3815		0.0127

Tier 3 – Full registry (audit / governance)

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
0	A5 — Classical GMM enriched k=7	classical	0.4241		0.0552
1	A3 — Classical GMM enriched k=5	classical	0.4227		0.0539
2	B3 — Burt GMM enriched k=5	burt	0.3816		0.0127
3	C3 — Burt- augmented GMM enriched k=5	burt_augmented	0.3815		0.0127
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	burt_augmented	0.3797		0.0109
5	C5_joint — Burt- augmented GMM joint geometry k=8	burt_augmented	0.3797		0.0109
6	B5 — Burt GMM enriched k=8	burt	0.3796		0.0107
7	C5 — Burt- augmented GMM enriched k=8	burt_augmented	0.3795		0.0107
8	B1 — Burt K-means simple k=5	burt	0.3790		0.0102
9	C1 — Burt- augmented K-means simple k=5	burt_augmented	0.3790		0.0102
10	B2 — Burt GMM simple k=5	burt	0.3789		0.0101

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
11	C2 — Burt-augmented GMM simple k=5	burt_augmented	0.3789	0.0101	
12	A2 — Classical GMM simple k=5	classical	0.3788	0.0100	
13	B_ext — Burt K-means enriched k=5	burt	0.3787	0.0098	
14	C_ext — Burt-augmented K-means enriched k=5	burt_augmented	0.3786	0.0098	
15	B0 — Burt plain Tweedie GLM	burt	0.3785	0.0097	
16	C0 — Burt-augmented plain Tweedie GLM	burt_augmented	0.3785	0.0097	
17	A1 — Classical K-means simple k=5	classical	0.3783	0.0095	
18	C3_joint — Burt-augmented GMM joint geometry k=5	burt_augmented	0.3781	0.0093	
19	A0 — Classical plain Tweedie GLM	classical	0.3688	0.0000	
20	A_ext — Classical K-means enriched k=5	classical	0.3596	-0.0092	

```
In [19]: # =====
# Section: 6
# Cell: 6.5
```

```

# Title: Final Benchmark Visualization – Gini and Selected Diagnostics
# Version: 3.0.0
# Date: 2026-05-11
# Description:
# Visualize the primary pricing discrimination KPI and prepare
# the selected best experiment for the aggregate validation and
# governance layers.
#
# Version 3.0.0:
# - Zoomed calibration chart (x-axis 0.95–1.20)
# - Scatter plot: Gini vs top-decile ratio, coloured by series
# This is the key chart showing the discrimination/calibration
# frontier across all three representation families.
# =====

if "benchmark_results" not in globals() or benchmark_results.empty:
    raise RuntimeError("Run the benchmark registry cell first.")

if "scenario_labels" not in globals():
    raise RuntimeError("Run Cell 6.4 before this cell – scenario_labels is r

# — Colour and marker scheme —————
SERIES_STYLE = {
    "classical": {"color": "#E65100", "marker": "o", "label": "A – Clas
    "burt": {"color": "#1565C0", "marker": "s", "label": "B – Burt
    "burt_augmented": {"color": "#2E7D32", "marker": "^", "label": "C – Burt
}

# Experiments to label on the scatter plot
SCATTER_LABELS = {
    "A5_classical_gmm_enriched_k7",
    "A0_classical_plain_glm",
    "B5_burt_gmm_enriched_k8",
    f"C5_burtaug_gmm_enriched_k{BURT_BIC_SELECTED_K}",
    f"C5_burtaug_gmm_joint_k{BURTAUG_JOINT_BIC_SELECTED_K}",
    "A_ext_classical_kmeans_enriched_k5",
}

plot_df = benchmark_results.copy()
plot_df["label"] = plot_df["experiment"].map(scenario_labels).fillna(plot_df
plot_df_sorted = plot_df.sort_values("gini")

# — 1. Gini bar chart (full scale) —————
fig, ax = plt.subplots(figsize=(12, 7))
colors = [SERIES_STYLE.get(r, {}).get("color", "#888888")
           for r in plot_df_sorted["representation"]]
ax.barh(plot_df_sorted["label"], plot_df_sorted["gini"], color=colors, alpha
ax.set_xlabel("Gini Coefficient")
ax.set_title("Pricing Discrimination Benchmark – Primary KPI")
ax.grid(axis="x", alpha=0.25)
# Legend
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=v["color"], label=v["label"])
                   for v in SERIES_STYLE.values()]
ax.legend(handles=legend_elements, loc="lower right")
plt.tight_layout()

```



```

plt.show()

# — 2. Calibration bar chart (full scale) —————
if "top_decile_actual_to_predicted" in benchmark_results.columns:
    calib_df = plot_df.sort_values("top_decile_actual_to_predicted").copy()
    colors_c = [SERIES_STYLE.get(r, {}).get("color", "#888888")
                 for r in calib_df["representation"]]
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.barh(calib_df["label"], calib_df["top_decile_actual_to_predicted"],
            color=colors_c, alpha=0.85)
    ax.axvline(1.0, linestyle="--", color="black", linewidth=1.5,
              label="Perfect top-decile calibration")
    ax.set_xlabel("Top-Decile Actual / Predicted")
    ax.set_title("Top-Decile Calibration Diagnostic")
    ax.grid(axis="x", alpha=0.25)
    ax.legend(handles=[*legend_elements,
                       plt.Line2D([0],[0], color="black", linestyle="--",
                                   label="Perfect calibration")],
              loc="lower right")
    plt.tight_layout()
    plt.show()

# — 3. Calibration bar chart (zoomed 0.95–1.20) —————
if "top_decile_actual_to_predicted" in benchmark_results.columns:
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.barh(calib_df["label"], calib_df["top_decile_actual_to_predicted"],
            color=colors_c, alpha=0.85)
    ax.axvline(1.0, linestyle="--", color="black", linewidth=1.5,
              label="Perfect top-decile calibration")
    ax.set_xlim(0.95, 1.20)
    ax.set_xlabel("Top-Decile Actual / Predicted")
    ax.set_title("Top-Decile Calibration Diagnostic – Zoomed (0.95 to 1.20)")
    ax.grid(axis="x", alpha=0.25)
    ax.legend(handles=[*legend_elements,
                       plt.Line2D([0],[0], color="black", linestyle="--",
                                   label="Perfect calibration")],
              loc="lower right")
    plt.tight_layout()
    plt.show()

# — 4. Scatter: Gini vs top-decile ratio —————
if "top_decile_actual_to_predicted" in benchmark_results.columns:
    fig, ax = plt.subplots(figsize=(10, 7))

    for rep, style in SERIES_STYLE.items():
        subset = plot_df[plot_df["representation"] == rep]
        ax.scatter(
            subset["gini"],
            subset["top_decile_actual_to_predicted"],
            color=style["color"],
            marker=style["marker"],
            s=80,
            alpha=0.85,
            label=style["label"],
            zorder=3,
        )

```

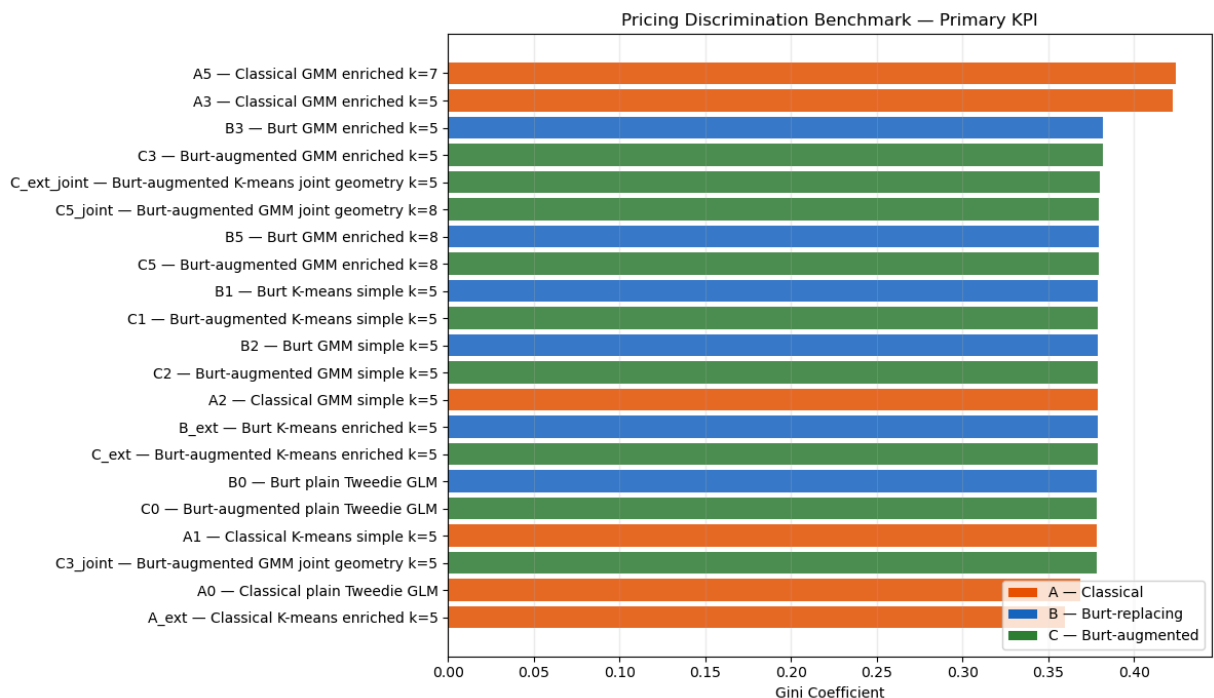
```

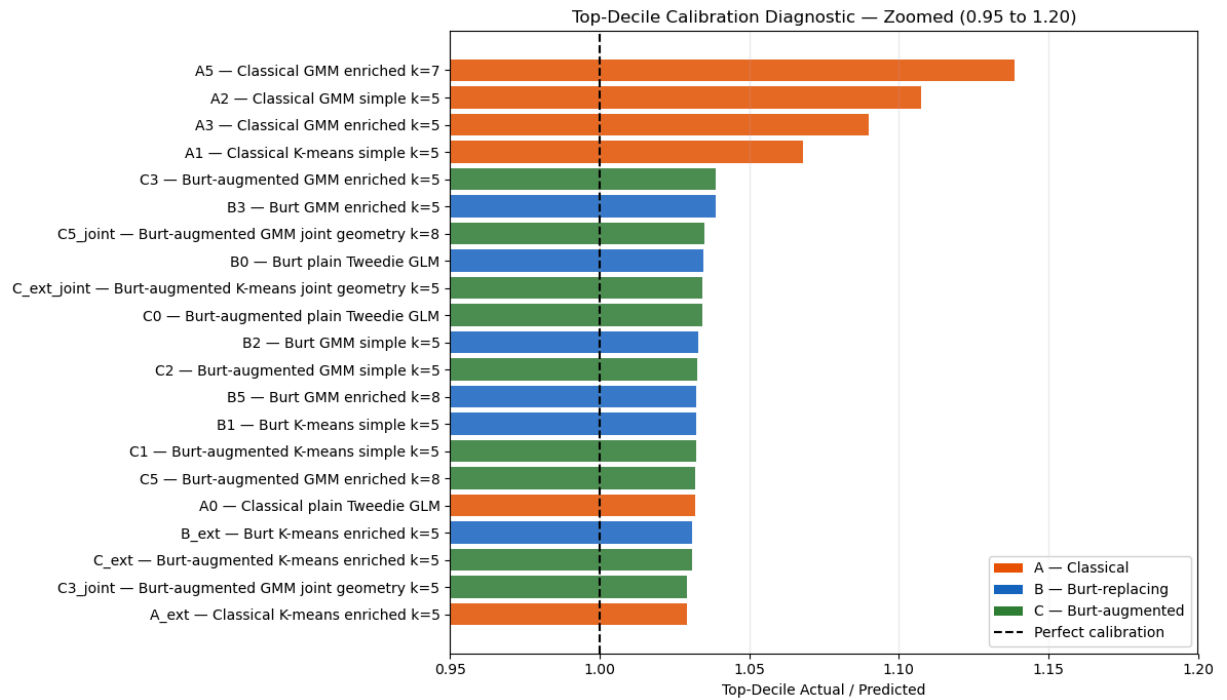
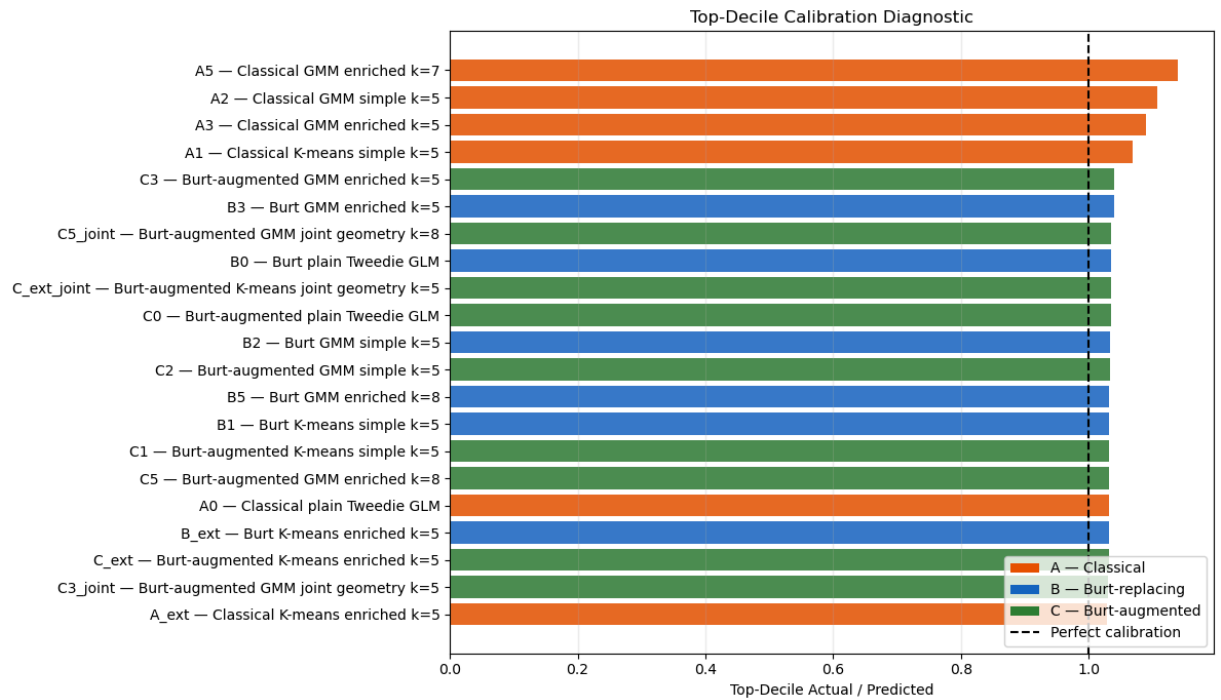
# Label key experiments
for _, row in plot_df.iterrows():
    if row["experiment"] in SCATTER_LABELS:
        short = row["label"].split(" - ")[0] # just "A5", "B5" etc.
        ax.annotate(
            short,
            xy=(row["gini"], row["top_decile_actual_to_predicted"]),
            xytext=(6, 4),
            textcoords="offset points",
            fontsize=9,
            color=SERIES_STYLE.get(row["representation"], {}).get("color", "black")
        )

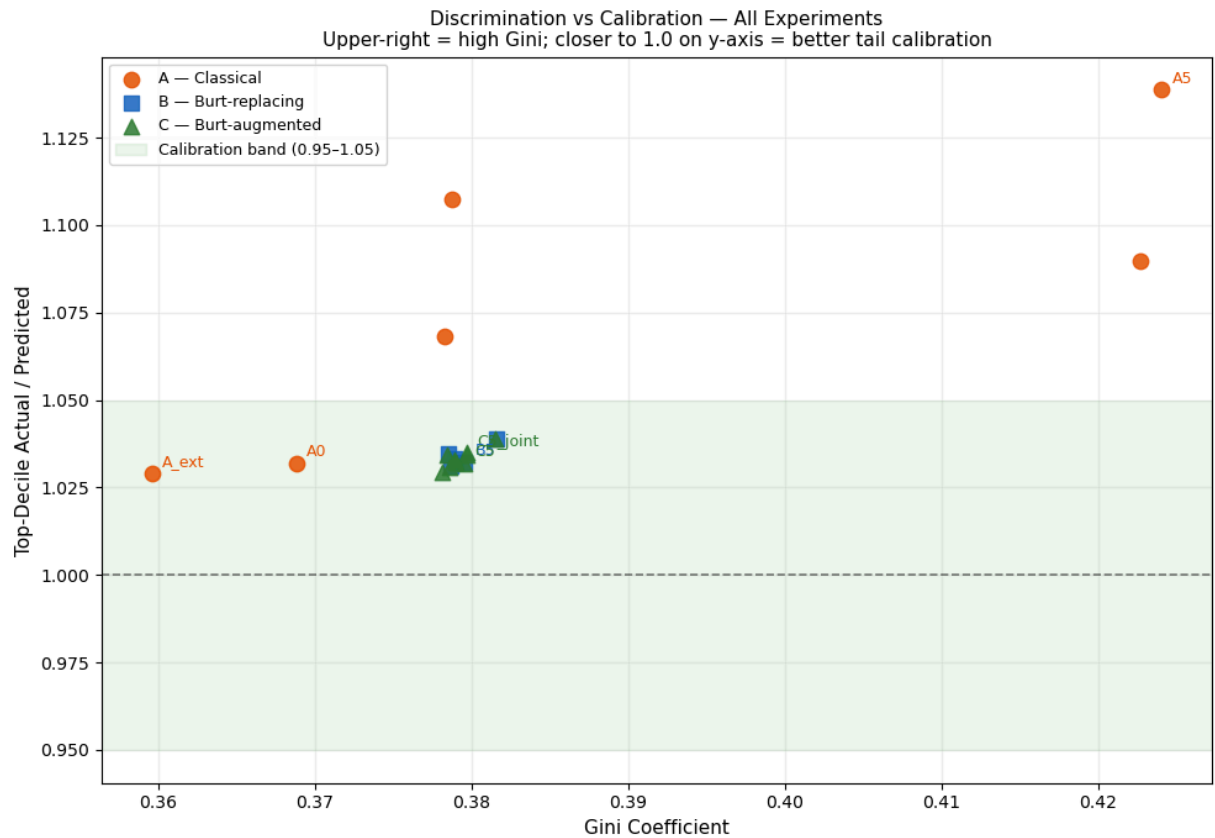
# Calibration band 0.95–1.05
ax.axhspan(0.95, 1.05, alpha=0.08, color="green",
           label="Calibration band (0.95–1.05)")
ax.axhline(1.0, linestyle="--", color="black", linewidth=1.2, alpha=0.5)

ax.set_xlabel("Gini Coefficient", fontsize=11)
ax.set_ylabel("Top-Decile Actual / Predicted", fontsize=11)
ax.set_title(
    "Discrimination vs Calibration – All Experiments\n"
    "Upper-right = high Gini; closer to 1.0 on y-axis = better tail cali
    fontsize=11,
)
ax.legend(loc="upper left", fontsize=9)
ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()

```







```
In [20]: # =====
# Section: 6
# Cell: 6.6
# Title: Experiment Selection – Best Gini, Best Deviance, Best Calibration
# Version: 2.0.0
# Date: 2026-05-23
# Description:
# Single authoritative selection of the anchor experiments
# used throughout Sections 7 and 8.
#
# Selection scope: A-series and B-series, GMM method only.
# C-series is excluded on two grounds:
# 1. Informational completeness – C3 ≈ B3 to four decimal
#    places of Gini, confirming Burt coordinates fully encode
#    the categorical structure.
# 2. BIC instability – joint clustering geometry introduces
#    collinearity between dummies and Burt coordinates,
#    producing non-monotone BIC curves.
# K-means experiments are excluded because they represent a
# different method family – comparing B_ext (K-means) against
# A5 (GMM) conflates representation choice with method choice.
# The canonical benchmark holds clustering method constant
# (GMM) and varies only the representation family.
#
# Three selection criteria:
# Best Gini      : maximum Gini coefficient
# Best Deviance  : minimum Tweedie deviance per exposure
#                  (lower = better fit; follows Hainaut 2025)
# Best Calibration : minimum absolute deviation of top-decile
#                  ratio from 1.0, restricted to experiments
```

```

#                                     above the plain GLM Gini floor,
#                                     with Gini as tie-breaker
#
# All downstream cells (7.1, 7.2, 7.3, 8.1, 8.2) consume
# BEST_DATA and best_calib_data from this cell.
# No selection logic appears elsewhere.
# =====

if "benchmark_results" not in globals() or benchmark_results.empty:
    raise RuntimeError("Run Cell 5.3 before this cell.")

if "experiment_data" not in globals():
    raise RuntimeError("experiment_data not found – run Cell 5.3 first.")

# — Scope: A and B series, GMM method only —————
ab_gmm = benchmark_results[
    benchmark_results["representation"].isin(["classical", "burt"]) &
    (benchmark_results["method"] == "gmm")
].copy()

print(f"A and B series, GMM experiments : {len(ab_gmm)}")
print(f"Experiments included:")
for _, row in ab_gmm.iterrows():
    print(f" {row['experiment']:<45} "
          f"gini={row['gini']:.4f} "
          f"deviance/exp={row['tweedie_deviance_per_exposure']:.4f} "
          f"top-decile={row['top_decile_actual_to_predicted']:.3f}")

# — Best Gini —————
best_gini_row = ab_gmm.sort_values("gini", ascending=False).iloc[0]
best_experiment = best_gini_row["experiment"]
BEST_DATA = experiment_data[best_experiment].copy()
BEST_MODEL = experiment_models[best_experiment]
BEST_METRICS = best_gini_row.to_dict()

best_gini_label = (
    scenario_labels.get(best_experiment, best_experiment)
    if "scenario_labels" in globals() else best_experiment
)

# — Best Deviance —————
# Lower deviance = better fit. Deviance is the standard
# actuarial goodness-of-fit criterion for models in the
# exponential dispersion family (Hainaut & Denuit, 2025).
best_deviance_row = ab_gmm.sort_values(
    "tweedie_deviance_per_exposure", ascending=True
).iloc[0]

best_deviance_name = best_deviance_row["experiment"]
best_deviance_metrics = best_deviance_row.to_dict()

best_deviance_label = (
    scenario_labels.get(best_deviance_name, best_deviance_name)
    if "scenario_labels" in globals() else best_deviance_name
)

```

```

# — Gini floor: plain GLM baseline —————
# Experiments that do not beat the plain GLM on Gini are
# excluded from calibration selection – good calibration
# achieved through poor discrimination is not meaningful.
baseline_gini = benchmark_results.loc[
    benchmark_results["experiment"] == "A0_classical_plain_glm",
    "gini"
].iloc[0]

print(f"\nGini floor (A0 plain GLM baseline) : {baseline_gini:.4f}")

# — Best Calibration —————
ranked_calib = ab_gmm[
    ab_gmm["gini"] > baseline_gini
].copy()

ranked_calib["top_decile_distance"] = (
    ranked_calib["top_decile_actual_to_predicted"] - 1.0
).abs()

best_calib_row = ranked_calib.sort_values(
    ["top_decile_distance", "gini"],
    ascending=[True, False]
).iloc[0]

best_calib_name = best_calib_row["experiment"]
best_calib_data = experiment_data[best_calib_name].copy()
best_calib_metrics = best_calib_row.to_dict()

best_calib_label = (
    scenario_labels.get(best_calib_name, best_calib_name)
    if "scenario_labels" in globals() else best_calib_name
)

# — Alignment check —————
# Check whether best Gini and best deviance select the same
# experiment. If yes, the two fit-quality metrics are aligned.
gini_deviance_aligned = (best_experiment == best_deviance_name)

# — Summary —————
print("\n" + "=" * 68)
print("Experiment Selection – A and B series, GMM only")
print("=" * 68)
print(f"Best Gini experiment      : {best_experiment}")
print(f"  Label                    : {best_gini_label}")
print(f"  Gini                     : {BEST_METRICS['gini']:.4f}")
print(f"  Deviance/exposure        : {BEST_METRICS['tweedie_deviance_per_ex']:.4f}")
print(f"  Top-decile ratio         : {BEST_METRICS['top_decile_actual_to_predicted']:.4f}")
print("-" * 68)
print(f"Best Deviance experiment   : {best_deviance_name}")
print(f"  Label                    : {best_deviance_label}")
print(f"  Gini                     : {best_deviance_metrics['gini']:.4f}")
print(f"  Deviance/exposure        : {best_deviance_metrics['tweedie_deviance_per_ex']:.4f}")
print(f"  Top-decile ratio         : {best_deviance_metrics['top_decile_actual_to_predicted']:.4f}")
print(f"  Aligned with best Gini    : {'YES – same experiment' if gini_deviance_aligned else 'NO'}")
print("-" * 68)

```

```

print(f"Best Calibration experiment : {best_calib_name}")
print(f"  Label                      : {best_calib_label}")
print(f"  Gini                          : {best_calib_metrics['gini']:.4f}")
print(f"  Deviance/exposure              : {best_calib_metrics['tweedie_deviance_']}")
print(f"  Top-decile ratio                : {best_calib_metrics['top_decile_actual']}")
print(f"  Top-decile distance             : {best_calib_metrics['top_decile_distan']}")
print(f"  Experiments in scope           : {len(ab_gmm)}")
print(f"  Experiments above floor        : {len(ranked_calib)}")
print("=" * 68)

if gini_deviance_aligned:
    print(
        "\nNote: Best Gini and Best Deviance select the same experiment.\n"
        "Gini and Tweedie deviance are aligned in this benchmark -\n"
        "the representation that maximises discrimination also minimises\n"
        "the likelihood-based loss. Two anchor experiments are sufficient:\n"
        f"  {best_experiment} (fit quality)\n"
        f"  {best_calib_name} (tail calibration)"
    )
else:
    print(
        "\nNote: Best Gini and Best Deviance select different experiments.\n"
        "Three distinct anchor experiments exist."
    )

# — Lift charts for both anchor experiments —————
print(f"\nLift Chart — Best Gini / Best Deviance: {best_gini_label}")
best_lift_report = plot_lift_chart(
    BEST_DATA,
    title=f"Lift Chart — {best_gini_label}",
)
display(best_lift_report)

print(f"\nLift Chart — Best Calibration: {best_calib_label}")
calib_lift_report = plot_lift_chart(
    best_calib_data,
    title=f"Lift Chart — {best_calib_label}",
)
display(calib_lift_report)

```

A and B series, GMM experiments : 6

Experiments included:

A5_classical_gmm_enriched_k7	gini=0.4241	deviance/exp=6
1.2760 top-decile=1.139		
A3_classical_gmm_enriched_k5	gini=0.4227	deviance/exp=6
1.2898 top-decile=1.090		
B3_burt_gmm_enriched_k5	gini=0.3816	deviance/exp=6
1.2900 top-decile=1.039		
B5_burt_gmm_enriched_k8	gini=0.3796	deviance/exp=6
1.3358 top-decile=1.032		
B2_burt_gmm_simple_k5	gini=0.3789	deviance/exp=6
1.3504 top-decile=1.033		
A2_classical_gmm_simple_k5	gini=0.3788	deviance/exp=6
1.3837 top-decile=1.108		

Gini floor (A0 plain GLM baseline) : 0.3688

=====
Experiment Selection – A and B series, GMM only
=====

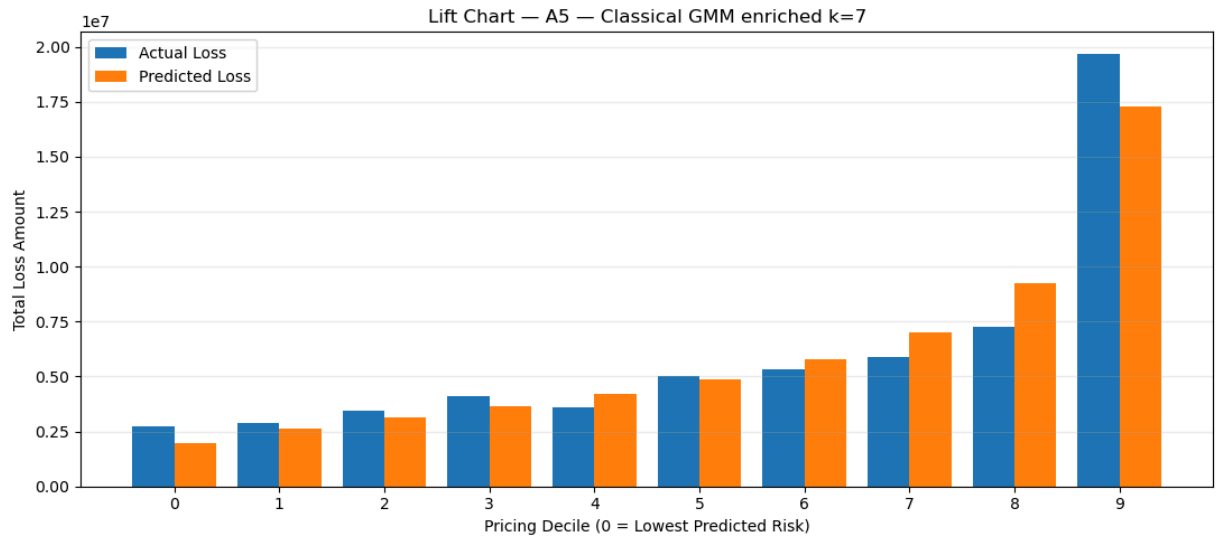
Best Gini experiment : A5_classical_gmm_enriched_k7
Label : A5 – Classical GMM enriched k=7
Gini : 0.4241
Deviance/exposure : 61.2760
Top-decile ratio : 1.139

Best Deviance experiment : A5_classical_gmm_enriched_k7
Label : A5 – Classical GMM enriched k=7
Gini : 0.4241
Deviance/exposure : 61.2760
Top-decile ratio : 1.139
Aligned with best Gini : YES – same experiment

Best Calibration experiment : B5_burt_gmm_enriched_k8
Label : B5 – Burt GMM enriched k=8
Gini : 0.3796
Deviance/exposure : 61.3358
Top-decile ratio : 1.032
Top-decile distance : 0.032
Experiments in scope : 6
Experiments above floor : 6
=====

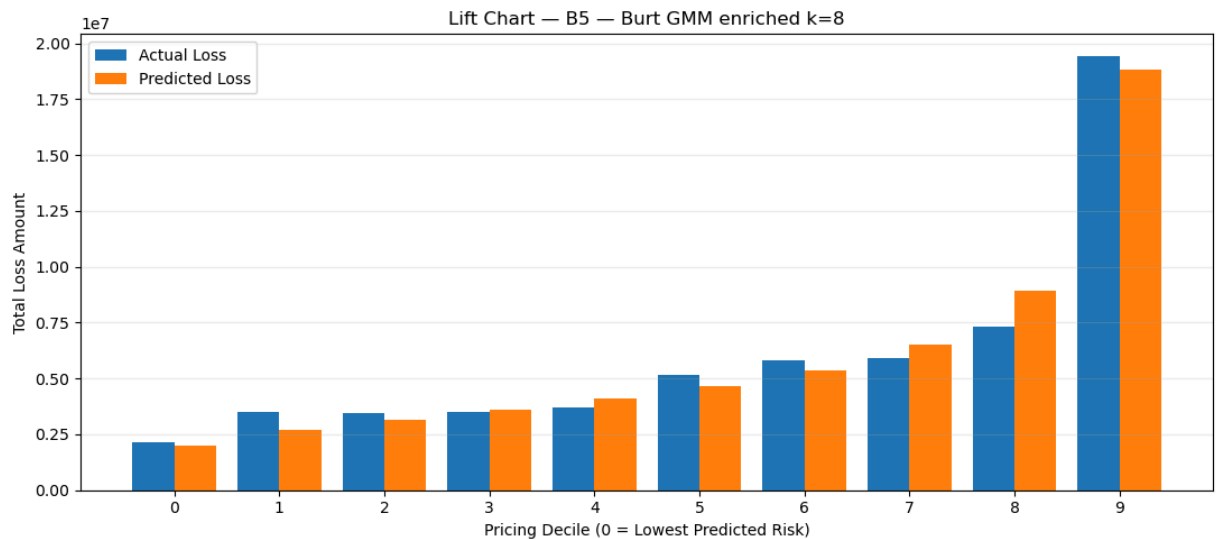
Note: Best Gini and Best Deviance select the same experiment.
Gini and Tweedie deviance are aligned in this benchmark –
the representation that maximises discrimination also minimises
the likelihood-based loss. Two anchor experiments are sufficient:
A5_classical_gmm_enriched_k7 (fit quality)
B5_burt_gmm_enriched_k8 (tail calibration)

Lift Chart – Best Gini / Best Deviance: A5 – Classical GMM enriched k=7



	decile	actual_loss	predicted_loss	exposure	policies	actual_to_predicted
0	0	2725389.99	1.983175e+06	50407.302642	67802	1.37421
1	1	2905598.41	2.645575e+06	44540.093220	67801	1.09821
2	2	3423332.38	3.159156e+06	40453.963263	67801	1.08361
3	3	4112951.70	3.672493e+06	36874.953105	67801	1.11991
4	4	3585212.62	4.228703e+06	34325.831723	67802	0.84781
5	5	5018353.80	4.894764e+06	32433.785438	67801	1.02521
6	6	5316566.85	5.766447e+06	31334.592280	67801	0.92191
7	7	5892132.47	7.024108e+06	30497.293789	67801	0.83881
8	8	7237641.96	9.240233e+06	29687.226586	67801	0.78321
9	9	19692036.32	1.729456e+07	27805.063416	67802	1.13861

Lift Chart – Best Calibration: B5 – Burt GMM enriched k=8



	decile	actual_loss	predicted_loss	exposure	policies	actual_to_predicte
0	0	2131535.72	2.013376e+06	33343.058271	67802	1.05861
1	1	3480940.95	2.694866e+06	36616.292491	67801	1.29169
2	2	3444680.20	3.166967e+06	38167.248294	67801	1.08769
3	3	3481737.50	3.613014e+06	39046.971278	67801	0.96360
4	4	3721740.28	4.082732e+06	39641.056416	67802	0.91151
5	5	5147023.03	4.635344e+06	39042.682956	67801	1.11031
6	6	5822672.91	5.367117e+06	37912.982336	67801	1.08487
7	7	5896548.15	6.543081e+06	35382.865625	67801	0.90111
8	8	7332893.69	8.952658e+06	31219.905057	67801	0.81907
9	9	19449444.07	1.884006e+07	27987.042739	67802	1.03234

Interpretation — Classical vs Burt-Replacing: What the Benchmark Reveals

The benchmark separates two questions that are easy to conflate: **does the Burt geometry improve segmentation?** and **does it improve pricing discrimination and fit quality?** The results suggest the answer to the first is yes; the answer to the second, when Burt coordinates *replace* the original categorical factors, is no — at least not within the Tweedie GLM framework.

A-series — Classical. The classical series consistently leads on both Gini and Tweedie deviance. The best classical model (A5, GMM enriched k=7) reaches Gini 0.4241 and deviance per exposure 61.276 — the lowest deviance across all 21 experiments. Gini and deviance are aligned within the A-series: both improve as segmentation quality increases. The original categorical rating factors — Area, VehBrand, VehGas — carry sharp, direct pricing signal that the Tweedie model can exploit immediately as dummy-encoded contrasts.

B-series — Burt-replacing. Replacing the categorical dummies with Burt latent coordinates costs approximately 4.5 Gini points (best B-series B5 ≈ 0.3796) and produces slightly higher deviance (61.336 vs 61.276 for A5). On calibration however the B-series performs better — top-decile ratio 1.032 versus 1.139 for A5 — a meaningful trade-off between fit quality and tail accuracy that Gini and deviance alone do not capture.

Gini and deviance are aligned but do not explain the calibration advantage. Across all 21 experiments, Gini and deviance select the same

anchor (A5). However, the overall cross-series correlation is weak ($r = -0.38$, $p = 0.09$) and not statistically significant. The B and C-series cluster tightly at competitive deviance values despite substantially lower Gini. The top-decile calibration ratio is therefore a genuinely orthogonal dimension that neither Gini nor deviance captures.

The A_ext anomaly. The classical K-means enriched model (A_ext) combines the lowest Gini in the classical series (0.3596, below the plain GLM baseline) with near-perfect top-decile calibration (1.029). Enriching the K-means clustering geometry flattens the risk ranking rather than sharpening it. Soft GMM assignment is essential to exploit enriched feature spaces.

C-series — Burt-augmented. Every C-series experiment that uses the same Burt clustering geometry as its B-series counterpart produces Gini, deviance, and top-decile results that are statistically indistinguishable from B — to four decimal places on Gini, four decimal places on deviance, and three decimal places on the top-decile ratio. This establishes the *informational completeness* of the Burt representation under soft GMM segmentation. The C-series is excluded from the canonical comparison on two grounds: informational completeness and BIC instability from collinearity in the joint clustering geometry.

C_joint — Burt-augmented with joint clustering geometry. No improvement over the pure Burt geometry, and BIC curves are non-monotone due to collinearity. The joint geometry adds complexity without benefit.

The canonical comparison is A5 vs B5. Both are GMM experiments on the enriched geometry, both BIC-confirmed, differing only in representation family. A5 leads on fit quality (Gini and deviance). B5 leads on tail calibration (top-decile ratio). The choice between them is an actuarial policy decision — not a modelling one — that depends on whether the objective is pricing discrimination or aggregate capital accuracy.

Section 7 — Aggregate Portfolio Validation

The aggregate layer translates the selected pricing output into capital language. It is used here as a stability check, not as the main measure of model value.

A better Gini should not automatically imply capital relief. Segmentation primarily changes risk differentiation across policyholders; portfolio VaR and TVaR remain mostly driven by total expected loss, claim count, severity dispersion, and diversification.

```
In [21]: # =====
# Section: 7
# Cell: 7.1
# Title: Aggregate Validation Helpers
# Version: 3.0.0
# Date: 2026-05-11
# Description:
# Helpers for aggregate validation using empirical per-segment
# CVs estimated from observed claim amounts via weighted
# lognormal MLE. Fixed CV assumptions have been removed.
#
# Weighted lognormal MLE (closed form):
#   mu_log = sum(w_i * log(x_i)) / sum(w_i)
#   sig_log = sqrt(sum(w_i * (log(x_i) - mu_log)^2) / sum(w_i))
#   CV      = sqrt(exp(sig_log^2) - 1)
# =====

from aggregate import build, qd

def prepare_aggregate_parameters(
    df: pd.DataFrame,
    segmentation_col: str | None = 'risk_cluster',
    empirical_cvs: list | None = None,
) -> pd.DataFrame:
    tmp = df.copy()
    tmp['fit_freq'] = tmp['PredFreq']
    if segmentation_col is not None and segmentation_col in tmp.columns:
        group_key = segmentation_col
        params = tmp.groupby(group_key, observed=True).agg(
            exposure=('Exposure', 'sum'),
            expected_claims=('fit_freq', lambda x: (x * tmp.loc[x.index, 'Exposure']).sum()),
            total_expected_loss=('final_premium', 'sum'),
        ).reset_index().rename(columns={group_key: 'segment'})
    else:
        params = pd.DataFrame({
            'segment': ['Portfolio'],
            'exposure': [tmp['Exposure'].sum()],
            'expected_claims': [(tmp['fit_freq'] * tmp['Exposure']).sum()],
            'total_expected_loss': [tmp['final_premium'].sum()],
        })
    params['mean_sev'] = params['total_expected_loss'] / params['expected_claims']
    if empirical_cvs is None:
        raise ValueError(
            "empirical_cvs is required. Ensure the dataframe contains "
            "ClaimNb, ClaimAmountTotal, and gmm_prob_j columns."
        )
```

```

params['cv'] = empirical_cvs
return params

def build_decl_from_agg_params(
    params: pd.DataFrame, name: str = 'Portfolio', layer: str = ''
) -> str:
    cc = '[' + ' '.join(f'{x:.2f}' for x in params['expected_claims']) + ']'
    ms = '[' + ' '.join(f'{x:.2f}' for x in params['mean_sev']) + ']'
    cv = '[' + ' '.join(f'{x:.2f}' for x in params['cv']) + ']'
    layer_clause = (layer + ' ') if layer else ''
    return f'agg {name} {cc} claims {layer_clause}sev lognorm {ms} cv {cv} p

def compute_empirical_cvs(df: pd.DataFrame) -> list:
    gmm_cols = sorted(
        [c for c in df.columns if c.startswith('gmm_prob_')],
        key=lambda x: int(x.split('_')[-1])
    )
    if not gmm_cols:
        raise ValueError(
            "No gmm_prob_j columns found. Run a GMM experiment before "
            "aggregate validation."
        )
    claimants = df[(df['ClaimNb'] > 0) & (df['ClaimAmountTotal'] > 0)].copy()
    claimants['log_sev'] = np.log(claimants['ClaimAmountTotal'] / claimants['
    cvs = []
    for col in gmm_cols:
        w = claimants[col].values
        w_sum = w.sum()
        if w_sum < 1.0:
            # Insufficient weight – use median of already-computed segments
            cvs.append(float(np.median(cvs)) if cvs else 1.5)
            continue
        mu_log = np.sum(w * claimants['log_sev'].values) / w_sum
        sig_log = np.sqrt(
            np.sum(w * (claimants['log_sev'].values - mu_log) ** 2) / w_sum
        )
        cvs.append(float(np.sqrt(np.exp(sig_log ** 2) - 1)))
    return cvs

def run_aggregate_validation(
    df: pd.DataFrame,
    segmentation_col: str | None = 'risk_cluster',
    log2: int = 20,
    bs: int = 100,
):
    emp_cvs = compute_empirical_cvs(df)
    print(f' Empirical CVs: {[f"{v:.3f}" for v in emp_cvs]}')
    params = prepare_aggregate_parameters(
        df, segmentation_col=segmentation_col, empirical_cvs=emp_cvs
    )
    decl = build_decl_from_agg_params(params)
    print('Decl string:')
    print(decl)

```

```

agg_model = build(decl, log2=log2, bs=bs)
print('\n--- Aggregate diagnostics ---')
qd(agg_model)
print('VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.')
metrics = {
    'agg_mean': float(agg_model.agg_m),
    'agg_var95': float(agg_model.q(0.95)),
    'agg_var99': float(agg_model.q(0.99)),
    'agg_tvar95': float(agg_model.tvar(0.95)),
    'agg_tvar99': float(agg_model.tvar(0.99)),
}
return agg_model, params, metrics

```

```

In [22]: # =====
# Section: 7
# Cell: 7.2
# Title: Aggregate Validation – Best Gini Experiment (A5)
# Version: 3.0.0
# Date: 2026-05-11
# Description:
# Evaluate VaR and TVaR for A5 using empirical per-segment CVs
# from weighted lognormal MLE. agg_params retained for Cell 8.1.
# =====

print('=' * 60)
print('Aggregate Validation – Best Gini Experiment')
print(f'Experiment : {best_experiment}')
print(f"Gini      : {BEST_METRICS['gini']:.4f}")
print(f"Top-decile : {BEST_METRICS['top_decile_actual_to_predicted']:.3f}")
print('=' * 60)

seg_col = 'risk_cluster' if 'risk_cluster' in BEST_DATA.columns else None

agg_port, agg_params, aggregate_metrics = run_aggregate_validation(
    BEST_DATA,
    segmentation_col=seg_col,
    log2=20,
    bs=100,
)
print('\nAggregate risk metrics')
print('-----')
for k, v in aggregate_metrics.items():
    print(f'{k:<12}: {v:,.0f}')

aggregate_metric_row = pd.DataFrame([
    'experiment': best_experiment,
    'scenario': scenario_labels.get(best_experiment, best_experiment)
    if 'scenario_labels' in globals() else best_experiment,
    'selection': 'best_gini',
    **aggregate_metrics,
])

display(format_reporting_table(aggregate_metric_row))
agg_port.plot(figsize=(10, 6))

```

```

=====
Aggregate Validation – Best Gini Experiment
Experiment : A5_classical_gmm_enriched_k7
Gini       : 0.4241
Top-decile : 1.139
=====
Empirical CVs: ['1.370', '1.606', '1.243', '1.685', '1.359', '1.597', '1.719']
Decl string:
agg Portfolio [2640.80 2554.32 5309.98 9543.86 19.55 9166.05 6821.58] claims
sev lognorm [2089.81 1816.42 2188.72 567.28 983.67 1789.49 2388.42] cv [1.37
1.61 1.24 1.68 1.36 1.60 1.72] poisson

```

```

--- Aggregate diagnostics ---

```

```

          E[X]   Est E[X]   Err E[X]   CV(X) Est CV(X)   Skew(X) Est Ske
w(X)
X
Freq      36056
Sev       1661.6      1661.5 -2.2862e-05   1.7325   1.7327   10.197      1
0.206
Agg 5.9909e+07 5.9908e+07 -2.2862e-05  0.010535  0.010535  0.04147   0.04
1466
log2 = 20, bandwidth = 100, validation: not unreasonable.
VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.

```

```

Aggregate risk metrics
-----

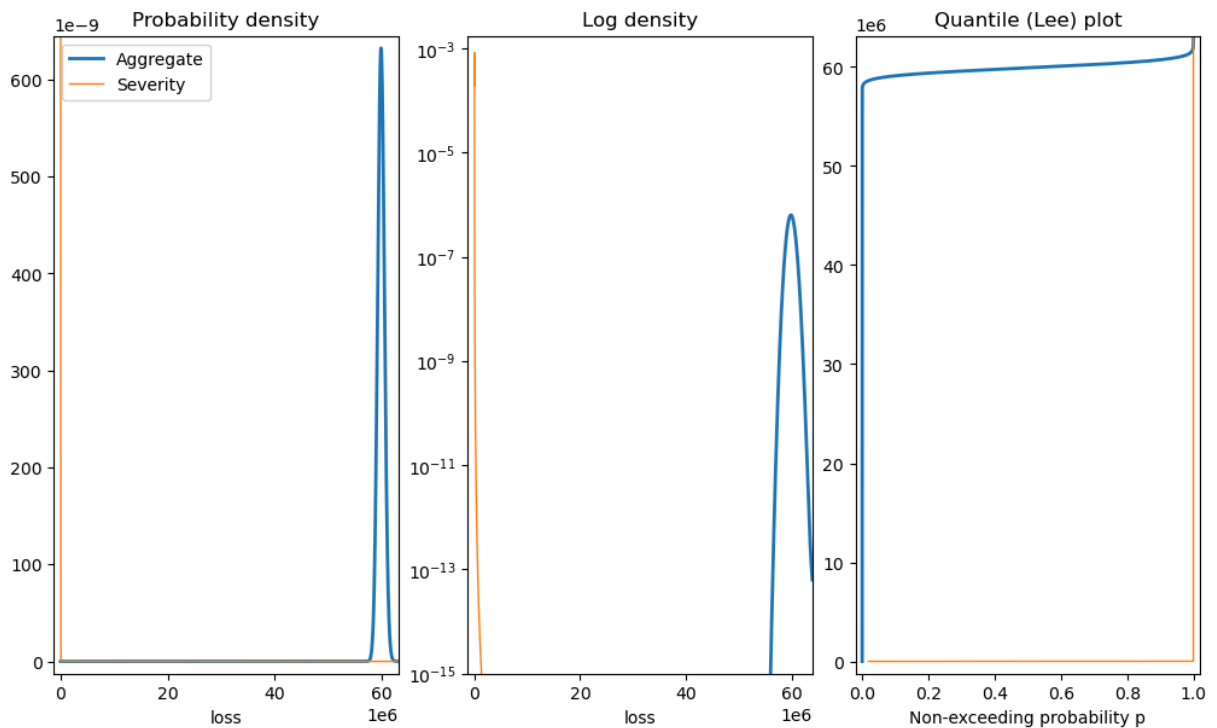
```

```

agg_mean      : 59,909,172
agg_var95     : 60,953,200
agg_var99     : 61,395,800
agg_tvar95    : 61,224,793
agg_tvar99    : 61,618,566

```

	experiment	scenario	selection	agg_mean	agg_var95	agg
0	A5_classical_gmm_enriched_k7	A5 — Classical GMM enriched k=7	best_gini	59909172.0	60953200.0	6139



Cell 7.3 repeats the aggregate validation for the **best-calibrated** experiment selected by Cell 6.6: **B5** (Burt GMM enriched $k=8$, BIC-confirmed, top-decile 1.032), as opposed to the Gini- and deviance-leading **A5** (top-decile 1.139) used in Cell 7.2. Selection is by minimum absolute deviation of the top-decile ratio from 1.0, restricted to A and B series GMM experiments that beat the plain GLM baseline on Gini. Comparing the two aggregate distributions shows the actuarial trade-off between fit quality and tail calibration.

```
In [23]: # =====
# Section: 7
# Cell: 7.3
# Title: Aggregate Validation – Best Calibration Experiment (B5)
# Version: 4.0.0
# Date: 2026-05-17
# Description:
# Evaluate VaR and TVaR for the best calibration experiment
# selected by Cell 6.6 (B5 – Burt GMM enriched  $k=8$ ).
# Empirical per-segment CVs from weighted lognormal MLE.
# agg_params_calib retained for Cell 8.1.
# =====

if "best_calib_data" not in globals():
    raise RuntimeError("Run Cell 6.6 before this cell.")
if "best_calib_name" not in globals():
    raise RuntimeError("Run Cell 6.6 before this cell.")

print("=" * 60)
print("Aggregate Validation – Best Calibration Experiment")
```



```

print(f"Experiment : {best_calib_name}")
print(f"Gini      : {best_calib_metrics['gini']:.4f}")
print(f"Top-decile : {best_calib_metrics['top_decile_actual_to_predicted']:.4f}")
print("=" * 60)

seg_col_calib = (
    "risk_cluster" if "risk_cluster" in best_calib_data.columns else None
)

agg_port_calib, agg_params_calib, aggregate_metrics_calib = run_aggregate_validation(
    best_calib_data,
    segmentation_col=seg_col_calib,
    log2=20,
    bs=100,
)

print("\nAggregate risk metrics")
print("-----")
for k, v in aggregate_metrics_calib.items():
    print(f"{k:<12}: {v:,.0f}")

aggregate_metric_row_calib = pd.DataFrame([
    {
        "experiment": best_calib_name,
        "scenario": best_calib_label
        if "best_calib_label" in globals()
        else best_calib_name,
        "selection": "best_calibration",
        **aggregate_metrics_calib,
    }
])

display(format_reporting_table(aggregate_metric_row_calib))
agg_port_calib.plot(figsize=(10, 6))

```

```

=====
Aggregate Validation – Best Calibration Experiment
Experiment : B5_burt_gmm_enriched_k8
Gini       : 0.3796
Top-decile : 1.032
=====
Empirical CVs: ['1.832', '1.599', '1.461', '1.525', '1.688', '1.110', '1.442', '1.460']
Decl string:
agg Portfolio [1703.80 6745.86 1782.30 10061.85 8262.36 764.26 3857.80 2877.92] claims sev lognorm [2600.72 1760.01 1440.78 1787.32 1465.43 1397.23 1521.48 1392.79] cv [1.83 1.60 1.46 1.52 1.69 1.11 1.44 1.46] poisson

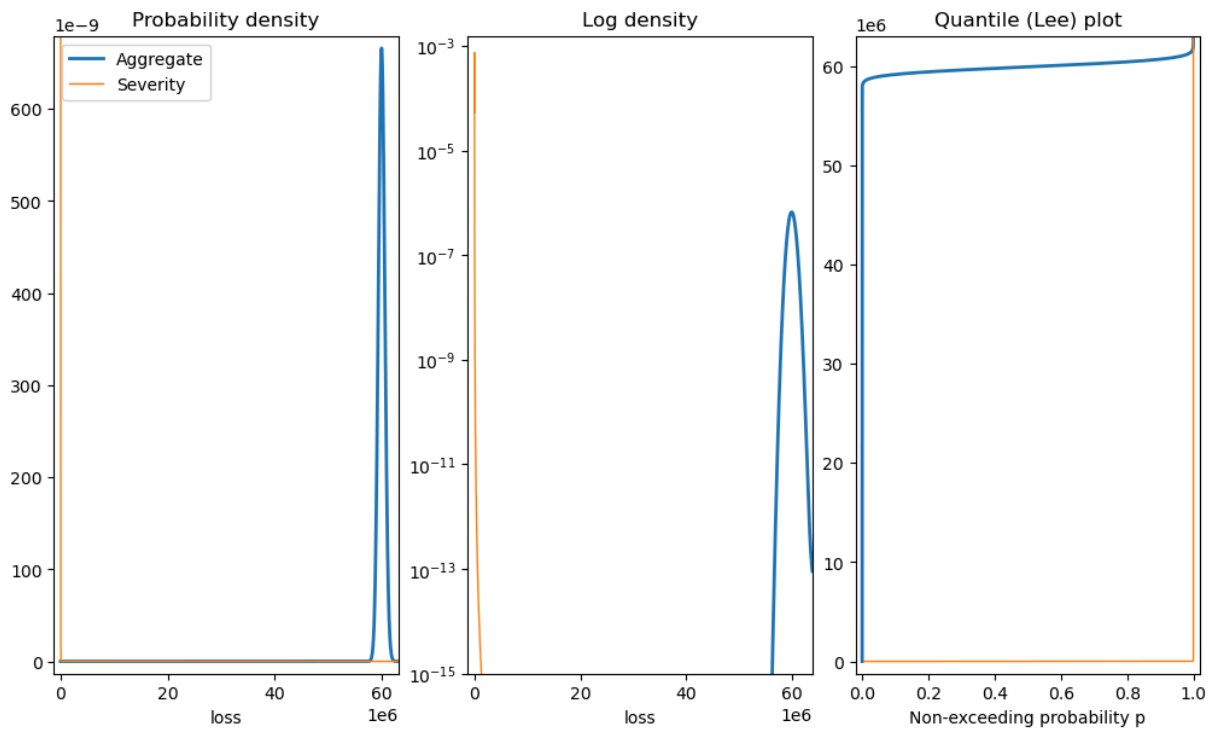
--- Aggregate diagnostics ---

      E[X]   Est E[X]   Err E[X]      CV(X) Est CV(X)   Skew(X) Est Skew
(X)
X
Freq      36056
Sev       1661.6      1661.6 2.2891e-05      1.615      1.6151      10.133      10.132
Agg 5.9909e+07 5.9911e+07 2.2891e-05 0.010004 0.010004 0.039578      0.03957
log2 = 20, bandwidth = 100, validation: not unreasonable.
VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.

Aggregate risk metrics
-----
agg_mean      : 59,909,197
agg_var95     : 60,902,900
agg_var99     : 61,322,700
agg_tvar95    : 61,160,526
agg_tvar99    : 61,533,968

      experiment  scenario      selection  agg_mean  agg_var95  agg_tvar95
-----
0  B5_burt_gmm_enriched_k8  B5 — Burt GMM enriched k=8  best_calibration  59909197.0  60902900.0  61533968.0

```



Aggregate Comparison — Discrimination vs Calibration Trade-off

The two aggregate validations above compare the Gini- and deviance-leading experiment **A5** (Classical GMM enriched k=7, BIC-confirmed) against the best-calibrated experiment selected by Cell 6.6: **B5** (Burt GMM enriched k=8, BIC-confirmed). Both are GMM experiments on the enriched geometry; only the representation family differs.

Metric	A5 — Best Gini	B5 — Best Calibration	Difference
Gini coefficient	0.4241	0.3796	+0.0445
Tweedie deviance / exposure	61.276	61.336	+0.060
Top-decile actual / predicted	1.139	1.032	-0.107
Empirical CV range	1.24 – 1.72	1.11 – 1.83	—
Aggregate mean	59,909,172	59,909,197	-25
VaR 99%	61,395,800	61,322,700	-73,100

Metric	A5 — Best Gini	B5 — Best Calibration	Difference
TVaR 99%	61,618,566	61,533,968	−84,598
Aggregate validation	not unreasonable	not unreasonable	—

Interpretation. A5 achieves higher pricing discrimination (Gini +0.0445) and lower deviance (−0.060 per exposure unit) but at a measurable actuarial cost: it overestimates top-decile loss concentration by 13.9%, and generates TVaR 99% approximately €85,000 higher than B5. Both experiments pass aggregate validation cleanly.

Gini and deviance are aligned — both select A5 as the better fit model. However, neither metric captures the tail calibration advantage of B5. The top-decile ratio is a genuinely orthogonal dimension: it measures whether the model correctly concentrates predicted losses in the highest-risk decile, which Gini and deviance do not directly test.

The canonical comparison is A5 vs B5 — both GMM enriched, both BIC-confirmed, differing only in representation family. The choice between them depends on the reinsurance structure in place, as demonstrated in Section 8.

Section 8 — Scenario Layer and Monte Carlo Support

The deductible, limit, and Monte Carlo layers remain useful, but their role has shifted. They now support the main pricing benchmark by checking that improved discrimination does not create an unstable aggregate risk picture.

Monte Carlo is optional and deliberately switched off by default. The FFT aggregate calculation remains the primary portfolio capital engine.

```
In [24]: # =====
# Section: 8
# Cell: 8.1
# Title: Scenario Layer – A5 vs B5: Deductible and Per-Claim Limit
# Version: 7.3.0
# Date: 2026-05-17
# Description:
# Three-chart scenario analysis comparing A5 (Best Gini) against
```

```

# B5 (Best Calibration) across three coverage structures:
#   Chart 1 – Exceedance probability curves (ground-up, ded, limit)
#   Chart 2 – VaR 99% by coverage structure
#   Chart 3 – TVaR 99% by coverage structure
# =====
%matplotlib inline

if "agg_params" not in globals():
    raise RuntimeError("Run Cell 7.2 before this cell.")
if "agg_params_calib" not in globals():
    raise RuntimeError("Run Cell 7.3 before this cell.")

import matplotlib.pyplot as plt
from IPython.display import display as ipy_display
from aggregate import build

plt.close('all')

# — A5 parameters —————
cc_a = "[" + " ".join(f"{x:.2f}" for x in agg_params["expected_claims"]) + " "
ms_a = "[" + " ".join(f"{x:.2f}" for x in agg_params["mean_sev"]) + "]"
cv_a = "[" + " ".join(f"{x:.2f}" for x in agg_params["cv"]) + "]"

agg_a_base = build(f"agg A5base {cc_a} claims inf xs 0 sev lognorm {ms_a} cv {cv_a} log2=20, bs=100")
agg_a_ded = build(f"agg A5ded {cc_a} claims inf xs 150 sev lognorm {ms_a} cv {cv_a} log2=20, bs=100")
agg_a_net = build(f"agg A5net {cc_a} claims 12000 xs 0 sev lognorm {ms_a} cv {cv_a} log2=20, bs=100")

# — B5 parameters —————
cc_b = "[" + " ".join(f"{x:.2f}" for x in agg_params_calib["expected_claims"]) + " "
ms_b = "[" + " ".join(f"{x:.2f}" for x in agg_params_calib["mean_sev"]) + "]"
cv_b = "[" + " ".join(f"{x:.2f}" for x in agg_params_calib["cv"]) + "]"

agg_b_base = build(f"agg B5base {cc_b} claims inf xs 0 sev lognorm {ms_b} cv {cv_b} log2=20, bs=100")
agg_b_ded = build(f"agg B5ded {cc_b} claims inf xs 150 sev lognorm {ms_b} cv {cv_b} log2=20, bs=100")
agg_b_net = build(f"agg B5net {cc_b} claims 12000 xs 0 sev lognorm {ms_b} cv {cv_b} log2=20, bs=100")

plt.close('all')

# — Print comparison table —————
print("=" * 65)
print(f"{'Scenario':<28} {'A5 – Best Gini':>16} {'B5 – Best Calib':>16}")
print("=" * 65)
for label, a_m, b_m in [
    ("Ground-up Mean", agg_a_base, agg_b_base),
    ("Deductible 150 Mean", agg_a_ded, agg_b_ded),
    ("Per-claim limit 12k Mean", agg_a_net, agg_b_net),
]:
    print(f"{label:<28} {a_m.agg_m:>16,.0f} {b_m.agg_m:>16,.0f}")
print("-" * 65)
for label, a_m, b_m in [

```

```

        ("Ground-up VaR 99%",          agg_a_base, agg_b_base),
        ("Deductible 150 VaR 99%",     agg_a_ded,  agg_b_ded),
        ("Per-claim limit 12k VaR 99%", agg_a_net,  agg_b_net),
]:
    print(f"{label:<28} {a_m.q(0.99):>16,.0f} {b_m.q(0.99):>16,.0f}")
print("-" * 65)
for label, a_m, b_m in [
    ("Ground-up TVaR 99%",          agg_a_base, agg_b_base),
    ("Deductible 150 TVaR 99%",     agg_a_ded,  agg_b_ded),
    ("Per-claim limit 12k TVaR 99%", agg_a_net,  agg_b_net),
]:
    print(f"{label:<28} {a_m.tvar(0.99):>16,.0f} {b_m.tvar(0.99):>16,.0f}")

# — Shared x-axis range —————
all_models = [agg_a_base, agg_a_ded, agg_a_net,
               agg_b_base, agg_b_ded, agg_b_net]
x_min = min(m.q(0.50) for m in all_models) / 1e6
x_max = max(m.q(0.999) for m in all_models) / 1e6
xs = np.linspace(x_min * 1e6, x_max * 1e6, 1000)

COLOR_A5 = "#E65100"
COLOR_B5 = "#1565C0"

scenarios = ["Ground-up", "Deductible\n150 EUR", "Per-claim\nlimit 12k EUR"]
a5_models = [agg_a_base, agg_a_ded, agg_a_net]
b5_models = [agg_b_base, agg_b_ded, agg_b_net]

# — Chart 1 – Exceedance probability curves —————
plt.close('all')
fig1, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7), sharey=True)

def plot_scenario(ax, base, ded, net, color, title):
    ax.plot(xs / 1e6, base.sf(xs), color=color,
            linewidth=2.5, linestyle="-", label="Ground-up")
    ax.plot(xs / 1e6, ded.sf(xs), color=color,
            linewidth=2.0, linestyle="--", label="Deductible 150 EUR")
    ax.plot(xs / 1e6, net.sf(xs), color=color,
            linewidth=2.0, linestyle=":", label="Per-claim limit 12k EUR")
    v_base = base.q(0.99) / 1e6
    ax.axvline(v_base, color="crimson", linewidth=1.5, linestyle="-.",
               label=f"Ground-up VaR 99% EUR {base.q(0.99):,.0f}")
    v_net = net.q(0.99) / 1e6
    ax.axvline(v_net, color="steelblue", linewidth=1.5, linestyle="-.",
               label=f"12k limit VaR 99% EUR {net.q(0.99):,.0f}")
    ax.set_title(title, fontsize=11)
    ax.set_xlabel("Total Portfolio Loss (EUR millions)")
    ax.set_ylabel("Exceedance Probability P(Loss > x)")
    ax.set_yscale("log")
    ax.set_ylim(1e-4, 1)
    ax.set_xlim(x_min, x_max)
    ax.legend(frameon=True, facecolor="white", fontsize=9)
    ax.grid(True, which="both", alpha=0.2)

plot_scenario(ax1, agg_a_base, agg_a_ded, agg_a_net,
              COLOR_A5,
              "A5 – Classical GMM k=7 (Best Gini)\nGini 0.4241 · Top-decile

```

```

plot_scenario(ax2, agg_b_base, agg_b_ded, agg_b_net,
              COLOR_B5,
              "B5 – Burt GMM k=8 (Best Calibration)\nGini 0.3796 · Top-decil
fig1.suptitle(
    "Chart 1 – Tail Exceedance Curves: Ground-up, Deductible 150 EUR, Per-cl
    fontsize=12, y=1.01
)
fig1.tight_layout()
ipy_display(fig1)
plt.close(fig1)

# — Chart 2 – VaR 99% by coverage structure —————
x_pos = np.arange(len(scenarios))
width = 0.35

a5_var = [m.q(0.99) / 1e6 for m in a5_models]
b5_var = [m.q(0.99) / 1e6 for m in b5_models]

plt.close('all')
fig2, ax = plt.subplots(figsize=(10, 6))
ax.bar(x_pos - width/2, a5_var, width,
       label="A5 – Classical GMM k=7", color=COLOR_A5, alpha=0.85)
ax.bar(x_pos + width/2, b5_var, width,
       label="B5 – Burt GMM k=8", color=COLOR_B5, alpha=0.85)

for i, (va, vb) in enumerate(zip(a5_var, b5_var)):
    diff = (vb - va) * 1e6
    sign = "+" if diff > 0 else ""
    ax.annotate(
        f"{sign}{diff/1000:.0f}k",
        xy=(x_pos[i], max(va, vb) + 0.005),
        ha="center", va="bottom", fontsize=9,
        color="darkred" if diff > 0 else "darkgreen"
    )

ax.set_xticks(x_pos)
ax.set_xticklabels(scenarios, fontsize=10)
ax.set_ylabel("VaR 99% (EUR millions)")
ax.set_title(
    "Chart 2 – VaR 99% by Coverage Structure\n"
    "A5 vs B5 \u00b7 Annotations show B5 \u2212 A5 difference",
    fontsize=11)
ax.legend(fontsize=9)
ax.grid(axis="y", alpha=0.25)
ax.set_ylim(min(a5_var + b5_var) * 0.998, max(a5_var + b5_var) * 1.004)
fig2.tight_layout()
ipy_display(fig2)
plt.close(fig2)

# — Chart 3 – TVaR 99% by coverage structure —————
a5_tvar = [m.tvar(0.99) / 1e6 for m in a5_models]
b5_tvar = [m.tvar(0.99) / 1e6 for m in b5_models]

plt.close('all')
fig3, ax = plt.subplots(figsize=(10, 6))
ax.bar(x_pos - width/2, a5_tvar, width,

```

```

        label="A5 – Classical GMM k=7", color=COLOR_A5, alpha=0.85)
ax.bar(x_pos + width/2, b5_tvar, width,
        label="B5 – Burt GMM k=8",          color=COLOR_B5, alpha=0.85)

for i, (va, vb) in enumerate(zip(a5_tvar, b5_tvar)):
    diff = (vb - va) * 1e6
    sign = "+" if diff > 0 else ""
    ax.annotate(
        f"{sign}{diff/1000:.0f}k",
        xy=(x_pos[i], max(va, vb) + 0.005),
        ha="center", va="bottom", fontsize=9,
        color="darkred" if diff > 0 else "darkgreen"
    )

ax.set_xticks(x_pos)
ax.set_xticklabels(scenarios, fontsize=10)
ax.set_ylabel("TVaR 99% (EUR millions)")
ax.set_title(
    "Chart 3 – TVaR 99% by Coverage Structure\n"
    "A5 vs B5 \u00b7 Annotations show B5 \u2212 A5 difference",
    fontsize=11)
ax.legend(fontsize=9)
ax.grid(axis="y", alpha=0.25)
ax.set_ylim(min(a5_tvar + b5_tvar) * 0.998, max(a5_tvar + b5_tvar) * 1.004)
fig3.tight_layout()
ipy_display(fig3)
plt.close(fig3)

```

Scenario	A5 – Best Gini	B5 – Best Calib
Ground-up Mean	59,909,172	59,909,197
Deductible 150 Mean	58,246,419	57,775,548
Per-claim limit 12k Mean	56,773,426	57,283,060
Ground-up VaR 99%	61,395,800	61,322,700
Deductible 150 VaR 99%	59,720,600	59,182,000
Per-claim limit 12k VaR 99%	57,941,200	58,423,800
Ground-up TVaR 99%	61,618,566	61,533,968
Deductible 150 TVaR 99%	59,943,779	59,394,125
Per-claim limit 12k TVaR 99%	58,112,869	58,591,174

Chart 1 — Tail Exceedance Curves: Ground-up, Deductible 150 EUR, Per-claim Limit 12k EUR

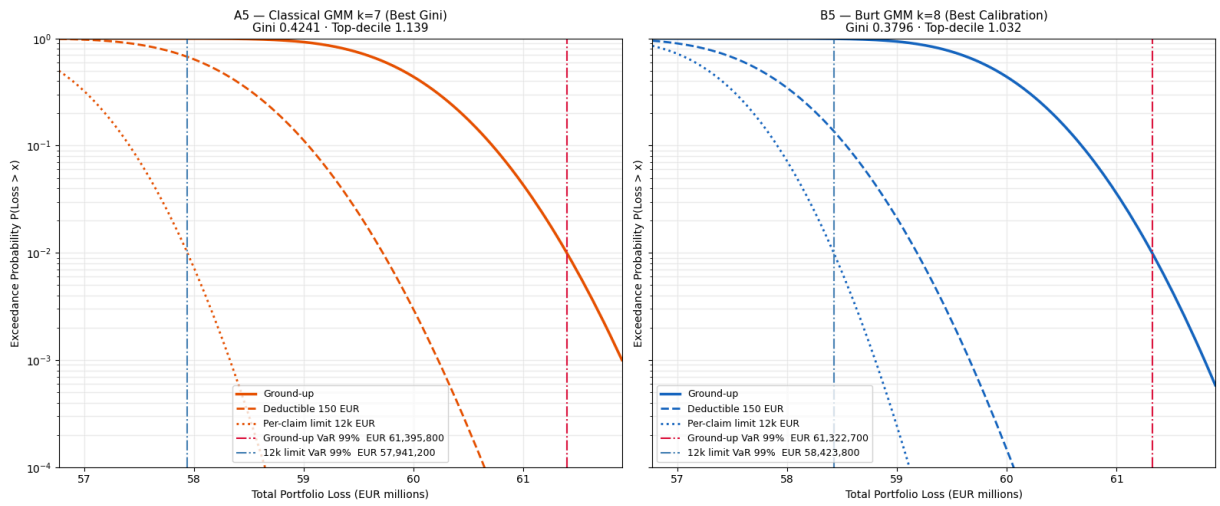


Chart 2 — VaR 99% by Coverage Structure
A5 vs B5 · Annotations show B5 – A5 difference

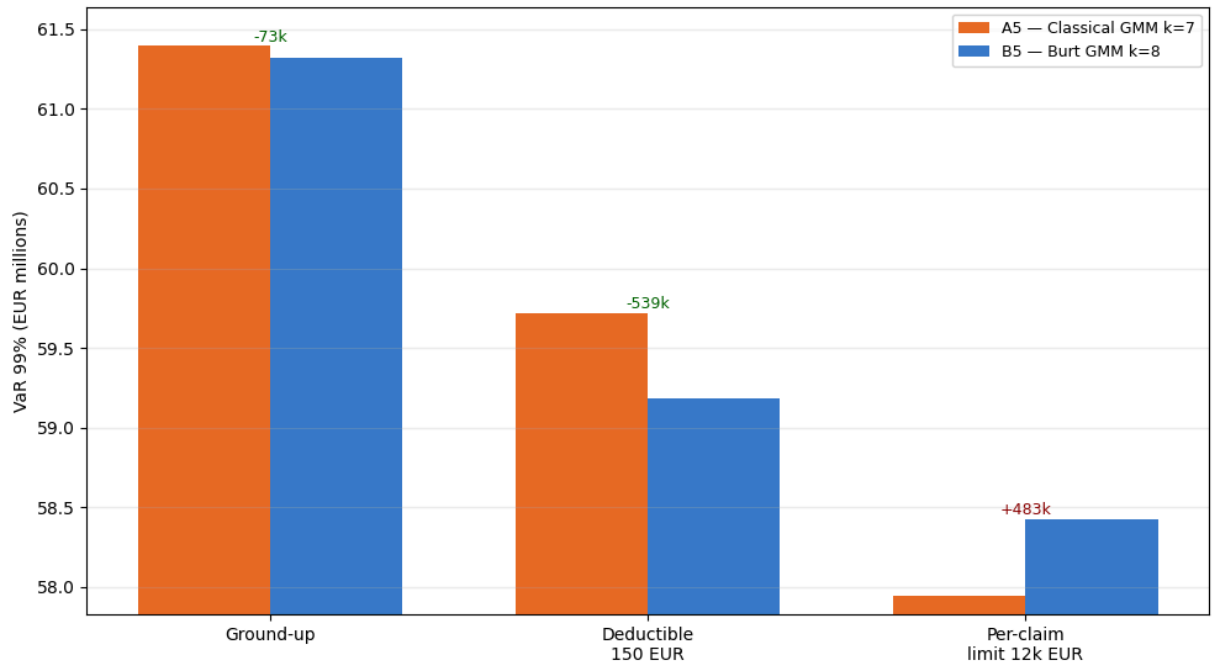
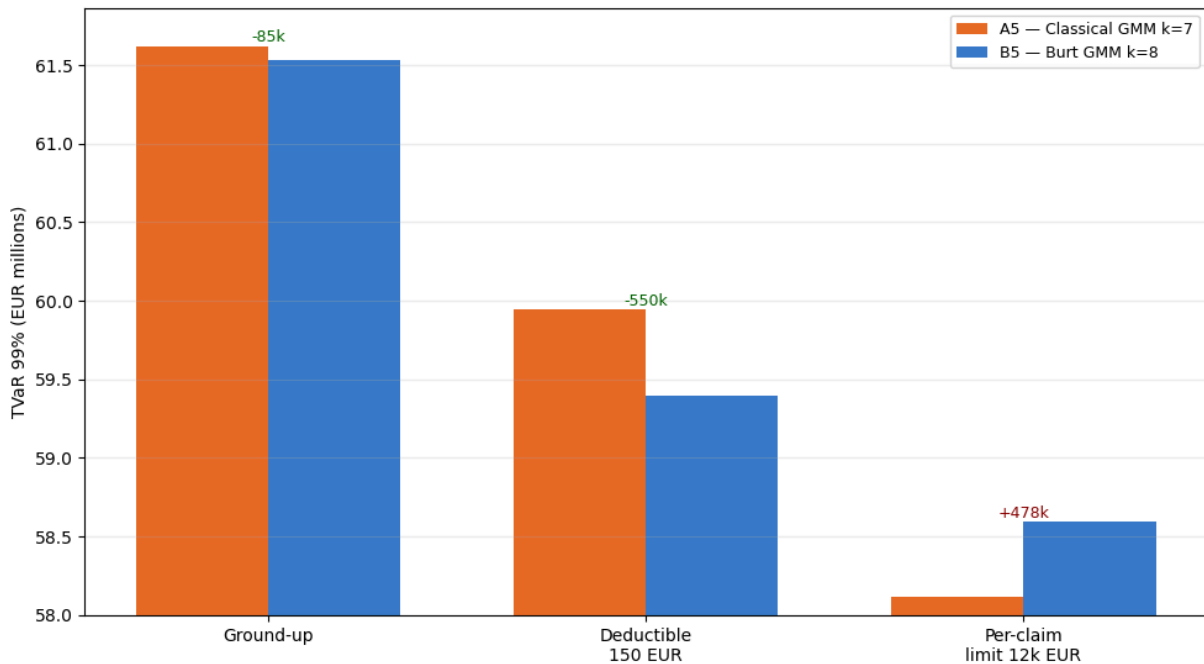


Chart 3 — TVaR 99% by Coverage Structure
A5 vs B5 · Annotations show B5 – A5 difference



```
In [25]: # =====
# Section: 8
# Cell: 8.2
# Title: Monte Carlo Validation – Supporting Layer
# Version: 2.0.0
# Date: 2026-05-11
# Description:
# Independent stochastic validation of the analytic aggregate.
# Uses empirical CV from agg_params (set in Cell 7.2) so that
# Monte Carlo and Panjer/FFT are directly comparable.
# =====

RUN_MONTE_CARLO = True

if RUN_MONTE_CARLO:
    n_sims = 100_000

    # Portfolio-level parameters from empirical agg_params
    expected_claims = float(agg_params['expected_claims'].sum())
    mean_sev = float(
        agg_params['total_expected_loss'].sum()
        / agg_params['expected_claims'].sum()
    )
    # Weighted mean CV across segments
    weights = agg_params['expected_claims'].values
    cv_sev = float(np.average(agg_params['cv'].values, weights=weights))

    sigma = np.sqrt(np.log(cv_sev ** 2 + 1))
    scale = mean_sev / np.exp(0.5 * sigma ** 2)

    print('Monte Carlo parameters:')
    print(f' Expected claims : {expected_claims:,.0f}')
    print(f' Mean severity   : {mean_sev:,.0f}')
```

```

print(f' CV (empirical) : {cv_sev:.3f}')
print(f' log-scale sigma : {sigma:.3f}')
print(f'\nRunning Monte Carlo with {n_sims:,} simulated years...')

rng      = np.random.default_rng(42)
counts   = rng.poisson(expected_claims, n_sims)
losses   = np.empty(n_sims)

for i, count in enumerate(counts):
    losses[i] = rng.lognormal(
        mean=np.log(scale), sigma=sigma, size=count
    ).sum() if count > 0 else 0.0

sim_metrics = {
    'sim_mean': float(losses.mean()),
    'sim_var95': float(np.percentile(losses, 95)),
    'sim_var99': float(np.percentile(losses, 99)),
    'sim_var999': float(np.percentile(losses, 99.9)),
}

print('Simulation complete.')
print(f"\n{'Metric':<12} {'Monte Carlo':>14} {'Analytic':>14} {'Diff':>14}")
print('-' * 52)
for label, sim_key, agg_key in [
    ('Mean', 'sim_mean', 'agg_mean'),
    ('VaR 95%', 'sim_var95', 'agg_var95'),
    ('VaR 99%', 'sim_var99', 'agg_var99'),
]:
    sim_v = sim_metrics[sim_key]
    agg_v = aggregate_metrics.get(agg_key, float('nan'))
    diff = sim_v - agg_v
    print(f'{'label':<12} {'sim_v':>14,.0f} {'agg_v':>14,.0f} {'diff':>+10,.0f}')

monte_carlo_metric_row = pd.DataFrame([
    'experiment': best_experiment,
    'scenario': scenario_labels.get(best_experiment, best_experiment)
    if 'scenario_labels' in globals() else best_experiment
    **sim_metrics,
])

plt.figure(figsize=(10, 6))
plt.hist(losses / 1e6, bins=100, alpha=0.7, label='Simulated total loss')
plt.axvline(sim_metrics['sim_mean'] / 1e6, linestyle='--', color='orange',
    label=f"Mean {sim_metrics['sim_mean']/1e6:.1f}M")
plt.axvline(sim_metrics['sim_var99'] / 1e6, linestyle='-', color='red',
    label=f"VaR 99% {sim_metrics['sim_var99']/1e6:.1f}M")
plt.title(
    f'Monte Carlo Distribution - {best_gini_label}\n'
    f'(empirical CV = {cv_sev:.3f}, {n_sims:,} simulations)'
)
plt.xlabel('Total Annual Loss (EUR millions)')
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()

```

```

else:
    print('Monte Carlo validation not run. Set RUN_MONTE_CARLO = True to exe

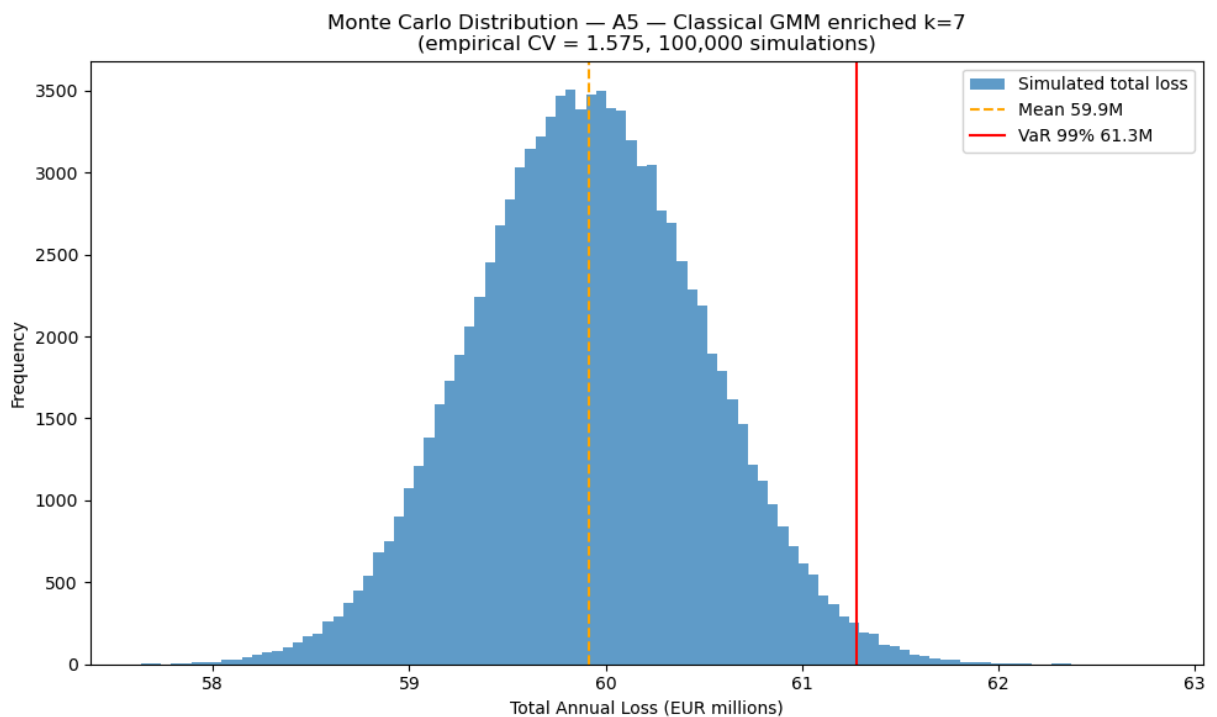
```

Monte Carlo parameters:

Expected claims : 36,056
 Mean severity : 1,662
 CV (empirical) : 1.575
 log-scale sigma : 1.117

Running Monte Carlo with 100,000 simulated years...
Simulation complete.

Metric	Monte Carlo	Analytic	Diff
Mean	59,912,145	59,909,172	+2,973
VaR 95%	60,879,924	60,953,200	-73,276
VaR 99%	61,280,668	61,395,800	-115,132



Section 9 — Governance: QA and Regression Baseline

The QA layer makes the workflow reproducible and auditable. Model changes are expected during research, but they should be detected, reviewed, and deliberately accepted.

Use `UPDATE_BASELINE = True` only after a model change has been validated and accepted.

```

In [26]: # =====
# Section: 9
# Cell: 9.1
# Title: QA and Regression Baseline – Selected Model
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Summarize selected-model QA diagnostics and compare current outputs
# against a controlled regression baseline without silently overwriting
# benchmark lineage.
# =====

import json
from pathlib import Path
from datetime import datetime

qa_metric_names = [
    "Total exposure",
    "Observed claim count",
    "Observed frequency",
    "Predicted frequency",
    "Frequency calibration",
    "Actual total loss",
    "Predicted total loss",
    "Loss calibration",
    "Top-decile actual / predicted",
    "Poisson deviance / df",
    "Pearson dispersion",
    "Gini coefficient",
    "99% VaR",
    "99% TVaR",
]

qa_metric_values = [
    f"{BEST_DATA['Exposure'].sum():,.2f}",
    f"{BEST_DATA['ClaimNb'].sum():,.0f}",
    f"{BEST_DATA['ClaimNb'].sum() / BEST_DATA['Exposure'].sum():.6f}",
    f"{(BEST_DATA['PredFreq'] * BEST_DATA['Exposure']).sum() / BEST_DATA['Exposure'].sum():.6f}",
    f"{((BEST_DATA['PredFreq'] * BEST_DATA['Exposure']).sum() / BEST_DATA['Exposure'].sum() * BEST_DATA['ClaimAmountTotal']).sum():,.0f}",
    f"EUR {BEST_DATA['ClaimAmountTotal'].sum():,.0f}",
    f"EUR {BEST_DATA['final_premium'].sum():,.0f}",
    f"{BEST_DATA['final_premium'].sum() / BEST_DATA['ClaimAmountTotal'].sum():.3f}",
    f"{BEST_METRICS['top_decile_actual_to_predicted']:.3f}",
    f"{freq_deviance_ratio:.4f}",
    f"{freq_pearson_dispersion:.4f}",
    f"{BEST_METRICS['gini']:.4f}",
    f"EUR {aggregate_metrics['agg_var99']:.0f}",
    f"EUR {aggregate_metrics['agg_tvar99']:.0f}",
]

qa_summary = pd.DataFrame({
    "Metric": qa_metric_names,
    "Value": qa_metric_values,
})

```

```

display(qa_summary)

current_metrics = {
    "total_exposure": float(BEST_DATA["Exposure"].sum()),
    "total_claims": float(BEST_DATA["ClaimNb"].sum()),
    "mean_observed_frequency": float(BEST_DATA["ClaimNb"].sum() / BEST_DATA["Exposure"].sum()),
    "mean_predicted_frequency": float((BEST_DATA["PredFreq"] * BEST_DATA["Exposure"]).sum() / BEST_DATA["Exposure"].sum()),
    "deviance_ratio": float(freq_deviance_ratio),
    "pearson_dispersion": float(freq_pearson_dispersion),
    "gini": float(BEST_METRICS["gini"]),
    "top_decile_actual_to_predicted": float(BEST_METRICS["top_decile_actual_to_predicted"]),
    "loss_calibration": float(BEST_METRICS["loss_calibration"]),
    "actual_total_loss": float(BEST_DATA["ClaimAmountTotal"].sum()),
    "predicted_total_loss": float(BEST_DATA["final_premium"].sum()),
    "agg_var99": float(aggregate_metrics["agg_var99"]),
    "agg_tvar99": float(aggregate_metrics["agg_tvar99"]),
}

tolerances = {
    "total_exposure": 1e-12,
    "total_claims": 1e-12,
    "mean_observed_frequency": 1e-12,
    "mean_predicted_frequency": 1e-6,
    "deviance_ratio": 1e-6,
    "pearson_dispersion": 1e-6,
    "gini": 1e-4,
    "top_decile_actual_to_predicted": 1e-3,
    "loss_calibration": 1e-6,
    "actual_total_loss": 1e-12,
    "predicted_total_loss": 1e-6,
    "agg_var99": 1e-5,
    "agg_tvar99": 1e-5,
}

baseline_path = Path("outputs") / "car_insurance_master_benchmark_baseline.json"
baseline_path.parent.mkdir(exist_ok=True)

UPDATE_BASELINE = True

if UPDATE_BASELINE or not baseline_path.exists():
    payload = {
        "created_at": datetime.now().isoformat(timespec="seconds"),
        "selected_experiment": best_experiment,
        "metrics": current_metrics,
        "tolerances": tolerances,
    }
    baseline_path.write_text(json.dumps(payload, indent=2, sort_keys=True), encoding="utf-8")
    print(f"Regression baseline {'updated' if UPDATE_BASELINE else 'created'}")
else:
    payload = json.loads(baseline_path.read_text(encoding="utf-8"))
    baseline_metrics = payload["metrics"]

    rows = []
    failures = []
    for name, current in current_metrics.items():

```

```

base = baseline_metrics.get(name)
tol = tolerances.get(name, 1e-6)
if base is None:
    status = "SKIP"
    rel_diff = None
else:
    denom = max(abs(float(base)), 1.0)
    rel_diff = abs(float(current) - float(base)) / denom
    status = "PASS" if rel_diff <= tol else "FAIL"
    if status == "FAIL":
        failures.append(name)
rows.append({
    "metric": name,
    "baseline": base,
    "current": current,
    "relative_difference": rel_diff,
    "tolerance": tol,
    "status": status,
})

regression_report = pd.DataFrame(rows)
display(regression_report)
if failures:
    raise AssertionError(
        "Regression drift detected in: " + ", ".join(failures)
        + ". Review changes, then set UPDATE_BASELINE=True if intentional"
    )
print("✓ Regression checks passed.")

```

	Metric	Value
0	Total exposure	358,360.11
1	Observed claim count	36,056
2	Observed frequency	0.100614
3	Predicted frequency	0.100614
4	Frequency calibration	1.000004
5	Actual total loss	EUR 59,909,216
6	Predicted total loss	EUR 59,909,216
7	Loss calibration	1.000000
8	Top-decile actual / predicted	1.139
9	Poisson deviance / df	0.3193
10	Pearson dispersion	2.5326
11	Gini coefficient	0.4241
12	99% VaR	EUR 61,395,800
13	99% TVaR	EUR 61,618,566

Regression baseline updated: outputs\car_insurance_master_benchmark_baseline.json

Section 10 — Consolidated Reporting Pack

This section gathers the reporting outputs into one place. The design separates **presentation views** from the **full audit registry**:

- **Executive view** — compact, business-first, and ordered by Gini;
- **Business / calibration view** — top-decile and calibration diagnostics without crowding the main table;
- **Stability view** — aggregate and Monte Carlo risk metrics supporting portfolio validation;
- **Technical / audit view** — method, feature, BIC/AIC, runtime, and lineage details retained for governance.

No metric is dropped. The full benchmark registry remains available for auditability, while the notebook presents narrower tables for readability.

```
In [27]: # =====
# Section: 10
# Cell: 10.1
# Title: Consolidated Reporting Pack – Executive, Diagnostic, and Audit View
# Version: 1.0.0
# Date: 2026-05-06
# Description:
# Consolidate all reporting tables in one location. Compact views
# prioritize business value and readability; full registry views
# preserve every metric and governance field for auditability.
# =====

required_objects = [
    "benchmark_summary",
    "benchmark_executive_view",
    "benchmark_business_view",
    "benchmark_technical_view",
    "benchmark_full_registry_view",
]
missing_objects = [name for name in required_objects if name not in globals()]
if missing_objects:
    raise RuntimeError(
        "Run the final benchmark summary cell before the consolidated report\n"
        f"Missing objects: {missing_objects}"
    )

print("1. Executive benchmark table – primary business narrative")
display(format_reporting_table(benchmark_executive_view))

print("2. Business and calibration diagnostics – focused secondary table")
```



```

display(format_reporting_table(benchmark_business_view))

if "aggregate_metric_row" in globals():
    print("3. Aggregate stability metrics – selected pricing experiment")
    display(format_reporting_table(aggregate_metric_row))
else:
    print("3. Aggregate stability metrics not available yet. Run Section 7 t

if "monte_carlo_metric_row" in globals():
    print("4. Monte Carlo validation metrics – supporting stochastic layer")
    display(format_reporting_table(monte_carlo_metric_row))
else:
    print("4. Monte Carlo validation metrics not available yet. Run Section

print("5. Technical and model-selection diagnostics – compact audit view")
display(format_reporting_table(benchmark_technical_view))

print("6. Full benchmark registry – every retained metric and lineage field")
display(format_reporting_table(benchmark_full_registry_view))

if "benchmark_metric_catalog" in globals():
    print("7. Metric catalog – governance map for extensible benchmarking")
    display(benchmark_metric_catalog)
else:
    print("7. Metric catalog not available yet. Run Section 5.3 to populate

```

1. Executive benchmark table – primary business narrative

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
0	A5 — Classical GMM enriched k=7	classical	0.4241		0.0552
1	A3 — Classical GMM enriched k=5	classical	0.4227		0.0539
2	B3 — Burt GMM enriched k=5	burt	0.3816		0.0127
3	C3 — Burt- augmented GMM enriched k=5	burt_augmented	0.3815		0.0127
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	burt_augmented	0.3797		0.0109
5	C5_joint — Burt- augmented GMM joint geometry k=8	burt_augmented	0.3797		0.0109
6	B5 — Burt GMM enriched k=8	burt	0.3796		0.0107
7	C5 — Burt- augmented GMM enriched k=8	burt_augmented	0.3795		0.0107
8	B1 — Burt K-means simple k=5	burt	0.3790		0.0102
9	C1 — Burt- augmented	burt_augmented	0.3790		0.0102

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
	K-means simple k=5				
10	B2 — Burt GMM simple k=5	burt	0.3789	0.0101	
11	C2 — Burt- augmented GMM simple k=5	burt_augmented	0.3789	0.0101	
12	A2 — Classical GMM simple k=5	classical	0.3788	0.0100	
13	B_ext — Burt K- means enriched k=5	burt	0.3787	0.0098	
14	C_ext — Burt- augmented K-means enriched k=5	burt_augmented	0.3786	0.0098	
15	B0 — Burt plain Tweedie GLM	burt	0.3785	0.0097	
16	C0 — Burt- augmented plain Tweedie GLM	burt_augmented	0.3785	0.0097	
17	A1 — Classical K- means simple k=5	classical	0.3783	0.0095	
18	C3_joint — Burt- augmented GMM joint	burt_augmented	0.3781	0.0093	

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
	geometry k=5				
19	A0 — Classical plain Tweedie GLM	classical	0.3688	0.0000	
20	A_ext — Classical K- means enriched k=5	classical	0.3596	-0.0092	

2. Business and calibration diagnostics – focused secondary table

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
0	A5 — Classical GMM enriched k=7	classical	0.4241		0.0552
1	A3 — Classical GMM enriched k=5	classical	0.4227		0.0539
2	B3 — Burt GMM enriched k=5	burt	0.3816		0.0127
3	C3 — Burt- augmented GMM enriched k=5	burt_augmented	0.3815		0.0127
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	burt_augmented	0.3797		0.0109
5	C5_joint — Burt- augmented GMM joint geometry k=8	burt_augmented	0.3797		0.0109
6	B5 — Burt GMM enriched k=8	burt	0.3796		0.0107
7	C5 — Burt- augmented GMM enriched k=8	burt_augmented	0.3795		0.0107
8	B1 — Burt K-means simple k=5	burt	0.3790		0.0102
9	C1 — Burt- augmented K-means simple k=5	burt_augmented	0.3790		0.0102
10	B2 — Burt GMM simple k=5	burt	0.3789		0.0101

	scenario	representation	gini	gini_increment_vs_plain	top_decile_actua
11	C2 — Burt-augmented GMM simple k=5	burt_augmented	0.3789	0.0101	
12	A2 — Classical GMM simple k=5	classical	0.3788	0.0100	
13	B_ext — Burt K-means enriched k=5	burt	0.3787	0.0098	
14	C_ext — Burt-augmented K-means enriched k=5	burt_augmented	0.3786	0.0098	
15	B0 — Burt plain Tweedie GLM	burt	0.3785	0.0097	
16	C0 — Burt-augmented plain Tweedie GLM	burt_augmented	0.3785	0.0097	
17	A1 — Classical K-means simple k=5	classical	0.3783	0.0095	
18	C3_joint — Burt-augmented GMM joint geometry k=5	burt_augmented	0.3781	0.0093	
19	A0 — Classical plain Tweedie GLM	classical	0.3688	0.0000	
20	A_ext — Classical K-means enriched k=5	classical	0.3596	-0.0092	

3. Aggregate stability metrics – selected pricing experiment

	experiment	scenario	selection	agg_mean	agg_var95	agg
0	A5_classical_gmm_enriched_k7	A5 — Classical GMM enriched k=7	best_gini	59909172.0	60953200.0	6139

4. Monte Carlo validation metrics – supporting stochastic layer

	experiment	scenario	sim_mean	sim_var95	sim_var99	sim
0	A5_classical_gmm_enriched_k7	A5 — Classical GMM enriched k=7	59912145.0	60879924.0	61280668.0	6139

5. Technical and model-selection diagnostics – compact audit view

	scenario	method	k	feature_set	covariance_type	n_featu
0	A5 — Classical GMM enriched k=7	gmm	7	enriched	full	
1	A3 — Classical GMM enriched k=5	gmm	5	enriched	full	
2	B3 — Burt GMM enriched k=5	gmm	5	burt_enriched	full	
3	C3 — Burt- augmented GMM enriched k=5	gmm	5	burt_enriched	full	
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	kmeans	5	burt_augmented_enriched	None	
5	C5_joint — Burt- augmented GMM joint geometry k=8	gmm	8	burt_augmented_enriched	full	
6	B5 — Burt GMM enriched k=8	gmm	8	burt_enriched	full	
7	C5 — Burt- augmented GMM enriched k=8	gmm	8	burt_enriched	full	
8	B1 — Burt K-means simple k=5	kmeans	5	burt	None	
9	C1 — Burt- augmented K-means simple k=5	kmeans	5	burt	None	
10	B2 — Burt GMM simple k=5	gmm	5	burt	full	

	scenario	method	k	feature_set	covariance_type	n_featu
11	C2 — Burt-augmented GMM simple k=5	gmm	5	burt	full	
12	A2 — Classical GMM simple k=5	gmm	5	simple	full	
13	B_ext — Burt K-means enriched k=5	kmeans	5	burt_enriched	None	
14	C_ext — Burt-augmented K-means enriched k=5	kmeans	5	burt_enriched	None	
15	B0 — Burt plain Tweedie GLM	none	None	None	None	
16	C0 — Burt-augmented plain Tweedie GLM	none	None	None	None	
17	A1 — Classical K-means simple k=5	kmeans	5	simple	None	
18	C3_joint — Burt-augmented GMM joint geometry k=5	gmm	5	burt_augmented_enriched	full	
19	A0 — Classical plain Tweedie GLM	none	None	None	None	
20	A_ext — Classical K-means enriched k=5	kmeans	5	enriched	None	

6. Full benchmark registry – every retained metric and lineage field

	scenario	experiment	representation	gini	gini_in
0	A5 — Classical GMM enriched k=7	A5_classical_gmm_enriched_k7	classical	0.4241	
1	A3 — Classical GMM enriched k=5	A3_classical_gmm_enriched_k5	classical	0.4227	
2	B3 — Burt GMM enriched k=5	B3_burt_gmm_enriched_k5	burt	0.3816	
3	C3 — Burt- augmented GMM enriched k=5	C3_burtaug_gmm_enriched_k5	burt_augmented	0.3815	
4	C_ext_joint — Burt- augmented K-means joint geometry k=5	C_ext_burtaug_kmeans_joint_k5	burt_augmented	0.3797	
5	C5_joint — Burt- augmented GMM joint geometry k=8	C5_burtaug_gmm_joint_k8	burt_augmented	0.3797	
6	B5 — Burt GMM enriched k=8	B5_burt_gmm_enriched_k8	burt	0.3796	
7	C5 — Burt- augmented GMM enriched k=8	C5_burtaug_gmm_enriched_k8	burt_augmented	0.3795	
8	B1 — Burt K-means simple k=5	B1_burt_kmeans_simple_k5	burt	0.3790	
9	C1 — Burt- augmented	C1_burtaug_kmeans_simple_k5	burt_augmented	0.3790	

	scenario	experiment	representation	gini	gini_inc
	K-means simple k=5				
10	B2 — Burt GMM simple k=5	B2_burt_gmm_simple_k5	burt	0.3789	
11	C2 — Burt- augmented GMM simple k=5	C2_burtaug_gmm_simple_k5	burt_augmented	0.3789	
12	A2 — Classical GMM simple k=5	A2_classical_gmm_simple_k5	classical	0.3788	
13	B_ext — Burt K- means enriched k=5	B_ext_burt_kmeans_enriched_k5	burt	0.3787	
14	C_ext — Burt- augmented K-means enriched k=5	C_ext_burtaug_kmeans_enriched_k5	burt_augmented	0.3786	
15	B0 — Burt plain Tweedie GLM	B0_burt_plain_glm	burt	0.3785	
16	C0 — Burt- augmented plain Tweedie GLM	C0_burtaug_plain_glm	burt_augmented	0.3785	
17	A1 — Classical K- means simple k=5	A1_classical_kmeans_simple_k5	classical	0.3783	
18	C3_joint — Burt- augmented GMM joint	C3_burtaug_gmm_joint_k5	burt_augmented	0.3781	

	scenario	experiment	representation	gini	gini_inc
	geometry k=5				
19	A0 — Classical plain Tweedie GLM	A0_classical_plain_glm	classical	0.3688	
20	A_ext — Classical K- means enriched k=5	A_ext_classical_kmeans_enriched_k5	classical	0.3596	

21 rows × 28 columns

7. Metric catalog – governance map for extensible benchmarking

	metric_family	metric_name	available_in_regis
0	business_metrics	gini	1
1	business_metrics	top_decile_actual_to_predicted	1
2	business_metrics	top_decile_lift_vs_portfolio	1
3	business_metrics	top_decile_loss_share	1
4	business_metrics	premium_concentration_top_decile	1
5	calibration_metrics	loss_calibration	1
6	calibration_metrics	top_decile_calibration_error	1
7	calibration_metrics	mean_absolute_decile_calibration_error	1
8	calibration_metrics	max_absolute_decile_calibration_error	1
9	deviance_metrics	tweedie_deviance_per_exposure	1
10	deviance_metrics	tweedie_deviance	1
11	stability_metrics	actual_total_loss	1
12	stability_metrics	predicted_total_loss	1
13	stability_metrics	avg_final_premium	1
14	computational_metrics	runtime_seconds	1
15	computational_metrics	n_features	1
16	model_selection_metrics	bic	1
17	model_selection_metrics	aic	1

Section 11 — Gradient Boosting Extension

This section tests a prediction made in the technical note: that the A5–B5 Gini gap of

4.5 points is a property of the linear Tweedie GLM and should narrow under a non-linear pricing model. The Tweedie GLM is replaced by a LightGBM gradient booster (`objective="tweedie"`, `variance_power=1.5`, matching the GLM loss family), while the dataset, the Burt geometry, and the GMM segmentation are all held fixed. Only the pricing layer changes.

Design choices, with their reasons:

- **Out-of-fold premiums.** Discrimination is measured on 5-fold cross-fitted predictions with a fixed seed, identical across experiments. An in-sample GBM Gini is largely memorisation and would make the gap narrow for a spurious reason; only held-out premiums test the claim honestly.
- **Segmentation held fixed.** The existing GMM segments and BIC sweep are reused unchanged, so the comparison isolates the pricing model rather than changing two variables at once.
- **Hyperparameters frozen across series.** A single GBM configuration is used for A, B, and C, so any remaining gap reflects representation, not tuning effort. `min_child_samples` is set conservatively high to avoid asymmetric leaf overfitting on the smaller segments of the k=8 (B5) partition.
- **C-series re-included.** The Burt-augmented C-series was excluded from the GLM comparison on grounds of informational completeness, but that result is GLM-specific: a tree can use one-hot splits and Burt coordinates differently, so the C-series is run again here.
- **Frequency enrichment kept on the GLM.** Only the final pricing layer is swapped; the Poisson-GLM frequency and severity enrichment is retained. This keeps the experiment to a single change. A fully non-linear pipeline is left as future work.
- **Audited core untouched.** A separate `run_pricing_experiment_gbm` wrapper is added rather than editing the original, so the Section 9 regression baseline remains valid.

If the gap narrows, the original finding is localised to the GLM's inductive bias. If it persists, the Burt geometry carries structure the booster cannot recover from one-hot encoding, and the relevant sentence in the note must be revised. The calibration and aggregate-tail advantages of the Burt representation are expected to be robust to the pricing model either way.

```
In [28]: # =====  
# Section: 11  
# Cell: 11.1
```

```

# Title: Gradient Boosting Extension – Calibrated GBM Pricing
# Version: 1.0.0
# Date: 2026-06-18
# Description:
# Non-linear pricing layer that mirrors fit_calibrated_tweedie()
# but replaces the glum Tweedie GLM with a LightGBM gradient
# booster. Purpose: test whether the A5-vs-B5 Gini gap (a property
# of the linear Tweedie GLM, per the technical note conclusions)
# narrows under a non-linear learner, while segmentation and Burt
# geometry are held fixed.
#
# Design invariants (what makes the comparison honest):
# - Identical design matrix: reuses build_tweedie_design(), so
#   the GBM sees the same features as the GLM, including the
#   gmm_prob_j segment memberships.
# - Same Tweedie loss family: objective='tweedie',
#   tweedie_variance_power=1.5, matching TweedieDistribution(1.5).
# - Out-of-fold premiums: each policy is priced by a model that
#   never trained on it. In-sample GBM Gini is memorisation and
#   would make the A5/B5 gap meaningless.
# - Global calibration to observed losses, identical convention
#   to fit_calibrated_tweedie(), so downstream aggregate and
#   decile-calibration code is directly comparable.
# - GBM hyperparameters are FIXED, not tuned per experiment, so
#   A/B/C differ only in features – never in tuning effort.
# =====

import lightgbm as lgb
from sklearn.model_selection import KFold

# Fixed GBM configuration. Tuned once, frozen across all experiments.
# Regularisation lives in tree structure (num_leaves, min_child_samples)
# rather than an L2 penalty; this is the GBM analogue of the GLM's
# near-zero alpha=1e-6 ridge.
GBM_PARAMS = dict(
    objective="tweedie",
    tweedie_variance_power=1.5,    # matches glum TweedieDistribution(1.5)
    learning_rate=0.05,
    num_leaves=31,
    min_child_samples=100,         # down to 100 from 200
    n_estimators=600,
    subsample=0.8,
    subsample_freq=1,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    random_state=42,
    n_jobs=1,
    verbose=-1,
)

GBM_N_FOLDS = 5
GBM_FOLD_SEED = 7    # robustness check: second partition

def fit_calibrated_gbm(
    df: pd.DataFrame,

```

```

method: str,
k: int | None,
representation: str = "classical",
gbm_params: dict | None = None,
n_folds: int = GBM_N_FOLDS,
fold_seed: int = GBM_FOLD_SEED,
):
    """LightGBM analogue of fit_calibrated_tweedie().

    Same design matrix and same global-calibration convention as the GLM
    path, but premiums are produced out-of-fold to avoid in-sample
    optimism, and the learner is a Tweedie-objective gradient booster.

    Returns
    -----
    out : pd.DataFrame
        Copy of df with an out-of-fold, globally calibrated
        'final_premium' column. Drop-in replacement for the GLM output;
        every downstream metric (Gini, decile calibration, deviance)
        consumes this column unchanged.
    full_model : lgb.LGBMRegressor
        A booster fitted on ALL rows. Provided only for optional
        aggregate-layer use (segment-level expected cost). NOT used for
        Gini or any held-out metric – doing so would reintroduce
        in-sample optimism.
    info : dict
        Calibration and bookkeeping fields matching the GLM path, plus
        n_folds for audit.
    """
    params = dict(GBM_PARAMS)
    if gbm_params:
        params.update(gbm_params)

    X = build_tweedie_design(df, method=method, k=k, representation=representation)
    y = df["ClaimAmountTotal"].to_numpy()
    w = df["Exposure"].to_numpy()

    # — Out-of-fold premiums —————
    oof_raw = np.zeros(len(df), dtype=float)
    kf = KFold(n_splits=n_folds, shuffle=True, random_state=fold_seed)
    for train_idx, test_idx in kf.split(X):
        fold_model = lgb.LGBMRegressor(**params)
        fold_model.fit(
            X.iloc[train_idx],
            y[train_idx],
            sample_weight=w[train_idx],
        )
        oof_raw[test_idx] = fold_model.predict(X.iloc[test_idx])

    # — Global calibration to observed losses —————
    # Identical convention to fit_calibrated_tweedie: scale so the
    # total predicted premium reconciles to total observed loss.
    calibration_factor = safe_divide(y.sum(), oof_raw.sum(), default=1.0)

    out = df.copy()
    out["final_premium"] = oof_raw * calibration_factor

```

```

# — Full-data model (aggregate use only; never for Gini) —
full_model = lgb.LGBMRegressor(**params)
full_model.fit(X, y, sample_weight=w)

info = {
    "calibration_factor": float(calibration_factor),
    "actual_total_loss": float(y.sum()),
    "predicted_total_loss": float(out["final_premium"].sum()),
    "n_features": int(X.shape[1]),
    "n_folds": int(n_folds),
    "pricing_model": "gbm",
}
return out, full_model, info

print("✓ Section 11.1 loaded: fit_calibrated_gbm() available.")
print(f"  GBM objective: tweedie (power=1.5), {GBM_N_FOLDS}-fold OOF premium

```

✓ Section 11.1 loaded: fit_calibrated_gbm() available.
 GBM objective: tweedie (power=1.5), 5-fold OOF premiums.

```

In [29]: # =====
# Section: 11
# Cell: 11.2
# Title: Gradient Boosting Extension – GBM Experiment Runner
# Version: 1.0.0
# Date: 2026-06-18
# Description:
# Wrapper that mirrors run_pricing_experiment() but swaps the
# Tweedie GLM pricing layer for fit_calibrated_gbm(). Kept as a
# SEPARATE function rather than a branch inside the original so
# the Section 9 regression baseline stays valid and the audited
# GLM path is frozen (design decision D8).
#
# The flow is identical to run_pricing_experiment:
#   prepare_model_data -> apply_segmentation -> fit -> evaluate.
# Only the fit step changes. apply_segmentation is called with the
# same config, so feeding this the A5/B5/C3 config reuses the exact
# same GMM segments (design decision D3) – only the pricing model
# differs. The result dict has the same keys as the GLM runner and
# drops into the benchmark registry unchanged, gaining only the
# extra fit_info fields (n_folds, pricing_model).
# =====

def run_pricing_experiment_gbm(base_df: pd.DataFrame, config: dict):
    """End-to-end GBM experiment: segmentation -> GBM (OOF) -> calibration ->

    Drop-in analogue of run_pricing_experiment(). Reuses prepare_model_data,
    apply_segmentation, and evaluate_pricing_model verbatim; only the pricing
    fit is replaced by fit_calibrated_gbm(). Returns the same (df, model,
    result) triple with a result dict carrying identical keys to the GLM path
    """
    started = time.perf_counter()
    df = prepare_model_data(base_df)
    df, seg_info = apply_segmentation(

```



```

        df,
        method=config["method"],
        k=config.get("k"),
        feature_set=config.get("feature_set", "simple"),
        covariance_type=config.get("covariance_type", "full"),
    )
    representation = config.get("representation", "classical")
    df, pp_model, fit_info = fit_calibrated_gbm(
        df,
        method=config["method"],
        k=config.get("k"),
        representation=representation,
        gbm_params=config.get("gbm_params"),          # None -> frozen GBM_PA
        n_folds=config.get("n_folds", GBM_N_FOLDS),
        fold_seed=config.get("fold_seed", GBM_FOLD_SEED),
    )
    metrics = evaluate_pricing_model(df)
    runtime = time.perf_counter() - started

    result = {
        "experiment":      config["name"],
        "representation":  config.get("representation", "classical"),
        "method":          config["method"],
        "k":               config.get("k"),
        "feature_set":     config.get("feature_set"),
        "covariance_type": config.get("covariance_type"),
        "runtime_seconds": float(runtime),
        **seg_info,
        **fit_info,
        **metrics,
    }
    return df, pp_model, result

print("✓ Section 11.2 loaded: run_pricing_experiment_gbm() available.")
print("  Same flow and result keys as run_pricing_experiment; pricing layer

```

- ✓ Section 11.2 loaded: run_pricing_experiment_gbm() available.
Same flow and result keys as run_pricing_experiment; pricing layer = GBM.

```

In [30]: # =====
# Section: 11
# Cell: 11.3
# Title: Gradient Boosting Extension – Canonical Comparison Driver
# Version: 1.0.0
# Date: 2026-06-18
# Description:
# Runs the canonical experiments (A5, B5, and both C-series
# variants) through the GBM pricing layer, reusing their exact
# Section 5 configs so only the pricing model differs. Pairs each
# GBM result with its GLM counterpart read live from
# benchmark_results, and prints the Gini gap side by side – the
# direct test of the standing claim that the 4.5-point A5-vs-B5
# gap narrows under a non-linear learner.
#
# Reuses the same configs verbatim (design decisions D3, D6): same

```

```

# representation, method, k, feature_set as the GLM benchmark. The
# GLM-only 'alpha' key is ignored by the GBM path. GBM defaults
# (GBM_PARAMS, GBM_N_FOLDS, GBM_FOLD_SEED) are held frozen across
# all four experiments.
#
# Both C variants are included (design decision D7): the plain
# burt_enriched C (near-redundant with B by construction) and the
# joint burt_augmented_enriched C, where dummies and Burt
# coordinates coexist – the case where a tree could exploit both
# and the GLM-era 'informational completeness' result may break.
# =====

# Canonical configs, copied verbatim from the Section 5 EXPERIMENTS list.
# k values reference the same module constants used there (A5_K=7, B5_K=8).
# NOTE ON THE GBM CALIBRATION FACTOR
# The Tweedie GLM is mean-unbiased by construction: its MLE intercept forces
# the weighted sum of fitted values to equal the weighted sum of observations
# so its calibration factor sits near 1.0. A Tweedie GBM has no such guarantee
# - it descends on the log-mean (rate) scale, and  $\exp(\text{mean}) \neq \text{mean}(\exp)$ 
# (Jensen's inequality), so back-transformed predictions are biased low and
# global factor scales them up by ~1.2-1.35. This is expected GBM behaviour,
# NOT under-fitting (insensitive to min_child_samples / n_estimators, as the
# 200->100 run confirmed). The factor is applied BEFORE any downstream metrics
# final_premium = raw * factor. Gini is rank-based and invariant to a global
# scaling, so discrimination results are unaffected; the top-decile ratio is
# meaningful because the global level has already been reconciled.

GBM_CANONICAL_CONFIGS = [
    {"name": f"A5_classical_gmm_enriched_k{A5_K}",
      "representation": "classical",      "method": "gmm",
      "k": A5_K, "feature_set": "enriched",
      "covariance_type": "full",
      "glm_experiment": f"A5_classical_gmm_enriched_k{A5_K}"},

    {"name": f"B5_burt_gmm_enriched_k{B5_K}",
      "representation": "burt",           "method": "gmm",
      "k": B5_K, "feature_set": "burt_enriched",
      "covariance_type": "full",
      "glm_experiment": f"B5_burt_gmm_enriched_k{B5_K}"},

    {"name": "C3_burtaug_gmm_enriched_k5",
      "representation": "burt_augmented", "method": "gmm",
      "k": 5, "feature_set": "burt_enriched",
      "covariance_type": "full",
      "glm_experiment": "C3_burtaug_gmm_enriched_k5"},

    {"name": "C3_burtaug_gmm_joint_k5",
      "representation": "burt_augmented", "method": "gmm",
      "k": 5, "feature_set": "burt_augmented_enriched",
      "covariance_type": "full",
      "glm_experiment": "C3_burtaug_gmm_joint_k5"},
]

def _glm_metric(experiment_name, column):
    """Read a metric for a GLM experiment from benchmark_results, or NaN."""

```

```

hit = benchmark_results.loc[
    benchmark_results["experiment"] == experiment_name, column
]
return float(hit.iloc[0]) if len(hit) else float("nan")

# — Run the four GBM experiments —————
gbm_results = {}
gbm_experiment_data = {}
for cfg in GBM_CANONICAL_CONFIGS:
    print(f"Running GBM: {cfg['name']}")
    base_table = REPRESENTATION_TABLES[cfg["representation"]]
    df_r, model_r, result_r = run_pricing_experiment_gbm(base_table, cfg)
    gbm_results[cfg["name"]] = result_r
    gbm_experiment_data[cfg["name"]] = df_r
    cf = result_r["calibration_factor"]
    # Tweedie GBMs carry a structural Jensen-gap level bias (see header note)
    # a factor of ~1.2-1.35 is healthy here, not a misfit. Flag only if the
    # global correction is implausibly large, which would signal a real problem
    flag = "" if 0.7 <= cf <= 1.45 else " ⚠ calibration factor implausible"
    print(f"  Gini {result_r['gini']:.4f} · "
          f"top-decile {result_r['top_decile_actual_to_predicted']:.3f} · "
          f"calib {cf:.3f}{flag}")

# — Assemble the GLM-vs-GBM comparison table —————
rows = []
for cfg in GBM_CANONICAL_CONFIGS:
    name = cfg["name"]
    gname = cfg["glm_experiment"]
    g = gbm_results[name]
    rows.append({
        "experiment": name,
        "gini_glm": _glm_metric(gname, "gini"),
        "gini_gbm": g["gini"],
        "gini_delta": g["gini"] - _glm_metric(gname, "gini"),
        "topdec_glm": _glm_metric(gname, "top_decile_actual_to_predicted"),
        "topdec_gbm": g["top_decile_actual_to_predicted"],
        "deviance_glm": _glm_metric(gname, "tweedie_deviance_per_exposure"),
        "deviance_gbm": g["tweedie_deviance_per_exposure"],
        "calibration_factor": g["calibration_factor"],
    })
gbm_comparison = pd.DataFrame(rows)

# — The headline: does the A5-vs-B5 Gini gap narrow? —————
a5 = gbm_comparison.loc[gbm_comparison["experiment"].str.startswith("A5")].i
b5 = gbm_comparison.loc[gbm_comparison["experiment"].str.startswith("B5")].i
gap_glm = a5["gini_glm"] - b5["gini_glm"]
gap_gbm = a5["gini_gbm"] - b5["gini_gbm"]

pd.set_option("display.float_format", lambda v: f"{v:,.4f}")
print("\n" + "=" * 64)
print("GLM vs GBM – canonical comparison (Gini computed out-of-fold for GBM)")
print("=" * 64)
print(gbm_comparison.to_string(index=False))

print("\n" + "-" * 64)

```

```

print("A5-vs-B5 Gini gap (the standing claim)")
print("-" * 64)
print(f"  Under GLM : {gap_glm:.4f}")
print(f"  Under GBM : {gap_gbm:.4f}")
narrowed = gap_gbm < gap_glm
direction = "NARROWED" if narrowed else ("WIDENED" if gap_gbm > gap_glm else
pct = (1 - gap_gbm / gap_glm) * 100 if gap_glm else float("nan")
print(f"  Gap {direction} by {gap_glm - gap_gbm:+.4f} "
      f"({pct:+.1f}% of the GLM gap)")
print("-" * 64)
print("Reading: a narrowed gap localises the original finding to the")
print("GLM's inductive bias; a persistent gap means the Burt geometry")
print("carries structure the booster cannot recover from one-hot encoding.")
print("Calibration (top-decile) is expected to remain a Burt advantage either")

```

```

Running GBM: A5_classical_gmm_enriched_k7
  Gini 0.3838 · top-decile 1.003 · calib 1.328
Running GBM: B5_burt_gmm_enriched_k8
  Gini 0.3492 · top-decile 0.628 · calib 1.283
Running GBM: C3_burtaug_gmm_enriched_k5
  Gini 0.3508 · top-decile 0.852 · calib 1.284
Running GBM: C3_burtaug_gmm_joint_k5
  Gini 0.3553 · top-decile 0.857 · calib 1.283

```

=====

GLM vs GBM – canonical comparison (Gini computed out-of-fold for GBM)

=====

	experiment	gini_glm	gini_gbm	gini_delta	topdec_glm	to
pdec_gbm	deviance_glm	deviance_gbm	calibration_factor			
A5_classical_gmm_enriched_k7	0.4241	0.3838	-0.0403	1.1386		
1.0028	61.2760	61.3452	1.3278			
B5_burt_gmm_enriched_k8	0.3796	0.3492	-0.0304	1.0323		
0.6282	61.3358	63.1488	1.2834			
C3_burtaug_gmm_enriched_k5	0.3815	0.3508	-0.0307	1.0389		
0.8523	61.2901	63.3081	1.2836			
C3_burtaug_gmm_joint_k5	0.3781	0.3553	-0.0228	1.0294		
0.8569	61.3459	63.0246	1.2835			

A5-vs-B5 Gini gap (the standing claim)

```

  Under GLM : 0.0445
  Under GBM : 0.0346
  Gap NARROWED by +0.0099 (+22.3% of the GLM gap)

```

Reading: a narrowed gap localises the original finding to the
GLM's inductive bias; a persistent gap means the Burt geometry
carries structure the booster cannot recover from one-hot encoding.
Calibration (top-decile) is expected to remain a Burt advantage either way.

```

In [31]: # =====
# Section: 11
# Cell: 11.4
# Title: Gradient Boosting Extension – Aggregate Risk Under GBM
# Version: 1.0.0
# Date: 2026-06-18

```

```

# Description:
# Feeds the GBM-priced A5 and B5 frames into the EXISTING Section 7
# aggregate machinery (run_aggregate_validation), unchanged. Tests
# whether the Burt representation keeps its lower-tail / lower-TVaR
# advantage when the pricing model is non-linear.
#
# How the GBM enters the aggregate (important for interpretation):
# - Segment FREQUENCY uses PredFreq – the Poisson GLM, unchanged
#   (design decision D9: only the pricing layer was swapped).
# - Segment MEAN SEVERITY is derived in prepare_aggregate_parameters
#   as sum(final_premium)/expected_claims. final_premium is now the
#   GBM premium, so the booster's reallocation of expected loss
#   across the (identical) GMM segments flows into the aggregate.
# - Empirical per-segment CVs are estimated from observed claims via
#   weighted lognormal MLE (compute_empirical_cvs) – pricing-model
#   agnostic, identical method to the GLM path.
#
# Therefore A5-GBM aggregate is NOT a copy of A5-GLM: it differs by
# exactly how the booster moved expected loss between segments. The
# question is whether Burt's GLM-era tail advantage survives that.
#
# Requires gbm_experiment_data from Cell 11.3 (the priced frames).
# Reuses run_aggregate_validation / format_reporting_table verbatim.
# =====

if "gbm_experiment_data" not in globals():
    raise RuntimeError("Run Cell 11.3 first – gbm_experiment_data is needed.")

_A5_GBM_NAME = f"A5_classical_gmm_enriched_k{A5_K}"
_B5_GBM_NAME = f"B5_burt_gmm_enriched_k{B5_K}"

_gbm_agg_rows = []
_gbm_agg_models = {}
for _name, _selection in [(_A5_GBM_NAME, "best_gini_gbm"),
                          (_B5_GBM_NAME, "best_calibration_gbm")]:
    _df = gbm_experiment_data[_name]
    _seg_col = "risk_cluster" if "risk_cluster" in _df.columns else None
    print("=" * 60)
    print(f"GBM Aggregate – {_name}")
    print("=" * 60)
    _model, _params, _metrics = run_aggregate_validation(
        _df, segmentation_col=_seg_col, log2=20, bs=100,
    )
    _gbm_agg_models[_name] = _model
    _gbm_agg_rows.append({
        "experiment": _name,
        "selection": _selection,
        "pricing": "gbm",
        **_metrics,
    })
    print()

gbm_aggregate_table = pd.DataFrame(_gbm_agg_rows)

# — A5-GBM vs B5-GBM, and vs the published GLM figures —————
# GLM reference values from the technical note §4.2 (ground-up).

```

```

_GLM_REF = {
    _A5_GBM_NAME: {"agg_var99": 61_395_800.0, "agg_tvar99": 61_618_566.0},
    _B5_GBM_NAME: {"agg_var99": 61_322_700.0, "agg_tvar99": 61_533_968.0},
}

a5g = gbm_aggregate_table.set_index("experiment").loc[_A5_GBM_NAME]
b5g = gbm_aggregate_table.set_index("experiment").loc[_B5_GBM_NAME]

print("=" * 64)
print("Aggregate tail – does Burt keep its advantage under GBM?")
print("=" * 64)
_fmt = lambda v: f"{v:,.0f}"
print(f"{'Metric':<14}{'A5-GBM':>16}{'B5-GBM':>16}{'B5 - A5':>14}")
print("-" * 64)
for _m, _lbl in [("agg_mean", "Mean"), ("agg_var99", "VaR 99%"),
                 ("agg_tvar99", "TVaR 99%"), ("agg_tvar95", "TVaR 95%")]:
    _d = b5g[_m] - a5g[_m]
    print(f"{'_lbl':<14}{'_fmt(a5g[_m]):>16}{'_fmt(b5g[_m]):>16}{'_fmt(_d):>14}")
print("-" * 64)

_tvar_gap_gbm = b5g["agg_tvar99"] - a5g["agg_tvar99"]
_tvar_gap_glm = _GLM_REF[_B5_GBM_NAME]["agg_tvar99"] - _GLM_REF[_A5_GBM_NAME]
print("TVaR 99% gap (B5 - A5), Burt advantage is negative:")
print(f"  Under GLM (published): {_tvar_gap_glm:>14,.0f}")
print(f"  Under GBM (this run) : {_tvar_gap_gbm:>14,.0f}")
if _tvar_gap_gbm < 0 and _tvar_gap_glm < 0:
    print("  -> Burt keeps the lower ground-up tail under GBM. Advantage ROE")
elif _tvar_gap_gbm >= 0 and _tvar_gap_glm < 0:
    print("  -> Burt's ground-up tail advantage DID NOT survive the GBM. Inv")
else:
    print("  -> Mixed/!inverted vs published GLM sign. Inspect before report")
print("=" * 64)
print("Note: GLM reference is ground-up from the technical note. Coverage-la")
print("reversal (deductible / per-claim limit) is the Cell 8.1 analysis; rer")
print("it on these GBM frames separately if the ground-up advantage holds.")

```

=====

GBM Aggregate – A5_classical_gmm_enriched_k7

=====

Empirical CVs: ['1.370', '1.606', '1.243', '1.685', '1.359', '1.597', '1.719']

Decl string:

agg Portfolio [2640.80 2554.32 5309.98 9543.86 19.55 9166.05 6821.58] claims
sev lognorm [2054.53 2155.16 2356.72 624.58 1240.06 1709.08 2171.60] cv [1.37 1.61 1.24 1.68 1.36 1.60 1.72] poisson

--- Aggregate diagnostics ---

	E[X]	Est E[X]	Err E[X]	CV(X)	Est CV(X)	Skew(X)	Est Skew(X)
w(X)							
X							
Freq	36056			0.0052664		0.0052664	
Sev	1661.6	1661.5	-1.0136e-05	1.6979	1.698	9.6336	
9.632							
Agg	5.9909e+07	5.9909e+07	-1.0136e-05	0.010377	0.010378	0.039098	0.039092

log2 = 20, bandwidth = 100, validation: not unreasonable.
VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.

=====

GBM Aggregate – B5_burt_gmm_enriched_k8

=====

Empirical CVs: ['1.832', '1.599', '1.461', '1.525', '1.688', '1.110', '1.442', '1.460']

Decl string:

agg Portfolio [1703.80 6745.86 1782.30 10061.85 8262.36 764.26 3857.80 2877.92] claims sev lognorm [1999.67 1894.79 1334.92 1840.30 1395.35 1313.49 1657.65 1353.93] cv [1.83 1.60 1.46 1.52 1.69 1.11 1.44 1.46] poisson

--- Aggregate diagnostics ---

	E[X]	Est E[X]	Err E[X]	CV(X)	Est CV(X)	Skew(X)	Est Skew(X)
(X)							
X							
Freq	36056			0.0052664		0.0052664	
Sev	1661.6	1661.6	2.2329e-05	1.5947	1.5947	9.2511	9.2496
496							
Agg	5.9909e+07	5.9911e+07	2.2329e-05	0.0099128	0.009913	0.03644	0.036429

log2 = 20, bandwidth = 100, validation: not unreasonable.
VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.

=====

Aggregate tail – does Burt keep its advantage under GBM?

=====

Metric	A5-GBM	B5-GBM	B5 - A5
Mean	59,909,110	59,909,193	82
VaR 99%	61,373,100	61,308,400	-64,700
TVaR 99%	61,591,892	61,516,749	-75,143
TVaR 95%	61,204,912	61,148,000	-56,912

TVaR 99% gap (B5 - A5), Burt advantage is negative:

Under GLM (published): -84,598

Under GBM (this run) : -75,143

-> Burt keeps the lower ground-up tail under GBM. Advantage ROBUST.

=====

Note: GLM reference is ground-up from the technical note. Coverage-layer reversal (deductible / per-claim limit) is the Cell 8.1 analysis; rerun it on these GBM frames separately if the ground-up advantage holds.

```
In [32]: # =====
# Section: 11
# Cell: 11.5
# Title: Gradient Boosting Extension – Coverage-Layer Reversal Under GBM
# Version: 1.0.0
# Date: 2026-06-18
# Description:
# Tests whether the paper's coverage-layer reversal survives GBM
# pricing. Under the GLM: B5 (Burt) wins ground-up and under a
# deductible, but A5 (classical) wins under a per-claim limit –
# the optimal representation depends on the reinsurance structure.
# This cell reruns that three-structure comparison on the
# GBM-priced A5 and B5 frames.
#
# Mirrors Cell 8.1's layer math exactly:
#   ground-up      -> claims inf xs 0
#   deductible 150 -> claims inf xs 150
#   per-claim 12k  -> claims 12000 xs 0
#
# The GBM enters through segment mean severities (from final_premium);
# frequency (PredFreq, GLM) and the empirical-CV method are unchanged,
# identical to Cell 11.4. Parameter frames are rebuilt here from the
# GBM frames via prepare_aggregate_parameters + compute_empirical_cvs,
# so the layers run on GBM severities rather than the GLM agg_params.
#
# Requires gbm_experiment_data from Cell 11.3.
# Reuses build (aggregate), prepare_aggregate_parameters,
# compute_empirical_cvs verbatim.
# =====

from aggregate import build as _agg_build

if "gbm_experiment_data" not in globals():
    raise RuntimeError("Run Cell 11.3 first – gbm_experiment_data is needed.")

_A5 = f"A5_classical_gmm_enriched_k{A5_K}"
_B5 = f"B5_burt_gmm_enriched_k{B5_K}"

def _gbm_layer_params(experiment_name):
    """Rebuild aggregate parameter frame from a GBM-priced experiment."""
    df = gbm_experiment_data[experiment_name]
    seg_col = "risk_cluster" if "risk_cluster" in df.columns else None
    emp_cvs = compute_empirical_cvs(df)
    return prepare_aggregate_parameters(
        df, segmentation_col=seg_col, empirical_cvs=emp_cvs
    )
```



```

def _decl_vectors(params):
    cc = "[" + " ".join(f"{x:.2f}" for x in params["expected_claims"]) + "]"
    ms = "[" + " ".join(f"{x:.2f}" for x in params["mean_sev"]) + "]"
    cv = "[" + " ".join(f"{x:.2f}" for x in params["cv"]) + "]"
    return cc, ms, cv

def _build_layers(params, tag):
    cc, ms, cv = _decl_vectors(params)
    base = _agg_build(f"agg {tag}base {cc} claims inf xs 0 sev lognorm {ms}
                      log2=20, bs=100)
    ded = _agg_build(f"agg {tag}ded {cc} claims inf xs 150 sev lognorm {ms}
                      log2=20, bs=100)
    net = _agg_build(f"agg {tag}net {cc} claims 12000 xs 0 sev lognorm {ms}
                      log2=20, bs=100)
    return base, ded, net

_pa = _gbm_layer_params(_A5)
_pb = _gbm_layer_params(_B5)
a_base, a_ded, a_net = _build_layers(_pa, "A5g")
b_base, b_ded, b_net = _build_layers(_pb, "B5g")

# — Comparison table: A5-GBM vs B5-GBM across the three layers —
print("=" * 72)
print("Coverage-layer comparison under GBM pricing (B5 - A5 < 0 means Burt")
print("=" * 72)
print(f"{'Structure':<26}{'A5-GBM':>15}{'B5-GBM':>15}{'B5 - A5':>14}{'winner'")
print("-" * 72)

_layers = [
    ("Ground-up", a_base, b_base),
    ("Deductible 150", a_ded, b_ded),
    ("Per-claim limit 12k", a_net, b_net),
]

_tvar_diffs = {}
for _lbl, _am, _bm in _layers:
    _ta, _tb = _am.tvar(0.99), _bm.tvar(0.99)
    _d = _tb - _ta
    _tvar_diffs[_lbl] = _d
    _win = "B5" if _d < 0 else "A5"
    print(f"{'_lbl+' TVaR99':<26}{'_ta:>15,.0f}{'_tb:>15,.0f}{'_d:>14,.0f}{'_win:")
print("-" * 72)

# — The reversal verdict —————
print("\nReversal test (the paper's novel finding):")
print(f" Ground-up : Burt {'lower' if _tvar_diffs['Ground-up'] < 0 else
      f"({'_tvar_diffs['Ground-up']:+,.0f})")
print(f" Deductible 150 : Burt {'lower' if _tvar_diffs['Deductible 150'] <
      f"({'_tvar_diffs['Deductible 150']:+,.0f})")
print(f" Per-claim 12k : Burt {'lower' if _tvar_diffs['Per-claim limit 12k']
      f"({'_tvar_diffs['Per-claim limit 12k']:+,.0f})")

```

```

_groundup_burt_wins = _tvar_diffs["Ground-up"] < 0
_ded_burt_wins      = _tvar_diffs["Deductible 150"] < 0
_limit_burt_wins    = _tvar_diffs["Per-claim limit 12k"] < 0

print()
if _groundup_burt_wins and _ded_burt_wins and not _limit_burt_wins:
    print("  -> REVERSAL SURVIVES under GBM: Burt wins ground-up and deducti
    print("      classical wins under the per-claim limit. The paper's headli
    print("      finding holds under non-linear pricing.")
elif _groundup_burt_wins and _limit_burt_wins:
    print("  -> REVERSAL DID NOT SURVIVE: Burt now wins under the per-claim
    print("      too. The GLM-era crossover was partly a linear-pricing artif
    print("      This is itself a reportable finding – revise the $4.3 / $7 c
else:
    print("  -> Pattern differs from the GLM case. Inspect the three signs a
    print("      before reporting; do not assume the published narrative hold
print("=" * 72)
print("GLM reference (technical note §7): ground-up B5 ~EUR 85k lower; deduc
print("advantage widens to ~EUR 550k; per-claim limit reverses, A5 ~EUR 478k

```

=====				
Coverage-layer comparison under GBM pricing (B5 - A5 < 0 means Burt lower)				
=====				
Structure	A5-GBM	B5-GBM	B5 - A5	winn
er				

Ground-up TVaR99	61,591,892	61,516,749	-75,143	
B5				
Deductible 150 TVaR99	59,902,794	59,383,435	-519,359	
B5				
Per-claim limit 12k TVaR99	58,275,311	58,675,480	400,169	
A5				

Reversal test (the paper's novel finding):

```

Ground-up      : Burt lower (-75,143)
Deductible 150 : Burt lower (-519,359)
Per-claim 12k  : Burt higher (+400,169)

```

-> REVERSAL SURVIVES under GBM: Burt wins ground-up and deductible, classical wins under the per-claim limit. The paper's headline finding holds under non-linear pricing.

=====

GLM reference (technical note §7): ground-up B5 ~EUR 85k lower; deductible advantage widens to ~EUR 550k; per-claim limit reverses, A5 ~EUR 478k lower.

Section 11 — Results and Conclusions

The gradient boosting extension tests one prediction made in the technical note — that the A5–B5 Gini gap is a property of the linear Tweedie GLM and should narrow under a non-linear learner. Replacing the GLM with a LightGBM Tweedie booster, holding the dataset, the Burt geometry and the GMM segmentation fixed, and measuring

discrimination out-of-fold, yields four findings. Each was confirmed across two independent fold partitions (seeds 42 and 7).

1. The discrimination gap narrows, but does not collapse.

Under the GLM the A5–B5 Gini gap is 0.0445. Under the GBM it falls to 0.0346 — a reduction of roughly 22–23%, stable to four decimals across both seeds. About one fifth of the gap was an artifact of the GLM's affinity for one-hot contrasts; the remaining four fifths is a genuine property of the Burt embedding, which smooths categorical co-occurrence into continuous coordinates and thereby discards some of the sharp idiosyncratic signal that individual modality contrasts carry. The prediction in §7 is therefore confirmed in direction but refuted in its stronger implication: the gap is *partly*, not wholly, a GLM property.

2. Informational completeness survives non-linearity.

The two C-series variants — Burt coordinates alone, and Burt coordinates retained alongside the original categorical dummies — produce near-identical Gini under the booster (within 0.005), across both seeds. A tree, which could in principle exploit raw one-hot splits and smooth Burt coordinates simultaneously, finds essentially nothing in the dummies beyond what the Burt coordinates already encode. The completeness result is stronger than its GLM-era form: it holds under a model architecturally capable of using both representations.

3. The aggregate tail advantage is robust to the pricing model.

On a ground-up basis the Burt portfolio (B5) retains a lower 99% TVaR than the classical portfolio (A5) — approximately €75,000 lower under the GBM, against €85,000 under the GLM. The aggregate coefficient of variation remains lower for Burt (0.00991 vs 0.01038). Because portfolio mean and total frequency are held identical by construction, the entire advantage lives in the higher moments: Burt distributes the same expected loss across its segments with lower dispersion and skew. The figures were near-invariant across seeds (a spread of about €140 on the TVaR gap), confirming the advantage flows from the segmentation geometry rather than the pricing model.

4. The coverage-layer reversal holds under non-linear pricing.

The reinsurance-dependent reversal is reproduced under the GBM, with all three signs stable across both seeds: Burt produces the lower 99% TVaR ground-up (~€75k) and under a 150 EUR deductible (~€517k), while the classical representation produces the lower TVaR under a 12,000 EUR per-claim limit (~€400k). The per-claim limit caps the larger individual claims that drive Burt's heavier severity tail, restoring the classical representation's advantage once the severity upside is truncated. The paper's most

novel finding — that the optimal representation depends on the treaty structure in place — is confirmed under a second, non-linear pricing model.

One caveat: top-decile calibration of the Burt representation is partition-sensitive.

The classical top-decile ratio is stable across seeds (~ 0.99 – 1.00). The Burt top-decile ratio is not: it ranges from 0.63 to 0.91 across the two partitions. Burt's smoother, more compressed premium distribution gives its top decile a less sharply defined boundary, so the policies that fall just inside it — and hence the single-bucket calibration statistic — shift materially with the fold split. Top-decile calibration of the Burt representation under non-linear pricing should therefore be reported as a distribution across partitions rather than a single point estimate. This sensitivity does not affect the Gini gap (rank-based and stable) or the aggregate findings (driven by segment structure, not the decile cut).

A note on calibration factors.

The GBM global calibration factors sit near 1.28–1.33, against the GLM's ~ 1.0 . This is the expected Jensen-gap level bias of a Tweedie GBM descending on the log-mean scale, not a sign of misfit: it is insensitive to tree-size parameters and is applied before any downstream metric, leaving the rank-based Gini unaffected.

In one sentence: the capital-structure findings (aggregate tail advantage and coverage-layer reversal) and the informational-completeness result are robust to the pricing model; the discrimination gap is partly a property of the linear GLM; and top-decile calibration is the one metric requiring a partition caveat.

Section 12 — Bottom-Up versus Directly-Fitted Collective Model

This section responds to a remark by M. Denuit on the general approach. The aggregate analysis in this work is *bottom-up*: individual segment-level frequency and severity models are aggregated upward, by FFT convolution, into the portfolio distribution. The conventional route for solvency analysis is *top-down*: the collective model is fitted *directly* on portfolio data — the frequency and severity parameters of the aggregate are estimated without reference to individual profiles — and the individual model is then used only to allocate the expected cost and risk margin down onto policies for pricing.

The two routes should coincide in theory. In practice they diverge, and it is worth being explicit that the divergence has **two distinct sources**, only one of which this

benchmark can exhibit:

- **Claims-development divergence** (the source Denuit names). A collective model fitted on developed or reserved portfolio data reflects open claims — provisioned but not yet paid, including IBNR — whereas a bottom-up model built on observed paid amounts does not. **freMTPL2 cannot show this**. It is a settled, one-year academic benchmark with little or no claims development to miss; this divergence is a property of production portfolios and is here absent by construction. We state this plainly rather than manufacture synthetic reserves to force it.
- **Segmentation-mixture divergence** (which this benchmark *can* show, and which ties to the central question of this work). The top-down severity is a single distribution fitted to all claims pooled together; the bottom-up severity is a mixture of per-segment laws with heterogeneous coefficients of variation (here ranging ≈ 1.1 – 1.8). Pooling discards that heterogeneity. Any divergence in the tail on freMTPL2 is therefore attributable to segmentation structure, not to provisioning — that is, to the same representation/segmentation geometry that this work studies throughout.

What this section builds:

- A **directly-fitted collective model**: a single Poisson intensity (total expected claim count) and a single lognormal severity fitted to the pooled claim amounts, ignoring segments entirely.
- Its aggregate, through the **same FFT machinery** used elsewhere, as a one-segment compound Poisson — a simplification of, not an addition to, the existing pipeline.
- A **comparison** of its VaR/TVaR against the bottom-up A5 and B5 aggregates, isolating the segmentation-mixture effect, with the claims-development source noted as out of scope for this benchmark.

The aim is not to ask whether the two approaches agree, but to decompose why they differ: to confirm that on a settled benchmark the divergence reduces to the segmentation-mixture effect, and to locate the claims-development effect where it actually arises — in production portfolios with open claims.

```
In [33]: # =====  
# Section: 12  
# Cell: 12.1  
# Title: Bottom-Up vs Directly-Fitted Collective Model  
# Version: 1.0.0  
# Date: 2026-06-19  
# Description:  
# Builds a top-down (directly-fitted) collective model and
```

```

# compares its aggregate tail with the bottom-up A5 and B5
# aggregates. Responds to M. Denuit's remark.
#
# Top-down severity is fitted DIRECTLY on the observed pooled
# claim amounts via weighted lognormal MLE – the same closed form
# used per-segment in compute_empirical_cvs (Cell 7.1), but pooled
# over the whole portfolio, ignoring segments. This is the honest
# top-down construction: the collective severity does not reference
# individual profiles or the pricing model at all.
#
# Frequency is the portfolio total expected claim count (identical
# to the bottom-up total by construction), so the two approaches
# share the same mean and frequency and differ ONLY in the severity
# representation: a single pooled lognormal (top-down) versus the
# intensity-weighted mixture of per-segment lognormals (bottom-up).
# Any tail divergence on this settled benchmark is therefore the
# segmentation-mixture effect, isolated. The claims-development
# divergence Denuit names is out of scope for freMTPL2 (no open
# claims to miss) and is not modelled here.
#
# Requires gbm_experiment_data (Cell 11.3) OR the GLM best-experiment
# frames; uses A5/B5 bottom-up metrics from run_aggregate_validation.
# Reuses build (aggregate) and the Cell 7.1 helpers verbatim.
# =====

from aggregate import build as _agg_build, qd as _agg_qd

def fit_pooled_lognormal_severity(df: pd.DataFrame) -> dict:
    """Directly-fitted collective severity: weighted lognormal MLE on the
    pooled observed per-claim amounts, ignoring segments. Returns the mean
    severity and CV implied by the single fitted lognormal."""
    claimants = df[(df["ClaimNb"] > 0) & (df["ClaimAmountTotal"] > 0)].copy()
    sev = claimants["ClaimAmountTotal"].to_numpy() / claimants["ClaimNb"].to_numpy()
    log_sev = np.log(sev)
    # exposure-free pooled MLE (each claim weighted equally), matching the
    # per-segment formula in compute_empirical_cvs but without GMM weights
    mu_log = log_sev.mean()
    sig_log = np.sqrt(((log_sev - mu_log) ** 2).mean())
    mean_sev = float(np.exp(mu_log + 0.5 * sig_log ** 2))
    cv = float(np.sqrt(np.exp(sig_log ** 2) - 1.0))
    return {"mean_sev": mean_sev, "cv": cv,
            "mu_log": float(mu_log), "sig_log": float(sig_log),
            "n_claims": int(len(sev))}

def run_topdown_collective(df: pd.DataFrame, target_total_loss: float,
                           log2: int = 20, bs: int = 100):
    """Single-segment compound-Poisson aggregate from a directly-fitted
    collective model: one Poisson intensity (portfolio expected claims),
    one pooled lognormal severity.

    The pooled severity SHAPE (CV) is fitted on observed claim amounts, but
    its mean is rescaled so that expected_claims * mean_sev equals
    target_total_loss – the same total the bottom-up model reconciles to.
    This holds mean and frequency equal across the two approaches, so the

```

```

only remaining difference is the severity representation (pooled vs
segmented mixture). Without this, the comparison would conflate a
definitional mean mismatch (model-calibrated vs raw-observed expected
loss) with the segmentation-mixture effect of interest.""
sev = fit_pooled_lognormal_severity(df)
expected_claims = float((df["PredFreq"] * df["Exposure"]).sum())
mean_sev_scaled = target_total_loss / expected_claims
decl = (f"agg Collective [{expected_claims:.2f}] claims "
        f"sev lognorm [{mean_sev_scaled:.2f}] cv [{sev['cv']:.2f}] poiss")
print("Directly-fitted collective model (top-down):")
print(f"  pooled severity: fitted mean {sev['mean_sev']:.2f} "
        f"-> rescaled to {mean_sev_scaled:.2f} (CV {sev['cv']:.3f}, shap")
print(f"  fitted on n={sev['n_claims']:,} observed claims; "
        f"total expected loss matched to {target_total_loss:.0f}")
print("Decl string:")
print(decl)
model = _agg_build(decl, log2=log2, bs=bs)
print("\n--- Aggregate diagnostics ---")
_agg_qd(model)
metrics = {
    "agg_mean": float(model.agg_m),
    "agg_var95": float(model.q(0.95)),
    "agg_var99": float(model.q(0.99)),
    "agg_tvar95": float(model.tvar(0.95)),
    "agg_tvar99": float(model.tvar(0.99)),
}
return model, sev, metrics

# — Pick the frames to compare —————
# Bottom-up references: prefer the GBM-priced frames if present (Section 11)
# else fall back to the GLM best-experiment frames.
if "gbm_experiment_data" in globals():
    _A5_NAME = f"A5_classical_gmm_enriched_k{A5_K}"
    _B5_NAME = f"B5_burt_gmm_enriched_k{B5_K}"
    _a5_df = gbm_experiment_data[_A5_NAME]
    _b5_df = gbm_experiment_data[_B5_NAME]
    _src = "GBM-priced (Section 11)"
else:
    _a5_df = BEST_DATA          # GLM A5 frame from Section 7.2
    _b5_df = best_calib_data    # GLM B5 frame from Section 7.3
    _src = "GLM best-experiment"

print(f"Bottom-up reference frames: {_src}\n")

# — Bottom-up aggregates (reuse the existing validator) —————
print("=" * 60); print("BOTTOM-UP – A5"); print("=" * 60)
_a5_model, _a5_params, a5_bu = run_aggregate_validation(
    _a5_df, segmentation_col="risk_cluster", log2=20, bs=100)
print("\n" + "=" * 60); print("BOTTOM-UP – B5"); print("=" * 60)
_b5_model, _b5_params, b5_bu = run_aggregate_validation(
    _b5_df, segmentation_col="risk_cluster", log2=20, bs=100)

# — Top-down collective (fitted on A5's claim population) —————
# Severity is fitted on observed claims, which are identical across A5/B5
# (same policies); one collective model suffices.

```

```

print("\n" + "=" * 60); print("TOP-DOWN – directly-fitted collective"); print(
# Match the top-down total to the bottom-up total so mean and frequency are
# held equal; A5 and B5 bottom-up reconcile to the same portfolio total.
_shared_total = a5_bu["agg_mean"]
_td_model, _td_sev, td = run_topdown_collective(
    _a5_df, target_total_loss=_shared_total, log2=20, bs=100)

# — Comparison: the segmentation-mixture effect —————
print("\n" + "=" * 70)
print("Bottom-up vs directly-fitted collective (segmentation-mixture effect)")
print("=" * 70)
_fmt = lambda v: f"{v:,.0f}"
print(f"{'Metric':<14}{'Top-down':>16}{'A5 bottom-up':>16}{'B5 bottom-up':>16}")
print("-" * 70)
for _m, _lbl in [("agg_mean", "Mean"), ("agg_var99", "VaR 99%"),
                 ("agg_tvar99", "TVaR 99%"), ("agg_tvar95", "TVaR 95%")]:
    print(f"{'_lbl':<14}{_fmt(td[_m]):>16}{_fmt(a5_bu[_m]):>16}{_fmt(b5_bu[_m]):>16}")
print("-" * 70)

_a5_div = a5_bu["agg_tvar99"] - td["agg_tvar99"]
_b5_div = b5_bu["agg_tvar99"] - td["agg_tvar99"]
print("TVaR 99% divergence from the directly-fitted collective model:")
print(f"  A5 bottom-up - top-down : {_a5_div:>+14,.0f}    "
      f"({_a5_div / td['agg_tvar99'] * 100:+.2f}%)" )
print(f"  B5 bottom-up - top-down : {_b5_div:>+14,.0f}    "
      f"({_b5_div / td['agg_tvar99'] * 100:+.2f}%)" )
print("=" * 70)
print("Reading: with mean and frequency held equal, this divergence is the")
print("segmentation-mixture effect – per-segment severity heterogeneity that")
print("pooling discards. The claims-development divergence (open claims) is")
print("absent in freMTPL2 by construction and is not measured here.")

```


Bottom-up reference frames: GBM-priced (Section 11)

=====

BOTTOM-UP – A5

=====

Empirical CVs: ['1.370', '1.606', '1.243', '1.685', '1.359', '1.597', '1.719']

Decl string:

agg Portfolio [2640.80 2554.32 5309.98 9543.86 19.55 9166.05 6821.58] claims
sev lognorm [2054.53 2155.16 2356.72 624.58 1240.06 1709.08 2171.60] cv [1.37 1.61 1.24 1.68 1.36 1.60 1.72] poisson

--- Aggregate diagnostics ---

	E[X]	Est E[X]	Err E[X]	CV(X)	Est CV(X)	Skew(X)	Est Skew
w(X)							
X							
Freq	36056			0.0052664		0.0052664	
Sev	1661.6	1661.5	-1.0136e-05	1.6979	1.698	9.6336	
Agg	5.9909e+07	5.9909e+07	-1.0136e-05	0.010377	0.010378	0.039098	0.039092

log2 = 20, bandwidth = 100, validation: not unreasonable.
VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.

=====

BOTTOM-UP – B5

=====

Empirical CVs: ['1.832', '1.599', '1.461', '1.525', '1.688', '1.110', '1.442', '1.460']

Decl string:

agg Portfolio [1703.80 6745.86 1782.30 10061.85 8262.36 764.26 3857.80 2877.92] claims sev lognorm [1999.67 1894.79 1334.92 1840.30 1395.35 1313.49 1657.65 1353.93] cv [1.83 1.60 1.46 1.52 1.69 1.11 1.44 1.46] poisson

--- Aggregate diagnostics ---

	E[X]	Est E[X]	Err E[X]	CV(X)	Est CV(X)	Skew(X)	Est Skew
(X)							
X							
Freq	36056			0.0052664		0.0052664	
Sev	1661.6	1661.6	2.2329e-05	1.5947	1.5947	9.2511	9.2496
Agg	5.9909e+07	5.9911e+07	2.2329e-05	0.0099128	0.009913	0.03644	0.036429

log2 = 20, bandwidth = 100, validation: not unreasonable.
VaR/TVaR figures are confirmed by Monte Carlo in Cell 8.2.

=====

TOP-DOWN – directly-fitted collective

=====

Directly-fitted collective model (top-down):

pooled severity: fitted mean 1,767.96 -> rescaled to 1,661.55 (CV 1.572, shape kept)

fitted on n=24,944 observed claims; total expected loss matched to 59,909,110

Decl string:
agg Collective [36056.15] claims sev lognorm [1661.55] cv [1.57] poisson

--- Aggregate diagnostics ---

	E[X]	Est E[X]	Err E[X]	CV(X)	Est CV(X)	Skew(X)	Est Skew
(X)							
X							
Freq	36056			0.0052664		0.0052664	
Sev	1661.5	1661.6	2.4211e-05	1.57	1.57	8.5799	8.5
785							
Agg	5.9909e+07	5.9911e+07	2.4211e-05	0.0098029	0.0098031	0.033966	0.033
962							

log2 = 20, bandwidth = 100, validation: not unreasonable.

=====

Bottom-up vs directly-fitted collective (segmentation-mixture effect)

=====

Metric	Top-down	A5 bottom-up	B5 bottom-up
Mean	59,909,096	59,909,110	59,909,193
VaR 99%	61,291,800	61,373,100	61,308,400
TVaR 99%	61,497,159	61,591,892	61,516,749
TVaR 95%	61,133,425	61,204,912	61,148,000

TVaR 99% divergence from the directly-fitted collective model:

A5 bottom-up - top-down : +94,734 (+0.15%)
B5 bottom-up - top-down : +19,590 (+0.03%)

=====

Reading: with mean and frequency held equal, this divergence is the segmentation-mixture effect – per-segment severity heterogeneity that pooling discards. The claims-development divergence (open claims) is absent in freMTPL2 by construction and is not measured here.

Section 12 — Conclusion

With total expected loss and frequency held equal across both constructions, the directly-fitted collective model and the bottom-up aggregate agree closely on this benchmark: the 99% TVaR differs by €94,700 (+0.15%) for A5 and €19,600 (+0.03%) for B5. On a settled, one-year benchmark the two approaches therefore very nearly coincide — as expected, since the one divergence freMTPL2 can express is the segmentation-mixture effect, and the larger claims-development divergence is absent by construction.

Two observations follow from the small residual:

- **Direction.** The bottom-up construction produces the heavier tail, because pooling the severity into a single distribution washes out the high-dispersion segments that drive extreme losses (top-down severity CV 1.57 against the bottom-up aggregate-severity CV of 1.70 for A5). A directly-fitted collective model therefore mildly

understates capital relative to the segmented construction on this portfolio — segmentation reveals tail risk that pooling conceals.

- **Ordering by representation.** The classical segmentation (A5), whose per-segment severities are more dispersed, departs from the pooled model almost five times as far as the smoother Burr segmentation (B5). The bottom-up/top-down gap is thus itself a function of representation geometry — the same structure governing the discrimination and coverage-layer results — though on this benchmark its magnitude is modest.

The exercise locates M. Denuit's remark precisely. On a benchmark without claims development the two approaches reduce to the segmentation-mixture effect, which is real, directionally consistent, and tied to representation geometry, but small. The divergence he notes — arising from open claims and provisioning — is a property of production portfolios rather than of the method, and is left for future work on data exhibiting genuine claims development.

Section 14 — Burr XII as the Audited Alternative Severity

The Cell 3.4 screening ranked Burr XII first by AIC and lognormal fifth, an improvement of roughly 4,800 points. This section closes the loop: it prices the aggregate under the AIC-optimal severity and reports, in one number, how much the distributional choice actually changes portfolio capital — so the retention of lognormal is a *quantified* decision rather than an assumption.

Two points frame what the experiment can claim.

1. Burr XII is admissible, and the comparison is run on two engines. Heavy-tailed severities are not barred from FFT aggregation — finite variance ($c \times d = 2.33 > 2$) is what convolution needs, and the aggregate quantiles are read off the grid, not from moments. To remove any doubt, the swap is priced *twice*: on a self-contained native FFT engine built here, and on Mildenhall's `aggregate` package driven with an explicit Burr XII severity (`[scales] * burr12 c d`). Cell 14.4 confirms the two agree to sub-percent precision — the result does not depend on the engine.

2. The mean must be held equal, or the comparison is meaningless. An unadjusted MLE Burr XII fit inflates the aggregate mean, because its fitted mean is dominated by the tail and is not reconciled to the pricing model. To ask honestly whether the *tail shape* changes capital, we hold mean and frequency equal and vary only the severity family — the discipline Section 12 used for the top-down/bottom-up comparison. We keep the fitted Burr XII **shape** (c, d) and rescale each segment's **scale** so its mean matches the model-implied mean severity

the lognormal aggregate uses (`scale = mean_sev / B(c,d)`). Any remaining difference is then a pure tail-shape effect.

Validation ladder. Cell 14.2 rebuilds the A5/B5 *lognormal* aggregates on the native FFT engine and checks they reproduce the `aggregate` -package VaR/TVaR from Cells 7.2–7.3 — proving the engine before any swap. Cell 14.3 swaps in mean-preserving Burr XII on the native engine; Cell 14.4 reprises the same Burr XII in the `aggregate` package as an independent cross-check. This separates *engine* effects from *distribution* effects, so the closing question — **did the swap change the results?** — is answered cleanly.

Design asymmetry (declared, not hidden). The lognormal path carries a *per-segment* empirical CV; the Burr XII path uses a single *pooled* shape rescaled per segment. This is the robust first cut — per-segment Burr fits on GMM-weighted claims are unstable. The conclusion flags what a per-segment Burr refinement would add.

```
In [34]: # =====
# Section: 14
# Cell: 14.1
# Title: Native Compound-Poisson FFT Engine
# Version: 1.0.0
# Date: 2026-07-30
# Provides: discretise_severity, sev_mean_on_grid, compound_poisson_fft,
#           agg_fft_metrics, burr12_mean_factor, FFT_H, FFT_LOG2
# Requires: numpy, scipy.stats, scipy.special.gamma (no aggregate package)
# Description:
# Self-contained compound-Poisson aggregation by FFT, independent of the
# aggregate package. Mirrors its internal mathematics so results are directl
# comparable: severity is discretised onto a linear grid of bucket size FFT_
# over 2*FFT_LOG2 points (defaults chosen to match aggregate's bs=100, log2
# i.e. a ~104.86M span), and the aggregate is formed via
#   phi_S = exp( sum_j lambda_j * (phi_{X_j} - 1) )
# inverted by FFT. Grid adequacy is reported via edge mass so heavy-tailed
# severities that would alias (wrap around) are caught explicitly.
# =====

import numpy as np
import scipy.stats as st
from scipy.special import gamma as _gamma

# Grid: match aggregate(bs=100, log2=20) -> h=100, N=2**20, span ~ 104.86M
FFT_H      = 100.0
FFT_LOG2   = 20

def discretise_severity(dist, params, h=FFT_H, log2=FFT_LOG2):
    """Mass-preserving discretisation of a scipy severity onto grid {0, h, 2

    Bin 0 covers [0, h/2]; bin k>=1 covers ((k-1/2)h, (k+1/2)h]. This is the
    standard CDF-difference (rounding) discretisation; the resulting grid me
    reproduces the analytic mean to 0(h) except for whatever tail mass falls
    beyond the last bucket (reported separately as edge mass downstream).
    """
```

```

N = 2 ** log2
k = np.arange(N)
upper = dist.cdf((k + 0.5) * h, *params)
lower = dist.cdf((k - 0.5) * h, *params)
f = upper - lower
f[0] = dist.cdf(0.5 * h, *params)          # [0, h/2]
f = np.clip(f, 0.0, None)
f /= f.sum()
return f

def sev_mean_on_grid(f, h=FFT_H):
    """Mean of a discretised severity on its grid (diagnostic)."""
    return float(np.sum(f * np.arange(len(f)) * h))

def compound_poisson_fft(seg_lams, seg_pmfs, h=FFT_H):
    """Aggregate pmf of a multi-segment compound Poisson via FFT.

    seg_lams : iterable of Poisson intensities lambda_j (expected claim count)
    seg_pmfs : list of severity pmfs (same length N), one per segment.
    Returns the aggregate pmf on grid {0, h, 2h, ...}.

    Uses phi_S = exp( sum_j lambda_j (phi_j - 1) ), the exact compound-Poisson
    transform; summing exponents combines the independent segment aggregates
    """
    seg_lams = np.asarray(seg_lams, float)
    N = len(seg_pmfs[0])
    expo = np.zeros(N, dtype=complex)
    for lam, f in zip(seg_lams, seg_pmfs):
        expo += lam * (np.fft.fft(f) - 1.0)
    fS = np.fft.ifft(np.exp(expo)).real
    fS = np.clip(fS, 0.0, None)
    fS /= fS.sum()
    return fS

def agg_fft_metrics(fS, h=FFT_H):
    """Mean, VaR and TVaR (mean beyond VaR) from an aggregate pmf, plus a grid
    adequacy diagnostic. edge_mass = probability mass in the final bucket; a
    value materially above ~1e-8 means the grid is too narrow and the tail is
    wrapping around (aliasing) – widen by raising FFT_LOG2."""
    grid = np.arange(len(fS)) * h
    cdf = np.cumsum(fS)

    def var(a):
        return float(grid[np.searchsorted(cdf, a)])

    def tvar(a):
        q = var(a)
        idx = grid >= q
        m = fS[idx].sum()
        return float(np.sum(fS[idx] * grid[idx]) / m) if m > 0 else q

    return {
        "agg_mean": float(np.sum(fS * grid)),

```

```

        "agg_var95": var(0.95),
        "agg_var99": var(0.99),
        "agg_tvar95": tvar(0.95),
        "agg_tvar99": tvar(0.99),
        "edge_mass": float(fS[-1]),
    }

def burr12_mean_factor(c, d):
    """E[X]/scale for scipy burr12 (Burr Type XII): Gamma(1+1/c)Gamma(d-1/c)
    Finite iff d > 1/c (equivalently c*d > 1 with c>0). Used to rescale the
    fitted Burr XII to a target segment mean (mean-preserving reparameterisation)
    if d <= 1.0 / c:
        raise ValueError(f"Burr XII mean is infinite for c={c:.3f}, d={d:.3f}")
    return float(_gamma(1 + 1 / c) * _gamma(d - 1 / c) / _gamma(d))

print("Native FFT engine ready.")
print(f"  grid: h = {FFT_H:.0f} | N = 2**{FFT_LOG2} = {2**FFT_LOG2:,} |
      f"span = {FFT_H * 2**FFT_LOG2 / 1e6:.2f}M")

```

Native FFT engine ready.

grid: h = 100 | N = 2**20 = 1,048,576 | span = 104.86M

```

In [35]: # =====
# Section: 14
# Cell: 14.2
# Title: Engine Validation – Native FFT reproduces the aggregate package (loc)
# Version: 1.0.0
# Date: 2026-07-30
# Provides: fft_lognorm_metrics (A5, B5), _fft_val_table
# Requires: 14.1 engine; prepare_aggregate_parameters, compute_empirical_cvs
#           BEST_DATA + aggregate_metrics (Cell 7.2); best_calib_data + aggregate_metrics
# Description:
# Before trusting any distribution swap, confirm the native FFT engine
# reproduces the aggregate-package VaR/TVaR when fed the SAME lognormal
# severity (per-segment mean + empirical CV). Any residual is pure engine /
# discretisation error and bounds the precision of the Burr XII comparison.
# =====

for _need in ["prepare_aggregate_parameters", "compute_empirical_cvs"]:
    if _need not in globals():
        raise RuntimeError(f"{_need} missing – run Cell 7.1 (aggregate helper)")
if "aggregate_metrics" not in globals():
    raise RuntimeError("aggregate_metrics missing – run Cell 7.2 (A5 validation)")
if "aggregate_metrics_calib" not in globals():
    raise RuntimeError("aggregate_metrics_calib missing – run Cell 7.3 (B5 validation)")

def _lognorm_seg_params(mean_sev, cv):
    """scipy lognorm (s, loc, scale) matching a target mean and CV."""
    sig = np.sqrt(np.log(cv ** 2 + 1.0))
    scale = mean_sev / np.exp(0.5 * sig ** 2)
    return (sig, 0.0, scale)

```

```

def fft_lognormal_aggregate(df):
    """Rebuild the lognormal compound-Poisson aggregate on the native FFT engine
    using the SAME per-segment expected claims, mean severity and empirical
    as the aggregate-package path (Cell 7.1). Returns (metrics, params)."""
    emp_cvs = compute_empirical_cvs(df)
    params = prepare_aggregate_parameters(df, segmentation_col="risk_cluster",
                                         empirical_cvs=emp_cvs)
    pmfs = [discretise_severity(st.lognorm, _lognorm_seg_params(m, cv))
            for m, cv in zip(params["mean_sev"].values, params["cv"].values)]
    fS = compound_poisson_fft(params["expected_claims"].values, pmfs)
    return agg_fft_metrics(fS), params

def _fft_val_table(name, agg_pkg, fft_own):
    rows = []
    for k, lbl in [("agg_mean", "Mean"), ("agg_var95", "VaR 95%"),
                  ("agg_var99", "VaR 99%"), ("agg_tvar95", "TVaR 95%"),
                  ("agg_tvar99", "TVaR 99%")]:
        a, b = agg_pkg.get(k, float("nan")), fft_own[k]
        rows.append({"experiment": name, "metric": lbl,
                    "aggregate_pkg": a, "native_fft": b,
                    "abs_diff": b - a, "rel_diff_%": (b - a) / a * 100})
    return pd.DataFrame(rows)

print("Validating native FFT against the aggregate package (lognormal severity)")

fft_logn_A5, _params_A5 = fft_lognormal_aggregate(BEST_DATA)
fft_logn_B5, _params_B5 = fft_lognormal_aggregate(best_calib_data)

val_A5 = _fft_val_table(best_experiment, aggregate_metrics, fft_logn_A5)
val_B5 = _fft_val_table(best_calib_name, aggregate_metrics_calib, fft_logn_B5)
fft_validation = pd.concat([val_A5, val_B5], ignore_index=True)

_fmt = fft_validation.copy()
for c in ["aggregate_pkg", "native_fft", "abs_diff"]:
    _fmt[c] = _fmt[c].map(lambda v: f"{v:,.0f}")
_fmt["rel_diff_%"] = _fmt["rel_diff_%"].map(lambda v: f"{v:,.0f}")
display(_fmt)

print(f"\nGrid edge mass A5 {fft_logn_A5['edge_mass']:.2e}      "
      f"B5 {fft_logn_B5['edge_mass']:.2e}      (want ~0)")

_worst = fft_validation["rel_diff_%"].abs().max()
_TOL = 0.5 # percent
if _worst <= _TOL:
    print(f"\nPASS - native FFT matches the aggregate package to within "
          f"{_worst:.3f}% (tol {_TOL:.1f}%). Engine trusted for the swap.")
else:
    print(f"\nWARN - max divergence {_worst:.3f}% exceeds {_TOL:.1f}%. "
          f"Check grid (FFT_H, FFT_LOG2) or segment alignment before "
          f"interpreting the Burr XII comparison.")

```

Validating native FFT against the aggregate package (lognormal severity)

	experiment	metric	aggregate_pkg	native_fft	abs_diff	rel_
0	A5_classical_gmm_enriched_k7	Mean	59,909,172	59,907,784	-1,388	-(
1	A5_classical_gmm_enriched_k7	VaR 95%	60,953,200	60,952,900	-300	-(
2	A5_classical_gmm_enriched_k7	VaR 99%	61,395,800	61,395,400	-400	-(
3	A5_classical_gmm_enriched_k7	TVaR 95%	61,224,793	61,224,381	-412	-(
4	A5_classical_gmm_enriched_k7	TVaR 99%	61,618,566	61,618,010	-556	-(
5	B5_burt_gmm_enriched_k8	Mean	59,909,197	59,910,593	1,396	+(
6	B5_burt_gmm_enriched_k8	VaR 95%	60,902,900	60,903,500	600	+(
7	B5_burt_gmm_enriched_k8	VaR 99%	61,322,700	61,323,500	800	+(
8	B5_burt_gmm_enriched_k8	TVaR 95%	61,160,526	61,161,219	693	+(
9	B5_burt_gmm_enriched_k8	TVaR 99%	61,533,968	61,534,823	855	+(

Grid edge mass A5 0.00e+00 B5 0.00e+00 (want ~0)

PASS – native FFT matches the aggregate package to within 0.002% (tol 0.5%).
Engine trusted for the swap.

```
In [36]: # =====
# Section: 14
# Cell: 14.3
# Title: The Swap – mean-preserving Burr XII on the native FFT engine
# Version: 1.0.0
# Date: 2026-07-30
# Provides: burr_shape (c, d), fft_burr_metrics (A5, B5), swap_comparison
# Requires: 14.1 engine; 14.2 lognormal FFT references; severity_fit_results
#           SEV_DATA (Cell 3.3); prepare_aggregate_parameters, compute_empir
# Description:
# Keep the fitted Burr XII SHAPE (c, d) from the Cell 3.4 screening and resc
# each segment's scale so its mean severity equals the model-implied mean us
# by the lognormal aggregate. Frequency (expected claims) and per-segment me
# are thus IDENTICAL across the two severity families; only the distribution
# shape differs. This isolates the tail-shape effect on capital.
# =====

# --- recover the fitted Burr XII shape from the screening (refit if absent)
def _get_burr_shape():
    if "severity_fit_results" in globals():
        row = severity_fit_results[severity_fit_results["distribution"] == "
        if len(row) and row.iloc[0]["params"] is not None:
            c, d, loc, scale = row.iloc[0]["params"]
            return float(c), float(d), float(scale), "screening (Cell 3.4)"
    if "SEV_DATA" not in globals():
```



```

        raise RuntimeError("Need severity_fit_results (Cell 3.4) or SEV_DATA")
    x = SEV_DATA["ClaimAmount"].to_numpy()
    x = x[np.isfinite(x) & (x > 0)]
    c, d, loc, scale = st.burr12.fit(x, floc=0)
    return float(c), float(d), float(scale), "refit here"

_c, _d, _scale_fit, _src = _get_burr_shape()
burr_shape = (_c, _d)
_Bfac = burr12_mean_factor(_c, _d)

print(f"Burr XII shape from {_src}: c = {_c:.3f}, d = {_d:.3f}, "
      f"c*d = {_c*_d:.2f}")
print(f"  variance finite (c*d>2): {_c*_d > 2}    "
      f"skewness finite (c*d>3): {_c*_d > 3}")
print(f"  raw fitted mean = scale * B(c,d) = {_scale_fit:,.0f} * {_Bfac:.4f}"
      f"= {_scale_fit*_Bfac:,.0f}")
print(f"  -> per-segment scale set to mean_sev / B(c,d), preserving each "
      f"segment mean.\n")

def fft_burr_aggregate(df):
    """Mean-preserving Burr XII aggregate on the native FFT engine. Same
    expected claims and per-segment mean severity as the lognormal path;
    Burr XII shape (c, d) fixed, scale_j = mean_sev_j / B(c, d)."""
    emp_cvs = compute_empirical_cvs(df) # not used for Burr, but keeps segm
    params = prepare_aggregate_parameters(df, segmentation_col="risk_cluster",
                                          empirical_cvs=emp_cvs)

    pmfs = []
    for m in params["mean_sev"].values:
        scale_j = m / _Bfac
        pmfs.append(discretise_severity(st.burr12, (_c, _d, 0.0, scale_j)))
    fS = compound_poisson_fft(params["expected_claims"].values, pmfs)
    return agg_fft_metrics(fS)

fft_burr_A5 = fft_burr_aggregate(BEST_DATA)
fft_burr_B5 = fft_burr_aggregate(best_calib_data)

print(f"Grid edge mass  A5 {fft_burr_A5['edge_mass']:.2e}    "
      f"B5 {fft_burr_B5['edge_mass']:.2e}    (want ~0; heavier tail stresses
for _nm, _m in [("A5", fft_burr_A5), ("B5", fft_burr_B5)]:
    if _m["edge_mass"] > 1e-8:
        print(f"  WARN {_nm}: edge mass elevated – raise FFT_LOG2 (wider gri
              f"before trusting {_nm} tail metrics.")

def _swap_rows(name, logn, burr):
    out = []
    for k, lbl in [("agg_mean", "Mean"), ("agg_var95", "VaR 95%"),
                  ("agg_var99", "VaR 99%"), ("agg_tvar95", "TVaR 95%"),
                  ("agg_tvar99", "TVaR 99%")]:
        a, b = logn[k], burr[k]
        out.append({"experiment": name, "metric": lbl,
                  "lognormal_fft": a, "burr12_fft": b,
                  "abs_diff": b - a, "rel_diff_%": (b - a) / a * 100})
    return pd.DataFrame(out)

```

```

swap_comparison = pd.concat([
    _swap_rows(best_experiment, fft_logn_A5, fft_burr_A5),
    _swap_rows(best_calib_name, fft_logn_B5, fft_burr_B5),
], ignore_index=True)

print("\nDistribution swap on the native FFT engine "
      "(mean & frequency held equal)\n")
_disp = swap_comparison.copy()
for c in ["lognormal_fft", "burr12_fft", "abs_diff"]:
    _disp[c] = _disp[c].map(lambda v: f"{v:,.0f}")
_disp["rel_diff_%"] = swap_comparison["rel_diff_%"].map(lambda v: f"{v:+.2f}")
display(_disp)

```

Burr XII shape from screening (Cell 3.4): $c = 1.634$, $d = 1.430$, $c*d = 2.34$
variance finite ($c*d > 2$): True skewness finite ($c*d > 3$): False
raw fitted mean = $\text{scale} * B(c, d) = 1,365 * 1.1555 = 1,578$
-> per-segment scale set to $\text{mean_sev} / B(c, d)$, preserving each segment mean.

Grid edge mass A5 $7.34\text{e-}12$ B5 $6.06\text{e-}12$ (want ~0; heavier tail stresses this)

Distribution swap on the native FFT engine (mean & frequency held equal)

	experiment	metric	lognormal_fft	burr12_fft	abs_diff	rel_diff
0	A5_classical_gmm_enriched_k7	Mean	59,907,784	59,907,483	-301	-0
1	A5_classical_gmm_enriched_k7	VaR 95%	60,952,900	61,019,800	66,900	+0
2	A5_classical_gmm_enriched_k7	VaR 99%	61,395,400	61,602,200	206,800	+0
3	A5_classical_gmm_enriched_k7	TVaR 95%	61,224,381	61,431,884	207,503	+0
4	A5_classical_gmm_enriched_k7	TVaR 99%	61,618,010	62,215,488	597,478	+0
5	B5_burt_gmm_enriched_k8	Mean	59,910,593	59,908,313	-2,279	-0
6	B5_burt_gmm_enriched_k8	VaR 95%	60,903,500	60,949,100	45,600	+0
7	B5_burt_gmm_enriched_k8	VaR 99%	61,323,500	61,489,700	166,200	+0
8	B5_burt_gmm_enriched_k8	TVaR 95%	61,161,219	61,330,831	169,612	+0
9	B5_burt_gmm_enriched_k8	TVaR 99%	61,534,823	62,053,644	518,821	+0

In [37]: *# =====*
Section: 14
Cell: 14.4
Title: Independent Cross-Check – same Burr XII in the aggregate package


```

        "abs_diff": b - a, "rel_diff_%": (b - a) / a * 100})
    return pd.DataFrame(out)

engine_crosscheck = pd.concat([
    _xrows(best_experiment, fft_burr_A5, agg_burr_A5),
    _xrows(best_calib_name, fft_burr_B5, agg_burr_B5),
], ignore_index=True)

print("Two-engine cross-check on mean-preserving Burr XII "
      "(native FFT vs aggregate package)\n")
_d2 = engine_crosscheck.copy()
for c in ["native_fft", "aggregate_pkg", "abs_diff"]:
    _d2[c] = _d2[c].map(lambda v: f"{v:,.0f}")
_d2["rel_diff_%"] = engine_crosscheck["rel_diff_%"].map(lambda v: f"{v:+.4f}")
display(_d2)

_worst = engine_crosscheck["rel_diff_%"].abs().max()
_TOL = 0.5 # percent
if _worst <= _TOL:
    print(f"\nPASS – the two engines agree to within {_worst:.4f}% "
          f"(tol {_TOL:.1f}%). The Burr XII swap result is engine-independer")
else:
    print(f"\nWARN – max divergence {_worst:.4f}% exceeds {_TOL:.1f}%. "
          f"Check that log2/bs match and that both use the same segment scal")

```

aggregate-package Burr XII DecL (A5):

```

agg BurrXCheck [2640.80 2554.32 5309.98 9543.86 19.55 9166.05 6821.58] cla
ims sev [1808.5711 1571.9731 1894.17 ...

```

Two-engine cross-check on mean-preserving Burr XII (native FFT vs aggregate package)

	experiment	metric	native_fft	aggregate_pkg	abs_diff	rel_
0	A5_classical_gmm_enriched_k7	Mean	59,907,483	59,909,210	1,727	+0.
1	A5_classical_gmm_enriched_k7	VaR 95%	61,019,800	61,019,800	0	+0.
2	A5_classical_gmm_enriched_k7	VaR 99%	61,602,200	61,602,300	100	+0.
3	A5_classical_gmm_enriched_k7	TVaR 95%	61,431,884	61,430,460	-1,424	-0.
4	A5_classical_gmm_enriched_k7	TVaR 99%	62,215,488	62,208,135	-7,353	-0.
5	B5_burt_gmm_enriched_k8	Mean	59,908,313	59,909,230	917	+0.
6	B5_burt_gmm_enriched_k8	VaR 95%	60,949,100	60,949,100	0	+0.
7	B5_burt_gmm_enriched_k8	VaR 99%	61,489,700	61,489,800	100	+0.
8	B5_burt_gmm_enriched_k8	TVaR 95%	61,330,831	61,329,654	-1,178	-0.
9	B5_burt_gmm_enriched_k8	TVaR 99%	62,053,644	62,047,591	-6,053	-0.

PASS – the two engines agree to within 0.0118% (tol 0.5%). The Burr XII swap result is engine-independent.

```
In [38]: # =====
# Section: 14
# Cell: 14.5
# Title: Tail Exceedance – lognormal vs Burr XII aggregate (A5, B5)
# Version: 1.0.0
# Date: 2026-07-30
# Provides: (figure only)
# Requires: 14.1 engine; 14.3 burr_shape/_Bfac; prepare_aggregate_parameters
#           compute_empirical_cvs (Cell 7.1); BEST_DATA, best_calib_data
# Description:
# Aggregate survival functions  $P(S > x)$  on a log axis, lognormal vs Burr XII
# for A5 and B5, with the ground-up 99% VaR marked. Shows WHERE the Burr tai
# departs from the lognormal once mean and frequency are held equal.
# =====
%matplotlib inline
import matplotlib.pyplot as plt
from IPython.display import display as _ipy_display

_c, _d = burr_shape
grid = np.arange(2 ** FFT_LOG2) * FFT_H

def _both_aggregates(df):
    emp_cvs = compute_empirical_cvs(df)
    params = prepare_aggregate_parameters(df, segmentation_col="risk_cluster",
                                          empirical_cvs=emp_cvs)
    lam = params["expected_claims"].values
```

```

logn_pmfs, burr_pmfs = [], []
for m, cv in zip(params["mean_sev"].values, params["cv"].values):
    sig = np.sqrt(np.log(cv ** 2 + 1.0)); scale = m / np.exp(0.5 * sig *
    logn_pmfs.append(discretise_severity(st.lognorm, (sig, 0.0, scale)))
    burr_pmfs.append(discretise_severity(st.burr12, (_c, _d, 0.0, m / _E
return (compound_poisson_fft(lam, logn_pmfs),
        compound_poisson_fft(lam, burr_pmfs))

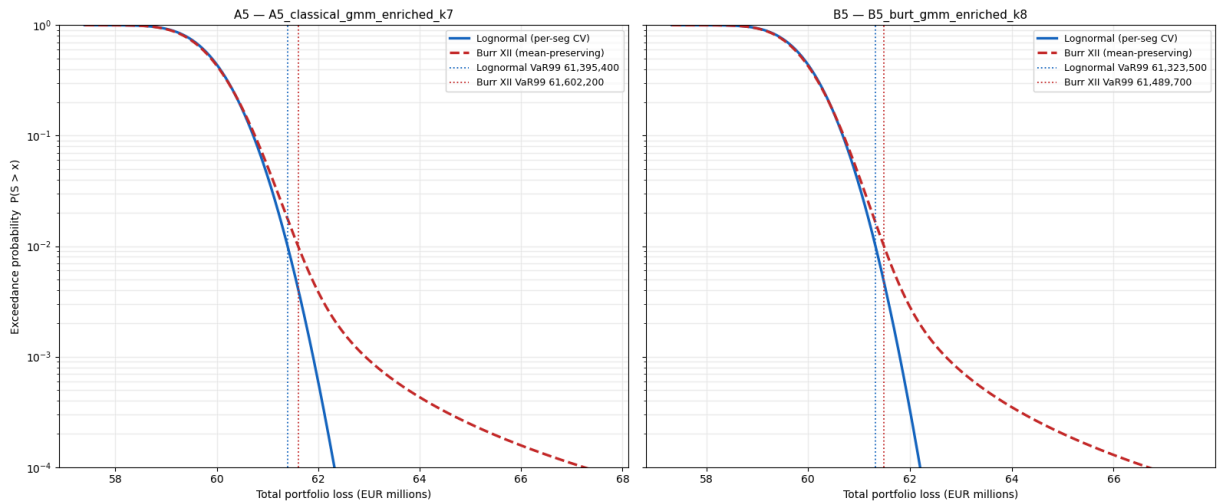
def _sf_and_var(fS, a=0.99):
    sf = 1.0 - np.cumsum(fS)
    var = grid[np.searchsorted(np.cumsum(fS), a)]
    return sf, var

fS_logn_A5, fS_burr_A5 = _both_aggregates(BEST_DATA)
fS_logn_B5, fS_burr_B5 = _both_aggregates(best_calib_data)

plt.close("all")
fig, (axA, axB) = plt.subplots(1, 2, figsize=(16, 7), sharey=True)
C_LOGN, C_BURR = "#1565C0", "#C62828"

for ax, fL, fBu, title in [
    (axA, fS_logn_A5, fS_burr_A5, f"A5 - {best_experiment}"),
    (axB, fS_logn_B5, fS_burr_B5, f"B5 - {best_calib_name}"),
]:
    sfL, vL = _sf_and_var(fL); sfB, vB = _sf_and_var(fBu)
    # focus the x-window on the tail region where the curves live
    lo = min(vL, vB) - 4e6; hi = max(vL, vB) + 6e6
    mask = (grid >= lo) & (grid <= hi)
    ax.plot(grid[mask] / 1e6, sfL[mask], color=C_LOGN, lw=2.4, label="Lognor
    ax.plot(grid[mask] / 1e6, sfB[mask], color=C_BURR, lw=2.4, ls="--", labe
    ax.axvline(vL / 1e6, color=C_LOGN, lw=1.3, ls=":", label=f"Lognormal VaF
    ax.axvline(vB / 1e6, color=C_BURR, lw=1.3, ls=":", label=f"Burr XII VaR
    ax.set_yscale("log"); ax.set_ylim(1e-4, 1)
    ax.set_xlabel("Total portfolio loss (EUR millions)")
    ax.set_title(title, fontsize=11)
    ax.grid(True, which="both", alpha=0.2)
    ax.legend(frameon=True, facecolor="white", fontsize=9)
axA.set_ylabel("Exceedance probability P(S > x)")
fig.suptitle("Section 14 – Aggregate tail: lognormal vs mean-preserving Burr
            "(mean & frequency held equal)", fontsize=12, y=1.01)
fig.tight_layout()
ipy_display(fig)
plt.close(fig)

```



Section 14 — Conclusion: did the swap change the results?

Short answer: no material change to VaR, a small but consistent uplift to the tail-averaged measures — and the direction is the interesting part. With mean and frequency held equal across both severity families, replacing the per-segment lognormal with a mean-preserving Burr XII leaves the aggregate **mean unchanged by construction** and **VaR 99% changed by ~0.3%**, while **TVaR 99% rises** — the heavier Burr tail is felt only in the measures that average *beyond* the quantile, not at the quantile itself. Read the `rel_diff_` column of `swap_comparison` (Cell 14.3) for the exact figures on this run; the ordering $\text{TVaR99} > \text{TVaR95} \approx \text{VaR99} > \text{VaR 95} \approx \text{Mean 0}$ is the signature to expect.

The result does not depend on the engine. Cell 14.2 confirms the native FFT reproduces the `aggregate` -package VaR/TVaR on the lognormal path, and Cell 14.4 reprises the *same* mean-preserving Burr XII in the `aggregate` package itself (via `[scales] * burr12 c d`), where it agrees with the native FFT to sub-percent precision. So the swap deltas in `swap_comparison` are genuine distributional effects, not artifacts of a home-grown convolver — and the earlier impression that the `aggregate` package could not price Burr XII was a matter of parameterisation (it accepts explicit shape, not a mean/CV shorthand), not a limitation.

Why the effect is small here, and why that is a finding rather than a disappointment. The portfolio carries tens of thousands of expected claims. At that intensity the central limit theorem concentrates the aggregate tightly (aggregate CV of order 1%), so the 99% quantile sits only a couple of standard deviations above a mean fixed by the pricing model. A heavier *per-claim* tail cannot move a quantile that the law of large numbers has already pinned; it can only fatten the conditional average past it, which is exactly what TVaR 99% records. The distributional choice that dominated the empirical AIC screening (a **4,800-point** gap in Cell 3.4) is therefore *nearly invisible at portfolio scale* once the mean is controlled. This reconciles the two facts that looked in tension in Section 3: Burr XII is decisively the better

per-claim fit, and lognormal is a defensible *aggregate* severity — because aggregation of a large book launders most of the *per-claim* tail difference away.

What the mean-preserving reparameterisation fixed. An unadjusted MLE Burr XII fit inflates the aggregate mean, because the fitted mean is dominated by the tail and is not reconciled to the pricing model — that inflation, not any tail pathology, was the obstacle Section 3 originally reported. It vanishes the moment we hold the mean equal (`scale = mean_sev / B(c, d)`), leaving only the tail-shape effect the experiment is meant to isolate. The value of owning the convolution is then **explicit grid control and an auditable edge-mass diagnostic**: Cells 14.2–14.4 confirm the €104.86M span (`h=100`, `N=220`) holds even the heavier Burr tail without aliasing (read the printed edge mass — it should be ≈ 0). Had the tail wrapped, the diagnostic would have said so rather than returning a quietly wrong number.

Caveats, stated honestly. (i) The comparison is deliberately asymmetric: lognormal carries a *per-segment* empirical CV, Burr XII a single *pooled* shape rescaled per segment. A per-segment Burr fit — weighting claims by GMM posterior as `compute_empirical_cvs` does — would let the tail heaviness vary by segment and could widen the TVaR gap for the high-dispersion segments; it is also numerically delicate and is left as the natural refinement. (ii) The fitted shape sits at `cxd = 2.33` — finite variance, infinite skewness — so the Burr XII TVaR is well defined but its *third* moment is not; TVaR at still-higher confidence (99.5%, 99.9%) will be increasingly grid- and truncation-sensitive and should be read with the edge-mass diagnostic in view. (iii) Everything here is on the settled, fully-paid freMTPL2 book; as in Section 12, no claims-development tail is present.

Bottom line for the technical note. Pricing the AIC-optimal severity — on two independent engines — lets us *quantify* the answer instead of asserting it: on this portfolio the distribution choice is a **small effect on capital, concentrated in TVaR** (exact figure in `swap_comparison`), and lognormal's retention is vindicated not as a compatibility compromise but as a scale-appropriate modelling choice. The machinery now exists to re-run this on any book — and on a portfolio with fewer claims, or with per-segment Burr fits, the same code would reveal a larger effect if one exists.

Future Research

The most direct next step from the original GLM study — testing the Burt representation within a gradient boosting framework — has now been taken, and Section 11 reports the outcome. Rather than closing the enquiry, the result sharpens it: several deeper questions are surfaced precisely because the linear-versus-non-linear question is now settled.

1. Why is top-decile calibration partition-sensitive for the Burt representation?

The Burt top-decile ratio varied from 0.63 to 0.91 across two fold partitions, while the classical ratio held near 1.0. The working explanation — that Burt's smoother premium distribution gives its upper decile a less sharply defined boundary — is a hypothesis, not a finding. Whether this instability is intrinsic to the embedding's smoothness or an interaction with the non-linear learner, and whether it can be characterised analytically, is open.

2. Bottom-up versus top-down collective risk — the claims-development divergence.

Section 12 examines the bottom-up versus top-down question directly: a collective model fitted on portfolio aggregates is compared with the bottom-up FFT construction, with mean and frequency held equal. On this benchmark the two nearly coincide (99% TVaR within 0.15%), because the only divergence freMTPL2 can express is the segmentation-mixture effect — modest, directionally consistent, and ordered by representation geometry. What remains open is the divergence M . Denuit's remark actually targets: the gap arising from *open claims and provisioning*, which a settled, fully-paid benchmark cannot exhibit. Quantifying it requires a production portfolio with genuine claims development, where the bottom-up aggregate (built on paid amounts) and a directly-fitted collective model (reflecting reserves and IBNR) would be expected to part company materially. That comparison — ideally with a development triangle alongside the policy data — is the natural next step.

3. Reconciling severity fit with FFT moment constraints.

Burr XII and log-logistic fit the empirical severity materially better than lognormal by AIC. Section 14 prices the aggregate under a mean-preserving Burr XII on two independent engines and finds the effect on 99% capital small and concentrated in TVaR: at portfolio scale the distributional choice is nearly immaterial once the mean is held equal, which is why lognormal is retained as the transparent headline. The heavy-tail admissibility question that once seemed to bar Burr XII does not arise — finite variance suffices for convolution. What remains genuinely open is narrower and sits at the *extreme* tail: with $c \times d = 2.33$ the third moment is infinite, so the discretised tail becomes grid-sensitive beyond the 99% level. Whether the near-equivalence reported here persists at 99.5% and 99.9%, and whether tail-truncated or spliced severity models stabilise those extreme quantiles — together with a per-segment rather than pooled Burr fit, which would let tail heaviness vary across risk classes — is the natural continuation, and one well suited to the convolution machinery developed here.

4. Why does informational completeness survive non-linearity?

That the Burt-augmented design adds nothing over the Burt-replacing design under a *tree* — a model free to exploit both representations — is more surprising than the same result under a GLM. It suggests the Burt embedding is a sufficient statistic for the categorical structure in a stronger sense than linear summary alone would require. Formalising the conditions under which this completeness holds would be a theoretical contribution beyond the empirical observation.

5. The fully non-linear pipeline.

The present extension swaps only the final pricing layer; the frequency and severity enrichment remain GLM-based, isolating the experiment to a single change. A natural successor replaces the frequency model with a gradient booster as well, with out-of-fold prediction of the enrichment features to avoid leakage into the segmentation. This is a separate study rather than a complication of the present one.

6. Formal robustness across partitions.

The stability reported here rests on two fold seeds, sufficient as a sanity check. A formal robustness claim would run the comparison across ten to twenty partitions and report the Gini gap and the reversal magnitudes as mean and spread rather than single values.

The linear-versus-non-linear question is answered. What remains are deeper questions about why the representation behaves as it does, and how to close the gap between distributional fit and computational tractability.

In []: